

MAGBO STUDIO

MAGBO Access Control

Le livre du système

LYCÉE MOLIÈRE · RIO DE JANEIRO

Sammy K. Magbo

2026

MAGBO Access Control

Le livre du système

Sammy K. Magbo

LYCÉE MOLIÈRE · RIO DE JANEIRO

2026

À composer par Sam.

COMMENT LIRE CE LIVRE

Chaque affirmation technique est vérifiable dans le dépôt : quand un fichier est cité, c'est qu'il dit ce qui est écrit. Ce qui n'a pas pu être vérifié porte [À VÉRIFIER] avec la commande ou la requête qui le confirmerait. Ce que seul Sam sait porte [À COMPLÉTER PAR SAM] avec la question précise.

Une documentation fausse est pire qu'absente, parce qu'on lui fait confiance. Si vous trouvez une affirmation que le dépôt contredit, corrigez-la dans `docs/livre/` et régénérez le livre.

L'état opérationnel du jour ne vit pas ici : il vit dans `docs/operacional/handoff.md`. C'est le document à ouvrir en premier en cas de problème ; celui-ci explique *pourquoi* le système est ce qu'il est.

TABLE DES MATIÈRES

Chapitre 1 — Vue d'ensemble	1
Chapitre 2 — Les points de passage	13
Chapitre 3 — Les règles métier	20
Chapitre 4 — Guide de l'opérateur	38
Chapitre 5 — Administration	44
Chapitre 6 — Exploitation	60
Chapitre 7 — Architecture technique	76
Chapitre 8 — Les leçons du projet	83
Chapitre 9 — Ce qui reste	91

Le livre du système — MAGBO Access Control

Lycée Molière, Rio de Janeiro. Écrit le 29/08/2026, contre l'état du dépôt au commit `7c4d54e`.

Ce livre existe parce que Sam — le propriétaire et unique développeur du système — est parti. Il s'adresse à quelqu'un qui n'a jamais vu MAGBO tourner et qui ne peut poser de question à personne.

⚠ Comment lire ce livre

Chaque affirmation technique est vérifiable dans le dépôt. Quand un fichier est cité, c'est qu'il dit ce qui est écrit. Deux marqueurs signalent les limites :

Marqueur	Ce qu'il veut dire
[À VÉRIFIER]	je n'ai pas pu le confirmer depuis le dépôt — suivi de la commande ou la requête qui le tranche
[À COMPLÉTER PAR SAM]	seul Sam le savait — suivi de la question précise
[CAPTURE : ...]	une capture d'écran manque à cet endroit

Une documentation fausse est pire qu'absente, parce qu'on lui fait confiance. Si vous trouvez une affirmation que le dépôt contredit, corrigez-la dans `docs/livre/` et régénérez le HTML.

Par où commencer

Il y a un problème maintenant → `docs/operacional/handoff.md`. C'est l'état opérationnel du jour : le défaut ouvert en tête, les cinq gestes d'urgence, le rite de déploiement. Ce livre explique le système ; le handoff le fait tourner.

Vous reprenez le système → chapitres 1, 2, 6, puis 9.

Vous reprenez le code → chapitres 1, 3, 7, 8.

Vous formez quelqu'un → chapitres 1, 4, 5.

Les neuf chapitres

#	Chapitre	Ce qu'il répond
1	Vue d'ensemble	Ce que le système fait, ce qu'il n'est pas , et le chemin d'un passage de la porte à l'écran

#	Chapitre	Ce qu'il répond
2	Les points de passage	Portail, CDI, cantine, infirmerie : le matériel, les règles propres, les pièges — et le défaut ouvert du portail
3	Les règles métier	Droits repas, créneaux, flags, exclusions, régimes, PPMS, déduplication, permissions — et à quelle heure chaque règle est jugée
4	Guide de l'opérateur	Par profil : ce que chaque alerte veut dire, et quoi faire
5	Administration	Opérateurs et permissions, imports et leurs pièges, photos, planning, écran de configuration
6	Exploitation	La VM, le <code>.env</code> , le rite de déploiement, les migrations, la sauvegarde, le portable, et le tableau symptôme → geste
7	Architecture technique	Backend, frontend, base, tests — et pourquoi ils sont faits comme ça
8	Les leçons du projet	Chaque défaut qui a coûté quelque chose : contexte → symptôme → cause → correction → règle
9	Ce qui reste	Liste datée et priorisée de ce qui est ouvert

Les trois choses à savoir avant tout le reste

Si vous ne lisez qu'une page, lisez celle-ci.

1. ⚠ Le système n'ouvre et ne ferme aucune porte. Le webhook est post-événement : quand MAGBO apprend qu'une personne est passée, elle est déjà passée. Quand un écran dit « refusé », c'est une décision **logique**, écrite pour être lue. Ce n'est pas une lacune à combler : c'est ADR-003 ([../architecture/decisoes/ADR-003-webhook-pos-evento.md](#)), et la moitié du code en dépend.

2. ⚠ « Je n'ai pas vu » n'est pas « il n'était pas là ». `PENDING`, `INCONNU`, « aucun passage vu », un compteur à `—` : ce sont des états qui disent *je ne sais pas*. Les lire comme des refus fait accuser quelqu'un à la place d'une case vide. C'est la leçon 13 du chapitre 8, et elle revient partout.

3. ⚠ Il y a toujours deux heures. Celle où la chose est arrivée, celle où le programme l'a apprise. Elles sont identiques — jusqu'au jour où une file se vide, où un conteneur démarre en UTC, ou où il est 21 h à Rio. Ce piège a mordu **quatre fois**. Il ouvre le chapitre 8.

Le livre imprimable

Deux commandes, et la seconde est celle qui compte pour l'imprimeur.

```
node scripts/build-livre.js      # le HTML, depuis docs/livre/*.md
node scripts/paginer-livre.js    # les vrais numéros de page + le PDF
```

La première produit `docs/livre/livre-complet.html` : tous les chapitres, un seul fichier autonome (aucun script, aucune requête réseau — il s'ouvre depuis une clé USB sur un poste hors ligne).

La seconde produit `docs/livre/livre-complet.pdf`, prêt à relier. Elle fait deux choses qu'une feuille de style ne sait pas faire :

1. **Elle numérote la table des matières.** En CSS d'impression cela s'écrit `target-counter()`, que Chrome ne connaît pas — mesuré : le parseur jette la déclaration entière, on perd le numéro *et* le texte qui l'accompagne. La seule méthode exacte est de poser la question au PDF : chaque chapitre ouvre sur une page neuve, donc on imprime le livre arrêté juste avant le chapitre *k*, on compte les pages, et on sait. Les numéros mesurés vivent dans `docs/livre/pagination.json` ; sans ce fichier, la table des matières renvoie aux chapitres **sans numéro** — dégradé, jamais faux.
2. **Elle refuse un livre réduit.** Quand un élément dépasse la largeur imprimable, Chrome ne le coupe pas et n'avertit pas : il réduit *tout le document*. Le livre sortait ainsi à 80,7 % — un corps déclaré à 10,5 pt imprimé à 8,5 pt, sur les 84 pages, et rien nulle part pour le dire. Le script mesure le facteur d'échelle dans le PDF et s'arrête s'il n'est pas exactement `0.750000`, en nommant les éléments qui débordent.

À la main, sans les scripts : ouvrir le HTML dans un navigateur, `Ctrl+P`, A4, marges « Par défaut », cocher « Graphiques d'arrière-plan », et « Taille réelle » — jamais « Ajuster à la page », qui détruirait les marges de reliure.

⚠ Le style d'impression porte `print-color-adjust: exact` — sans cette ligne, le navigateur **jette les fonds colorés** et le livre sort en gris. C'est la même ligne qui fait la couleur de l'affiche cantine ; la leçon a déjà été payée une fois.

Ce que ce livre ne remplace pas

Document	Ce qu'il porte
<code>docs/operacional/handoff.md</code>	l'état du jour — à ouvrir en premier en cas de problème
<code>docs/manual-utilisateur.md</code>	chaque écran, bouton par bouton
<code>docs/operacional/reconstruire-do-zero.md</code>	reconstruire et restaurer, commandes exactes
<code>docs/operacional/guide-installation-postes.md</code>	installer un poste
<code>deploy/migrations/README.md</code>	chaque migration, une par une
<code>docs/architecture/decisoies/</code>	pourquoi chaque décision structurelle a été prise
<code>.claude/rules/</code>	les pièges par domaine, pour qui écrit du code

⚠ Ce que la mise en page n'a pas encore — mesuré le 03/09/2026

Ces quatre points ont été **mesurés**, et délibérément **non corrigés** : ils changent l'objet physique, et cela se décide avant d'être imprimé, pas après.

1. Trois chapitres sur neuf s'ouvrent encore sur un VERSO (une page de gauche) : les chapitres **2, 8 et 9**, aux feuilles 24, 94 et 102. Les six autres, chapitre 1 compris, s'ouvrent bien au recto.

Ce qui gouverne cela est la **parité** : les liminaires occupent **douze** pages, un nombre pair, donc le folio 1 tombe sur la feuille 13, une page de droite. Quand elles étaient **onze**, c'était l'inverse — six chapitres au verso, et tous les folios impairs à gauche. Cette page-ci a fait passer les liminaires de onze à douze, et a donc corrigé six ouvertures sur neuf **par accident**. ⚠ Ce qui est fragile est exactement cela : la parité tient à la longueur d'un texte, et personne ne le saura en le modifiant.

`break-before: right`, qui réglerait tout en une ligne, **ne fonctionne pas dans Chrome** (vérifié : 4 pages produites au lieu de 6, dans les deux orthographes). *Le correctif* : `scripts/paginer-livre.js` doit **calculer** les pages blanches — il connaît déjà la parité de chaque début de chapitre — et les faire écrire par `build-livre.js`, avec une passe de convergence, puisque insérer une page blanche déplace tous les chapitres suivants. C'est un petit chantier, pas un réglage, et c'est pour cela qu'il n'a pas été fait dans la précipitation d'un dernier jour.

2. Le PDF porte dix-neuf liens `file:///C:/Users/...`, fabriqués par Chrome qui absolutise les liens relatifs en annotations. Le texte imprimé, lui, est juste. Ces annotations sont mortes pour tout lecteur et **font voyager un nom de compte Windows** jusque chez l'imprimeur. *Le correctif* : imprimer le PDF final depuis une copie où les `href` qui ne commencent ni par `#` ni par `http` ont été retirés — les ancrs internes du sommaire, elles, doivent survivre.

3. « L'architecture en une page » tient sur deux pages (folios 2 et 3) : le titre ne garde que cinq lignes d'un bloc d'une soixantaine, et la figure se lit à cheval sur un pli. `pre.code { break-inside: auto }` est le bon choix pour les blocs longs — mais celui-là est une **figure**, pas du code à dérouler.

4. Cette page-ci n'a ni folio ni entrée au sommaire : la section `00` est traitée comme une liminaire, alors qu'elle porte du contenu véritable — celui que vous lisez. Personne ne peut le citer par un numéro de page. Le colophon annonce d'ailleurs « neuf chapitres, plus le sommaire », alors qu'il y a bien une dixième section.

⚠ **Et une limite qui vaut pour tout ce livre** : chaque nombre ci-dessus a été mesuré avec **Chrome 152.0.7977.75**. `trouverChrome()` accepte Edge ou Chromium en remplacement **sans contrôler la version**. Une autre machine peut donner un autre nombre de pages pour le même fichier — c'est pourquoi les numéros du sommaire portent une empreinte et se refusent quand elle ne colle plus, plutôt que d'être crus sur parole.

Chapitre 1 — Vue d'ensemble

Ce chapitre s'adresse à la personne qui reprend MAGBO sans avoir jamais vu le système tourner et sans pouvoir poser de question à personne. Il répond à trois questions, dans cet ordre : **ce que le système n'est pas, ce qu'il fait, et par où passe un passage** — de la porte jusqu'à l'écran de l'opérateur. Les chapitres suivants entrent dans le détail ; celui-ci donne la carte.

Convention du livre : `[A VERIFIER]` marque une affirmation que je n'ai pas pu confirmer dans le dépôt, suivie de la commande ou de la requête qui la tranche. `[A COMPLETER PAR SAM]` marque ce que seul Sam savait.

1. ⚠ Avant tout : ce que MAGBO n'est PAS

MAGBO n'ouvre aucune porte. MAGBO ne ferme aucune porte.

Ce n'est pas une limite technique qu'on finira par lever : c'est une décision d'architecture, écrite, prouvée avec du matériel, et sur laquelle repose la moitié du code. Elle porte un nom : **ADR-003 — le webhook est post-événement** (`docs/architecture/decisoes/ADR-003-webhook-pos-evento.md`).

La preuve tient en une séquence d'événements observée sur le terminal de banc le 13/07/2026 : `21` (la porte s'ouvre) → `75 / 1` (l'authentification) → `22` (la porte se ferme). **Quand la requête HTTP arrive chez nous, la porte a déjà fonctionné.** Le test CANT-09 l'a confirmé dans l'autre sens : validité d'une personne placée dans le passé, le terminal a refusé **à la voix** avant tout HTTP, et l'événement `subEventType=8` est arrivé ensuite. L'appareil ignore purement et simplement la réponse du backend — c'est pour cela que le webhook renvoie `200` sur tous les chemins normaux, y compris quand il refuse logiquement (`HikvisionWebhookController`, méthode `handleEvent`).

Donc, quand le code écrit `DENIED`, cela veut dire : *classification logique et audit*. Jamais : *action physique*.

Conséquences qu'il faut avoir comprises avant de lire le reste :

- Un élève sans droit au repas, ou sans autorisation de sortie, **passe physiquement**. MAGBO enregistre la tentative et la montre à l'écran. La direction de l'école le sait ; c'est écrit dans les Conséquences de l'ADR-003.
- Cet écart a un nom et un chiffre : le KPI `divergenciaHoje` (`auth_result=SUCCESS` **ET** `authorization_result=DENIED` — « la porte s'est ouverte, mais MAGBO ne l'a pas comptée comme un accès valide »).
- **Effet de bord voulu** : si la VM ou le réseau tombe, le déjeuner et le portail **continuent de fonctionner**. Le terminal authentifie localement avec les identités déjà distribuées, met les événements en file, et les renvoie quand le serveur revient. L'observationnalité est une **propriété désirée**, pas un trou.

La seule exception documentée : la cantine

Elle ne contredit rien : elle nomme *qui* applique la règle. **ADR-004 — blocage opérationnel assisté**

(docs/architecture/decisoes/ADR-004-bloqueo-operacional-assistido.md) :

Qui	Valide quoi
Le terminal	l' identité (visage / carte)
MAGBO	la règle (droit au repas, créneau, doublon)
L'opérateur, à la main	l' exception

Il n'y a **pas** de blocage physique de cantine via HikCentral — ni aujourd'hui, ni dans la feuille de route. Les alternatives ont été examinées et rejetées par écrit : synchroniser les droits en *access levels* HikCentral (propagation lente, échecs partiels par personne, rollback coûteux — incompatible avec une file d'attente de midi), et piloter le relais depuis le backend (impossible, ADR-003).

Ce qui en découle directement : **le flux des tentatives refusées est une pièce critique, pas un gadget**. C'est par lui que l'opérateur voit, en temps réel, qui a été refusé et pourquoi

(js/components/DeniedAttemptsFeed.js , embarqué dans js/components/CantineMonitor.js). Sans opérateur devant l'écran, la règle de cantine n'existe pas. Le manuel le dit à l'utilisateur en une phrase : « MAGBO ne verrouille aucune porte. Celui qui empêche physiquement un élève de passer, c'est **vous** » (docs/manual-utilisateur.md §0.1).

2. Ce que MAGBO fait

Quatre verbes, et rien d'autre :

1. **Identifie** — relie un événement d'appareil à une personne du fichier (app_users.id = matricule Pronote, chaîne de caractères, zéros de tête conservés).
2. **Classe** — applique les règles de l'endroit (cantine, portail, CDI, infirmerie) et tranche : accès effectif, observation, ou refus.
3. **Enregistre** — dans deux tables distinctes qui ne se mélangent jamais (§4).
4. **Signale** — affiche à l'opérateur, compte pour la direction, et laisse une trace consultable après coup.

3. L'architecture en une page

```

APPAREILS                                RESEAU DE L'ECOLE                        SERVEUR (VM Ubuntu)

[ Terminal MinMoe DS-K1T344MX ]
  visage -> subEventType 75
  carte  -> subEventType 1
  IL OUVRE LA PORTE LUI-MEME
      |
      | multipart/form-data, part "AccessControllerEvent" (JSON)
      |
[ Camera DeepinView iDS-2CD7A46G2 ]
  COMPARE un visage a sa bibliotheque faciale ; n'ouvre rien

```

```

|
| multipart/form-data, part "alarmResult" (JSON)
v
+-----+
| " Ecoute HTTP " -- configuree DANS L'APPAREIL : |
| IP du serveur, port 8080, URL avec le token, HTTP |
+-----+
|
| POST /api/hikvision/webhook (token en en-tete ou ?token=)
| POST /api/hikvision/webhook/t/{token} (token dans le CHEMIN -- cameras)
v
+-----+
| HikvisionWebhookController |
| 1. token, MessageDigest.isEqual (503 si non configure, 401 sinon) |
| 2. parsePayload : multipart tolerant, ou JSON pur |
| 3. dedup d'INGESTION (le meme paquet renvoie) -> 200, rien en base |
| 4. heartbeat / evenement sans personne -> 200, fin |
+-----+
|
v
+-----+
| EventTimeResolver -- QUELLE HEURE part en base |
| dateTime du payload -> instant -> America/Sao_Paulo |
| 3 garde-fous -> repli sur l'heure de reception, + 1 ligne INFO |
+-----+
|
v
+-----+
| AccessDecisionService (@Transactional -- LE chef d'orchestre) |
| classification du sous-type -> door_mappings -> personne -> actif |
| cantine : dedup repas -> droit au repas -> creneau |
| portail : permission de sortie -> regime de sortie (observation) |
| commun : meme passage (30 s) -> poste fixe -> presence ouverte |
+-----+
|
|
AUTORISE | REFUSE |
v v
+-----+ +-----+
| access_logs | | access_attempts |
| ce qui EST un | | ce qui a ete TENTE |
| passage effectif | | et n'est pas passe |
+-----+ +-----+
\ /
\ PostgreSQL 16 -- magbodb /
+-----+
|
v API REST, JWT 8 h, roles + secteurs
+-----+
| 27 controleurs sous /api/... |
+-----+
|
v HTTP depuis chaque poste
[ Portail ] [ Cantine ] [ CDI ] [ Infirmerie ] [ Direction ]
Electron + React (sans bundler) ; le raccourci fixe MAGBO_API_URL
et MAGBO_SECTOR -- ouvrir le .exe seul donne une application VIDE

```

Maillon par maillon

1. Les appareils. Deux familles, deux formats, un seul webhook. Les terminaux **MinMoe** (DS-K1T344MX , firmware V4.13.0) authentifient localement et envoient un `multipart/form-data` dont la part utile s'appelle `AccessControllerEvent` . Les **caméras DeepinView** de la portaria envoient un format à elles, part `alarmResult` , qui contient le résultat d'une *comparaison* faciale — pas d'une authentification. Tout ce qui a été mesuré sur ces payloads (la table des sous-types, le `score` emballé dans `{"value": 52}` , la translittération des accents, la troncature du nom à 32 caractères) est dans `.claude/rules/hikvision.md` : c'est le document à lire avant de toucher au parseur.

2. L'« Écoute HTTP ». Ce n'est pas du code : c'est un réglage **dans l'appareil** (`Configuration` → `Réseau` → `Service réseau` → `Écoute HTTP`) qui contient l'IP du serveur, le port et le token. ⚠ **C'est le maillon le plus fragile de la chaîne, et il casse en silence** : une IP qui change par DHCP n'émet aucune erreur, les événements cessent simplement d'arriver. Vérifier ce réglage fait partie de toute session matériel (`.claude/rules/hikvision.md`). Les caméras DeepinView ont exigé une route à part, `/webhook/t/{token}` : elles jettent la *query string* et ne savent pas envoyer d'en-tête personnalisé — prouvé au tcpdump le 28/07/2026, l'appareil renvoyant en boucle ~1 req/s de 401.

3. Le webhook.

`backend/src/main/java/com/magbo/access/controllers/HikvisionWebhookController.java` (652 lignes). Il fait quatre choses avant toute décision : valider le token (`MessageDigest.isEqual` , **deny-by-default** — token absent de la configuration = 503), extraire le JSON de la bonne part, écarter les paquets déjà vus (dedup d'ingestion, 60 s, clé IP + `serialNo`), et écarter les événements sans personne (heartbeat toutes les ~30 s, ouverture/fermeture de porte). ⚠ **Tout rejet laisse une ligne INFO** avec l'IP et le `serialNo` : le niveau du paquet est INFO en production, donc un `log.debug` disparaîtrait du fichier, et « un passage qui disparaît sans trace » est la pire panne possible ici — le commentaire du code l'explique en dix lignes, elles valent la lecture.

4. `EventTimeResolver` .

`backend/src/main/java/com/magbo/access/services/EventTimeResolver.java` (117 lignes). Il répond à une seule question : **quelle heure va dans la base**. Réponse : l'heure de l'événement (`dateTime` du payload, ISO 8601 avec décalage — converti en *instant*, puis en `America/Sao_Paulo`), jamais l'heure de réception. Il existe à cause d'un incident daté : le 03/08/2026, un terminal a vidé sa file hors-ligne — 33 événements en deux minutes, à 14 h 51, de passages survenus des heures plus tôt — et les 33 sont entrés comme s'ils avaient eu lieu à 14 h 51. Les rapports ont affiché des **durées moyennes négatives**. Trois garde-fous font retomber sur l'heure de réception, chacun avec une ligne INFO : `dateTime` absent ou illisible, horloge de l'appareil en avance de plus de **5 min**, événement plus vieux que **30 jours**.

⚠ **Le piège à retenir de ce fichier** : l'heure du *registre* et l'heure des *règles* sont deux horloges différentes. `eventTime` va dans la colonne `timestamp` ; les règles (créneau de cantine, dedup de repas, permission de sortie) sont évaluées contre `now` , l'heure de la décision — pour qu'une file hors-ligne ne change pas rétroactivement un ALLOW en DENY. Deux règles font exception **volontairement**, parce qu'elles *décrivent* au lieu de *refuser* : le créneau de cantine (`MealSlotService`) et le **régime de sortie** (`RegimeSortieService`). Le javadoc de `AccessDecisionService.process` et les commentaires ⚠⚠ dans le corps de la méthode expliquent chacune, avec le nom du test qui les verrouille.

5. AccessDecisionService .

backend/src/main/java/com/magbo/access/services/AccessDecisionService.java (868 lignes).

C'est la seule classe qui connaît l'ordre des règles — et c'est sa raison d'être. Elle est

@Transactional , elle s'appuie sur une dizaine de services spécialisés (HikvisionEventClassifier , DeduplicationService , MealEntitlementService , MealsSlotService , ExitPermissionService , RegimeSortieService , SamePassageService , PostoFixoService , PresencaAbertaService , AccessAttemptService), et les deux branches d'entrée — terminaux et caméras — convergent vers la

même méthode finale (registrarPassagem , via le record Passagem) précisément pour que l'ordre des règles n'existe pas en double. L'ordre exact, secteur par secteur, est dans

docs/architecture/fluxos.md §4 à §6.

6. PostgreSQL 16. Base magbodb , utilisateur magbo , conteneur magbo-postgres . Le schéma est créé par Hibernate (ddl-auto=update) **plus** 26 migrations SQL versionnées appliquées par-dessus (deploy/migrations/). ⚠ ddl-auto=update **ajoute** mais ne retire ni ne relâche jamais rien : c'est la raison pour laquelle certaines contraintes CHECK doivent être élargies à la main par une migration, et pourquoi elles échouent **uniquement sur la VM** quand on l'oublie (voir .claude/rules/database.md , la note sur denial_reason).

7. L'API REST. 27 contrôleurs sous /api/... , JWT valable 8 h, rôles ADMIN / OPERATOR , plus une autorisation par **secteur** (@areaSecurity.can('cantine')) et dix permissions granulaires (backend/src/main/java/com/magbo/access/security/Permissions.java , liste TODAS). Seules six routes sont publiques : /api/auth/login , /api/auth/password-reset-request , /api/health , les deux webhooks Hikvision, /h2-console/** et /error (security/SecurityConfig.java , lignes 39-74 — le commentaire sur /error vaut la lecture : son absence faisait mentir toutes les erreurs serveur au front, une importation ratée disait « Session expirée »). Le catalogue des routes est dans docs/architecture/endpoints.md .

8. Les postes Electron. Une application Electron (main.js , preload.js , index.html) qui charge React 18 + Babel + Tailwind **en local**, sans bundler ; tout est vendorisé dans libs/ . ⚠ **Aucun CDN** : un <script src="https://..."> ajouté ne produit pas d'erreur — l'écran ne s'affiche simplement pas en mode kiosque hors ligne. Se vérifie en une commande : grep -cE 'src="https?://|cdn\.|unpkg|jsdelivr' index.html → 0. Chaque poste est configuré par des variables passées par le raccourci : MAGBO_API_URL et MAGBO_SECTOR (lues par js/utils/posteConfig.js , exposées au front par preload.js sous window.magboConfig , qui publie apiUrl , sector , source , doitConfigurer , isProduction , cheminFichier , version), plus NODE_ENV=production (main.js:33) qui arme le mode kiosque — à condition que le poste soit déjà réglé (posteConfig.verrouillable).

⚠ **MAGBO_KIOSK_PIN n'est PAS exposée au front, et il n'existe aucune sortie du kiosque par code.**

La variable est bien lue (main.js:32) et preload.js:112-123 publie un second pont,

window.magboIpc , avec verifyKioskPin , exitKiosk et onRequestAdminPin ; main.js:383

enregistre Ctrl+Shift+Alt+Q et émet request-admin-pin . Mais **aucun fichier de js/ ,**

d' index.html ni de tests/ ne consomme ce pont (mesuré le 03/09/2026 : grep -rn

'magboIpc\MAGBO_KIOSK_PIN' js/ index.html tests/ ne rend rien). Un webContents.send vers un canal sans écouteur ne lève rien et ne journalise rien : le raccourci est un **no-op silencieux**, et celui qui le tape ne peut pas distinguer « l'application est plantée » de « ce geste n'a jamais rien fait ». On ferme

un poste verrouillé par Ctrl+Alt+Suppr → Gestionnaire des tâches. ⚠ Ne pas confondre avec

`ADMIN_PIN`, qui est le PIN du **backend** (`/api/admin/verify`, `js/components/AdminPinModal.js`) : celui-là fonctionne et reste obligatoire. Voir ADR-007. Ouvrir le `.exe` directement donne une application **vide, sans message** — c'est la panne la plus fréquente signalée dans le manuel, et ce n'en est pas une.

4. Les deux tables, et pourquoi elles sont deux

C'est l'invariant structurel du système. **ADR-001** (`docs/architecture/decisoes/ADR-001-attempts-vs-logs.md`) :

Table	Contient	Conséquence
<code>access_logs</code>	uniquement l'accès effectif et autorisé	être dans cette table signifie déjà « réussi » — il n'y a pas de colonne <code>granted</code>
<code>access_attempts</code>	tout ce qui a été tenté et n'est pas passé	4 axes : méthode / résultat de l'appareil / décision MAGBO / motif

L'alternative — un drapeau `granted` dans `access_logs` et un filtre dans chaque requête — a été rejetée pour une raison à garder en tête à chaque requête nouvelle : plus de quinze requêtes dérivent de `access_logs` (présence, repas, occupation, KPIs), et **un seul oubli de filtre** aurait compté un refus comme une présence, **en silence**. La contrainte est devenue une propriété du schéma au lieu d'une discipline de rédaction.

Un cas grave écarté par cette séparation : avant la Phase B, un `subEventType=8` (refus du terminal) devenait un `access_log` valide — donc un **repas fictif**.

Cas particulier voulu : le mode `OBSERVATION` écrit **dans les deux tables** (le passage réel + une trace d'audit). C'est ainsi qu'on mesure une règle sans l'appliquer.

5. Les points en service aujourd'hui

Le mapping point → aire vit **en deux endroits qu'il faut modifier ensemble** :

`backend/src/main/java/com/magbo/access/config/AreaMapping.java` et `js/data/constants.js` (`ACCESS_POINTS`).

Point	Aire	Lieu
<code>PORT1</code> , <code>PORT2</code> , <code>PORT3</code>	<code>portail</code>	portail principal, portail terrain, garage
<code>BIBLIO</code>	<code>cdi</code>	CDI / bibliothèque
<code>ENFERM</code>	<code>infirmierie</code>	infirmierie
<code>REFEI1</code> , <code>REFEI2</code>	<code>cantine</code>	réfectoires 1 et 2

S'y ajoutent des **points virtuels** qui ne sont pas des portes mais des écrans : `CANTINA_MONITOR`, `CANTINA_REPORT`, `INFIRMARY_REPORT`, `GENERAL_REPORT`, `PPMS`, et les écrans de gestion (`MEAL_ENTITLEMENT_MANAGEMENT`, `EXIT_PERMISSION_MANAGEMENT`, `REGIME_MANAGEMENT`, `MEAL_SLOT_MANAGEMENT`, `CDI_EXCLUSION_MANAGEMENT`).

⚠ Une distinction que le code fait et qu'il faut connaître : `AreaMapping.temPresencaConfiavel()` répond `false` pour le **portail** et `true` partout ailleurs. Au portail, « la personne est encore dedans » n'est pas une affirmation fiable : on sort par ailleurs, hors du champ de la caméra, ou collé à quelqu'un d'autre. Deux règles en dépendent, et le javadoc de cette méthode explique pourquoi mieux que n'importe quel résumé.

Les appareils physiques

Source : Sam, le 28/08/2026. Non vérifiable dans le dépôt — la table `door_mappings` vit dans la base de production, pas dans le code.

Appareil	Rôle annoncé
<code>.167</code>	portail — ENTRÉE (caméra DeepinView)
<code>.166</code>	portail — SORTIE (caméra DeepinView)
<code>.15</code> , <code>.16</code>	CDI
<code>.10</code> , <code>.12</code> , <code>.13</code> , <code>.14</code>	cantine

⚠ Trois réserves, toutes importantes :

1. Sam annonce « six terminaux » et énumère **huit** adresses. La lecture la plus probable est six terminaux MinMoe (CDI + cantine) **plus** deux caméras au portail, mais je ne l'ai pas vérifiée.
2. Les adresses sont données en dernier octet seul. Le serveur est en `192.168.1.253` (`docs/operacional/guide-installation-postes.md`, §Avant de commencer), l'ancien banc d'essai était en `172.20.40.x` : **le préfixe réel des appareils n'est écrit nulle part dans le dépôt.**
3. La caméra `.166` (SORTIE) est en **panne physique connue** depuis au moins le 27/08 — c'est écrit noir sur blanc dans `docs/operacional/diagnostic-portaria-2026-08-27.md`, §1 et §3. Toute lecture des chiffres du portail doit **isoler ce point**, sinon la panne masque tout le reste : une chute qui ne touche que les `SAIDA`, c'est elle, et rien d'autre à chercher.

[A VERIFIER] L'inventaire réel des points actifs, avec le sens de chaque appareil, se lit en une requête sur la VM :

```
docker exec magbo-postgres psql -U magbo -d magbodb -tAc \
"SELECT terminal_ip, door_no, reader_no, point_id, action, label, ativo
FROM door_mappings ORDER BY terminal_ip, door_no, reader_no;"
```

[A VERIFIER] Quels points reçoivent réellement du trafic, sur 21 jours :

```
docker exec magbo-postgres psql -U magbo -d magbodb -tAc \
"SELECT point_id, action, count(*) FROM access_logs
WHERE timestamp >= current_date - 21 GROUP BY 1,2 ORDER BY 1,2;"
```

⚠ **Avant toute session matériel** : une IP qui change par DHCP casse à la fois l'« Écoute HTTP » de l'appareil et son `door_mappings`, **sans aucune erreur**. C'est arrivé le 16/07/2026 (terminal `.12` → `.10`, `docs/operacional/procedimento-hikcentral.md`). La checklist complète est dans `.claude/rules/hikvision.md`.

[A COMPLETER PAR SAM] Quel est le préfixe réseau des appareils Hikvision — `192.168.1.x` comme le serveur, ou un autre VLAN ? Et la réservation DHCP demandée au SI par Fabiano a-t-elle été mise en place, oui ou non ?

[A COMPLETER PAR SAM] La caméra `.166` : la panne a-t-elle été signalée, à qui, et qui est censé la réparer ?

CAPTURE D'ÉCRAN ATTENDUE

la page « Écoute HTTP » d'un terminal, montrant l'IP du serveur, le port 8080 et l'URL avec le token — c'est l'écran à comparer quand les événements cessent d'arriver

6. Qui utilise le système

Six métiers, six usages. Le manuel est organisé exactement ainsi : il contient une section « Par où commencer selon votre rôle » — c'est la porte d'entrée à donner à un utilisateur (`docs/manual-utilisateur.md`, lignes 100-155).

Rôle	Ce qu'il fait avec MAGBO	Écran principal
Vie Scolaire (portail)	suit les mouvements du jour, enregistre un passage à la main, autorise une sortie ponctuelle, cherche où est un élève	poste <code>PORT1</code> , écrans Sorties et Rapport Général
Cantine	garde l'écran ouvert pendant le service, voit les refus en direct, gère les droits au repas et le planning	Monitor Cantine, Droits Repas, Planning Cantine
CDI	fait pointer les élèves, sort les statistiques, fait l'appel en cas d'urgence	module CDI (<code>js/cdi/</code>), point <code>BIBLIO</code>
Infirmierie	enregistre arrivée et départ, édite le relevé des visites et des séjours longs	poste <code>ENFERM</code> , Rapport Infirmierie
Direction	trois chiffres et un rapport ; crée et désactive les comptes opérateurs	Rapport Général, Panneau administrateur
Portaria	le passage physique au portail	(pas d'écran propre — voir ci-dessous)

Deux remarques qui évitent des malentendus :

- **La « portaria » n'a pas d'écran à elle**. C'est le lieu, servi par les deux caméras et par le poste de la Vie Scolaire. Le mot survit dans le code comme *catégorie* d'affichage (`category: 'portaria'` dans `js/data/constants.js`) et comme *département* d'une personne — le département **suggère** un poste fixe dans l'écran Personnels, il ne le décide jamais (`.claude/rules/frontend.md`).

- **Le PPMS** (`js/components/PpmsView.js`, l'écran « Qui est à l'intérieur ») traverse tous les rôles : il est délibérément **non caché** dans la liste des points, parce que chercher son chemin au milieu d'une évacuation équivaut à ne pas avoir l'écran. Il est en revanche restreint par la permission `PPMS_READ` : la liste est **nominative**, et Sam a tranché le 14/08 « restreindre, pas fermer » (javadoc de `Permissions.PPMS_READ`). **⚠ Cet écran ne remplace pas l'appel** — il le dit lui-même, au-dessus du nombre.

7. Ce qui tourne, et ce qui dort

Toutes les règles de décision sont des **propriétés**, modifiables sans recompiler. Valeurs par défaut dans `backend/src/main/java/com/magbo/access/config/PolicyProperties.java` ; ce que la **production** applique réellement est dans `backend/src/main/resources/application-prod.properties`.

Politique	Défaut Java	Production
<code>meal-not-entitled</code>	DENY	DENY (l. 71)
<code>meal-pending</code>	OBSERVATION	⚠ DENY (l. 72)
<code>outside-meal-time</code>	OBSERVATION	OBSERVATION (l. 73)
<code>meal-slot-not-configured</code>	OBSERVATION	— (défaut)
<code>duplicate-meal</code>	OBSERVATION	OBSERVATION (l. 74)
<code>exit-not-authorized</code>	DENY	DENY (l. 75)
<code>user-inactive</code>	DENY	DENY (l. 76)
<code>missing-door-mapping</code>	FALLBACK	FALLBACK (l. 79)

⚠ **La ligne qui compte** : `meal-pending=DENY` en production. « En attente » n'est pas une décision, c'est **une case vide** — un élève dont personne n'a renseigné le droit. En production, cette case vide **refuse**. Le prérequis opérationnel est donc absolu : *la liste des autorisés doit être importée avant le jour 1 du service* (décision D5, ADR-004 ; répété dans `docs/manual-utilisateur.md` §A.1). Le mode `dev` garde `OBSERVATION`, ce qui signifie qu'un comportement observé en développement **ne prouve rien** sur ce que fera la production.

Ce qui dort, et c'est voulu :

- **Le régime de sortie** — le droit annuel déclaré par écrit par les responsables légaux (circulaire n° 96-248) — est **désactivé** : `magbo.regime.habilitado=false` (`application.properties` l. 248, et le champ Java `RegimeProperties.habilitado` l. 90 : le défaut est `false` **dans les deux endroits**, délibérément — deux sources pour un même défaut, et c'est celle de la documentation qui ne valait pas). Tant qu'il est éteint, `RegimeSortieService` répond `NON_APPLICABLE` pour tout le monde (l. 129). Quand il sera allumé : `magbo.regime.desconhecido=OBSERVATION`, parce qu'au jour 1 **aucun des 923 élèves n'a de régime saisi** et qu'un `DENY` peindrait l'école entière en rouge — exactement l'erreur déjà documentée pour `meal-pending`.
- ⚠ **Avant d'allumer** `magbo.regime.habilitado` **sur la VM, appliquer V015** : elle élargit le `CHECK` de `access_attempts.denial_reason` (`REGIME_NOT_ALLOWED`, `REGIME_UNKNOWN`). Sans elle, l'`INSERT` échoue **uniquement en production** — le PC et les tests ne reproduisent pas la panne.

- **La règle de permission de sortie ne tourne PAS sur les caméras du portail.** Le champ `Passagem.aplicarPermissaoDeSaida` vaut `true` sur les terminaux et `false` sur les caméras, avec une justification de quinze lignes dans `AccessDecisionService` : `ExitPermissionService.evaluate()` refuse toute personne sans permission active et ne regarde pas le type d'utilisateur ; l'allumer ferait de **chaque sortie de personnel** un `EXIT_NOT_AUTHORIZED`, des centaines par jour, sans que personne ne l'ait décidé. C'est une décision de politique laissée en suspens — et elle exige probablement de restreindre d'abord la règle aux élèves, puisque l'entité s'appelle `StudentExitPermission`.

[A COMPLETER PAR SAM] Le régime de sortie : les régimes des élèves existent-ils quelque part (papier, tableur, Pronote) ? Sans eux, allumer la règle n'a aucun sens ; avec eux, il faut savoir où ils sont pour les charger.

8. Repères chiffrés du dépôt

Mesurés le **03/09/2026**, sur `main` augmentée des trois chantiers de cette date (`fix/adresse-du-login`, `feat/livre-imprimable`, `docs/verifications-finales`). Comment : les comptes de fichiers par `find`, les suites à **froid** (`cd backend && rm -rf target && mvn -o test`, puis `npm test`). ⚠ **L'ancre est une date, pas un SHA** — ces chantiers seront fusionnés par Sam et les SHA changeront ; l'état d'**avant** eux était `fc4359c`.

Quoi	Combien
Contrôleurs / services / modèles / repositories / DTO	28 / 47 / 36 / 22 / 37
Fichiers de test Java / JS	93 / 42
Suites	<code>mvn test</code> 1055 , <code>npm test</code> 889
Migrations SQL	V001 → V027 (<code>deploy/migrations/</code>)
Permissions granulaires	10 (<code>security/Permissions.java</code> , liste <code>TODAS</code>)

⚠ **Le critère de la suite n'est pas un total, c'est : 0 échec et exactement 2 @Disabled**. Le total monte à chaque livraison ; un total **inférieur** à la référence signifie que quelqu'un a supprimé un test, et `Skipped ≠ 2` signifie que quelqu'un a désactivé une des deux requêtes natives PostgreSQL. Ces deux-là sont désactivées parce que H2 ne les exécute pas — elles exigent une **vérification manuelle**, section **6-bis** de `docs/frontend-smoke-checklist.md`. ⚠ Mesurer à **froid** (`cd backend && rm -rf target && mvn -o test`) : la compilation incrémentale a déjà produit un `BUILD SUCCESS` mensonger.

Rollbacks : chaque migration a le sien dans `deploy/migrations/rollback/`, **sauf V006, V008, V009 et V023** — V023 est une *seed*, ses lignes partent avec `R021`. C'est documenté dans `deploy/migrations/README.md`.

État de production au 03/09/2026. Ce qui concerne la VM n'est pas vérifiable depuis le dépôt : **V001 à V027** y sont toutes appliquées — V027 est `licence_clock`, posée au déploiement de la licence le 01/09, et la licence est valide jusqu'au **2027-03-31**. Le portable (le paquet Electron distribué aux postes) est complet : **96/96** fichiers obligatoires, mesuré le 03/09/2026 par `node scripts/verify-`

`package.js`. ⚠ **Ce 96 n'est pas une constante** : la liste des fichiers obligatoires est **dérivée** d' `index.html` (`scripts/indexAssets.js`), donc elle monte à chaque écran nouveau. Ce qui doit rester vrai, c'est l'égalité des deux nombres, pas leur valeur.

[A VERIFIER] Confirmer les migrations réellement appliquées sur la VM avant d'en appliquer une nouvelle — la procédure et la vérification propre à chaque migration sont dans `deploy/migrations/README.md` et dans `docs/operacional/nuit-27-28-08-rapport.md` §5.

9. Où aller ensuite

Le livre organise ; il ne recopie pas. Voici les documents qui font autorité.

Pour...	Lire
Se servir du système, écran par écran	<code>docs/manual-utilisateur.md</code> (964 lignes, en français)
L'état opérationnel réel (⚠ arrêté au 05/08)	<code>docs/operacional/handoff.md</code>
Reconstruire de zéro / restaurer une sauvegarde	<code>docs/operacional/reconstruir-do-zero.md</code>
Installer un poste	<code>docs/operacional/guide-installation-postes.md</code>
Fabriquer le portable	<code>docs/operacional/release-portable.md</code>
Mettre la VM à jour	<code>docs/operacional/mise-a-jour-vm.md</code> , <code>deploy/README.md</code>
Le détail d'un flux de bout en bout	<code>docs/architecture/fluxos.md</code>
Le catalogue des routes	<code>docs/architecture/endpoints.md</code>
Le schéma de la base	<code>docs/architecture/banco-de-dados.md</code> , <code>.claude/rules/database.md</code>
Le matériel Hikvision (payloads, pièges mesurés)	<code>.claude/rules/hikvision.md</code>
Ce qui est configurable sans toucher au code	<code>docs/operacional/inventaire-configurabilite.md</code>
Le rôle de HikCentral	<code>docs/operacional/procedimento-hikcentral.md</code>
Les deux dernières nuits de travail	<code>docs/operacional/nuit-26-27-08-rapport.md</code> , <code>nuit-27-28-08-rapport.md</code>

Les décisions d'architecture sont dans `docs/architecture/decisoes/` : **ADR-001** (deux tables), **ADR-002** (le numéro de carte n'est pas stocké — le terminal ne l'envoie jamais), **ADR-003** (observationnel), **ADR-004** (cantine assistée), **ADR-005** (voir le piège ci-dessous).

10. Trois pièges du dépôt lui-même

- ⚠ Il y a **DEUX** fichiers **ADR-005**, et ils ne parlent pas de la même chose : `ADR-005-creneaux-cantine.md` (26/08 — le planning de cantine devient une configuration, migrations V021-V023) et `ADR-005-totvs-rastreabilidade-no-dono-do-dado.md` (14/08 — TOTVS). **Le numéro a été attribué deux fois**. Je le signale sans le corriger : renuméroter casserait les renvois existants ; c'est une décision, pas une retouche. [A COMPLETER PAR SAM] Lequel des deux garde le numéro 005 ?

2. ⚠ `docs/operacional/handoff.md` ne s'arrête plus au 05/08/2026. Il a été repris le 29/08 et porte en tête « Date de coupe : 2026-09-01 », dernier merge couvert `5632d0a` (PR #87 — la licence) : vérifié le 03/09/2026 à `handoff.md:6`. Ce qui lui manque désormais, ce sont les deux PR postérieures, #89 (premier lancement) et #90 (poste administratif, ADR-007). Là où il diverge du code, **le code gagne**.
3. ⚠ **Sept endpoints ne sont pas gardés** (`/api/users` , `/api/access/logs/*` , `registerAccess`). C'est une dette **documentée et assertée en test** — `ControllerAuthorizationGuardTest.DIVIDA_CONHECIDA` , 7 entrées, taille vérifiée par le test lui-même. Les fermer cassera des écrans : c'est un chantier avec ses propres preuves, pas une correction rapide.

Chapitre 2 — Les points de passage

Ce chapitre décrit les quatre endroits où le système regarde passer des gens : le **portail**, le **CDI**, la **cantine** et l'**infirmerie**. Pour chacun : le matériel, ce qu'il envoie, les règles qui lui sont propres, et les écrans qui le servent.

Convention du livre : `[À VÉRIFIER]` marque une affirmation que je n'ai pas pu confirmer dans le dépôt, suivie de la commande qui la tranche. `[À COMPLÉTER PAR SAM]` marque ce que seul Sam savait.

1. Deux familles de matériel, et elles ne se ressemblent pas

C'est la première chose à comprendre, parce que presque tous les pièges du portail viennent de là.

	MinMoe (CDI, cantine)	DeepinView (portail)
Modèle	DS-K1T344MX-E1	iDS-2CD7A46G2-IZHSY
Ce que c'est	un terminal de contrôle d'accès	une caméra
Ce qu'il fait	il authentifie : il connaît la personne, il ouvre	il compare : il dit « ce visage ressemble à... »
Identité dans le payload	<code>employeeNoString</code> — l'identifiant, net	un <code>certificateNumber</code> et un nom , avec une similarité
Qui résout l'identité	le terminal	MAGBO , via <code>CameraIdentityService</code>

⚠ **La caméra du portail n'authentifie pas : elle compare.** Elle envoie le résultat d'une comparaison contre sa bibliothèque de visages, et c'est le MAGBO qui décide si cette comparaison désigne quelqu'un. Toute la fragilité du portail est dans cette phrase.

2. Le portail — et le défaut ouvert le plus grave du système

2.1 Le matériel

IP	Rôle	Nom de canal observé
<code>.167</code>	ENTRÉE	<code>ENTRADA-INTERNA-01</code>
<code>.166</code>	SORTIE	—

Firmware relevé : **V5.9.10** (`.claude/rules/hikvision.md`).

△ Ces IP viennent de Sam (28/08) et ne sont pas dans le dépôt. La source de vérité en production est `door_mappings`. **[À VÉRIFIER]**

```
docker exec magbo-postgres psql -U magbo -d magbodb -tAc \
"SELECT terminal_ip, point_id, action, label, ativo FROM door_mappings ORDER BY terminal_ip;"
```

2.2 △ Le défaut ouvert : le portail ne reconnaît presque plus personne

Depuis le 25/08, ~46 élèves distincts par jour, contre ~500 jusqu'au 24/08. Le personnel n'est pas touché (~40/jour avant et après). Sur ~1160 visages non identifiés par jour, seuls 22 atteignent la comparaison de nom, avec des similarités de 0,13 à 0,46.

Le backend reçoit et traite normalement — ce n'est pas un défaut logiciel. Le diagnostic du 27/08 a écarté avec preuve l'hypothèse la plus sérieuse (le changement de parser multipart) et n'a trouvé aucun défaut de code.

La chute **coïncide** avec les imports de photos des 25 et 26/08 — et depuis le 31/08 on sait que ces imports sont passés **par HikCentral**, donc par **les bibliothèques faciales des caméras**, et non par l'écran Photos du MAGBO (qui remplit `user_photos`, sans aucun effet sur la reconnaissance).

△ **Ce n'est donc plus une simple coïncidence de dates : il existe un mécanisme.** Les caméras ne font que *comparer* un visage à cette bibliothèque ; un repeuplement change ce à quoi elles comparent. C'est arrivé une fois déjà : le format du `certificateNumber` a changé le 08/08 pour la même raison.

La cause n'est pas prouvée pour autant, et ce livre ne conclut pas à sa place. Un mécanisme plausible est plus qu'une coïncidence et moins qu'une preuve. **Mesurez avant de réparer** — en commençant par le nombre de personnes ayant encore un `camera_person_id` : s'il s'est effondré, la bibliothèque est bien en cause.

Les chiffres complets, les cinq requêtes SQL à lancer sur la VM et les actions HikCentral sont dans :

- `docs/operacional/handoff.md` — en tête, c'est le document à ouvrir en premier
- `docs/operacional/diagnostic-portaria-2026-08-27.md` — le détail de ce qui a été écarté et de ce qui reste à mesurer

△ **Avant de comparer avec « avant », lisez la note historique.** La caméra `.166` (SORTIE) était en panne jusqu'au 24/08 : le portail produisait ~950 ENTRÉES et zéro SORTIE par jour. **Ces 950 n'étaient pas 950 personnes** — c'était la même population recomptée à chaque retour, faute de lecture de sortie. Le seul nombre comparable est celui des personnes **distinctes**.

2.3 Comment le MAGBO met un nom sur un visage

`CameraIdentityService` essaie quatre choses, **dans cet ordre**, et l'ordre est la décision :

1. `camera_person_id` **déjà enregistré** (colonne de `app_users`, ajoutée par la V008). Déterministe : cette personne a déjà été reconnue une fois, et on a retenu son `certificateNumber`.

2. Le `certificateNumber` lu comme **matricule** ou comme `hikvision_employee_id`, normalisé. ⚠ **Avant le nom, délibérément** : un identifiant a été saisi une fois et pointe vers une ligne ; un nom arrive tronqué et se compare par ressemblance. Si le numéro et le nom se contredisent, **le numéro gagne**.
3. **Le nom normalisé, s'il correspond à exactement UNE personne active.**
4. **Le nom tronqué** (voir §2.4), aussi seulement si un seul actif correspond.

Zéro ou plusieurs correspondances → `UNKNOWN_FACE` ou `AMBIGUOUS_NAME`. **Le service ne crée jamais une personne.**

⚠ **Depuis le 08/08/2026, le `certificateNumber` est la matricule de l'élève**, complétée de zéros jusqu'à 16 chiffres : `0000000000003535` pour l'élève `0003535`. Les enregistrements plus anciens portent encore l'identifiant à 10 chiffres du HikCentral. **La comparaison retire les zéros des deux côtés** — comparer brut ne trouve rien, et l'échec est **silencieux**.

2.4 Trois pièges mesurés en production

Ils sont tous les trois dans `.claude/rules/hikvision.md`, et tous les trois ont coûté des passages perdus.

a) La caméra translittère les accents. Le diacritique devient un caractère ASCII **avant** la lettre : `S'A` pour `SÁ`, `BRAND~AO` pour `BRANDÃO`, `C^ORTE` pour `CÔRTE`, `` CHAUVI ERE `` pour `CHAUVIÈRE`, `Isma"el`` pour `ISMAËL`. **57 personnes reconnues ne correspondaient à aucun enregistrement à cause de ça** (mesuré le 11/08/2026).

Pourquoi ça cassait si fort : pour la normalisation, ces cinq caractères sont de la ponctuation, donc des **séparateurs**. `LABB'E` devenait `labb e`, le `e` isolé était jeté par la règle des initiales, et il restait `labb` contre `labbe`. Avec l'accent dans la première syllabe, c'est pire : `C^ORTE` devient `orte` — c'est le `C` qui disparaît.

⚠ `PersonNameMatcher.normalizeRecebido` produit **deux lectures** du nom reçu (la littérale et la décodée) et **ajoute** sans jamais remplacer, parce que la translittération est **ambiguë** : `S'A` est `SÁ` et `D'AVILA` est `D'ÁVILA` — les mêmes trois caractères, deux sens. **Le nom enregistré, lui, n'est jamais décodé** : le décoder transformerait une apostrophe légitime en accent et corromprait la donnée de référence.

b) Le nom est tronqué à 32 caractères bruts par l'appareil. Deux cas réels sont devenus `UNKNOWN_FACE` alors que la personne existait : `Luis Fernando FIGUEIREDO DOS SAN` (pour `...DOS SANTOS`) et `Marcos Vinicius CLEMENTE FERREIR` (pour `...FERREIRA`).

⚠ Pire que l'échec lui-même : **la personne ne recevait jamais son `camera_person_id`**, celui qui aurait résolu tous les passages suivants — elle restait piégée dans le défaut pour toujours.

Le traitement accepte le préfixe si l'enregistrement **commence par** le nom reçu, si **un seul** actif correspond, et au-dessus d'un **plancher de 16 caractères normalisés**. Le 16 n'est pas rond par hasard : « maria santos », « ana carolina » et « carlos souza » font **exactement 12** — un plancher à 12 admettrait justement la famille de noms génériques que le plancher existe pour exclure.

c) Les échelles de similarité sont mélangées dans le même payload : `similarity` arrive en **fraction** (0.95) et `FDLibThreshold` en **pourcentage** (70). Comparer brut recalerait tout bon reconnaissance en silence. `normalizarSimilaridade` ramène les deux sur 0..1.

2.5 Deux champs qui n'identifient pas ce qu'on croit

Champ	Ce qu'il identifie	Mesuré
<code>pId</code>	la détection	38 valeurs distinctes en 38 occurrences
<code>faceId</code>	rien	2 valeurs en 18 occurrences — le même 69205 sur un échec puis, 8 s après, sur une autre personne

⚠ `faceId` ne peut être la clé de rien. L'utiliser comme clé de déduplication jetterait des passages réels en silence.

⚠ `faces[].score` arrive **emballé** — `{"value": 52}`, pas un nombre. Le modéliser en `Double` fait échouer le parsing sur **tous** les événements réels : le contrôleur répond 200, le log dit « ignoré », et le portail devient invisible.

3. Le CDI

Terminaux `.15` et `.16` (MinMoe), point `BIBLIO`.

⚠⚠ Sur le `.15`, le champ « Nom de la porte » affiche `CDI-SAIDA`. Il ment. **Fiez-vous à l'IP, jamais au nom affiché par l'appareil.** C'est le genre de détail qui fait perdre une matinée à quelqu'un qui débranche le bon terminal en croyant débrancher l'autre.

Ce qui est propre au CDI :

- **Fermeture automatique à 17:00.** Qui entre et ne repasse pas le visage en sortant resterait « dedans » indéfiniment, et l'écran ouvrirait le lendemain avec les gens de la veille. La sortie synthétique est **déclarée** : `flag=FECHAMENTO_AUTO`, `created_by_user=system`, et elle porte l'heure de **fermeture**, pas celle où le travail a tourné.
- **Présence déjà ouverte** (`flag=JA_PRESENTE`) : une ENTRÉE dans un point où la personne a déjà une présence ouverte n'ouvre pas une visite nouvelle. En production le 10/08/2026, l'élève `0003053` est entré 4 fois en 5 minutes sans sortir. ⚠ Cette règle **ne tourne pas au portail** : là-bas la sortie échappe, « il est déjà dedans » est une supposition, et marquer cacherait une **entrée réelle**.
- **Capacité, état déclaré et exclusions** (V025), avec leur registre (V026). Voir le chapitre 3.
- **Un mode urgence** (confinement), avec appel nominal.

Écrans : l'écran CDI (`js/cdi/BibliotecaView.js`), et l'écran d'exclusions (`CdiExclusionManagement`, derrière `CDI_EXCLUSION_WRITE`).

4. La cantine

Terminaux `.10`, `.12`, `.13`, `.14` (MinMoe), points `REFEI*` / `CANTINA*`.

[À VÉRIFIER] Quel terminal sert quel point : la réponse est dans `door_mappings` (requête au §2.1).
Le dépôt ne l'écrit nulle part.

Deux problèmes matériels connus (déclarés par Sam, non vérifiables ici) :

- Le `.10` **n'est pas enregistré au HikCentral** : erreur `SYS[904]`, numéro de série en conflit.
- Le `.14` **est en Wi-Fi** — c'est celui qui perdra des paquets et videra une file d'un coup. Voir la leçon sur l'heure de l'événement (chapitre 8).

Ce qui est propre à la cantine :

- C'est **le seul point où le MAGBO influe sur ce qui se passe** — et encore, par un geste humain (ADR-004 : le terminal valide l'identité, MAGBO valide la règle, **l'opérateur applique l'exception**).
- L'ordre des règles est fixé et compte : **déduplication** → **droit au repas** → **créneau**. La première qui refuse arrête tout.
- Quatre familles de flags : avant le créneau, après le créneau, passage trop court, séjour trop long.
- Le **Moniteur** en direct (polling de 3 s) avec ses trois colonnes, et le flux des tentatives refusées.

Tout le détail est au chapitre 3.

5. L'infirmirie

Point `ENFERM`.

[À VÉRIFIER] Le dépôt ne dit pas quel matériel sert l'infirmirie, ni s'il y en a un. Le rapport de nuit du 14/08 mentionne l'enfermaria comme un point « sans fermeture automatique, dont l'enregistrement est manuel » — ce qui suggère une saisie à la main plutôt qu'un terminal. **[À COMPLÉTER PAR SAM]** : l'infirmirie a-t-elle un terminal, ou les visites sont-elles saisies à la main ?

Ce qui est propre à l'infirmirie :

- Les visites longues sont comptées à part (`countLongInfirmaryStays`).
- **Pas de fermeture automatique** : quelqu'un peut donc rester « dedans » à la fin de la journée, et c'est visible dans le rapport comme « sortie non enregistrée ». C'est voulu — une visite à l'infirmirie qui ne se ferme pas est une information, pas une nuisance.

Écran : le rapport infirmirie (`InfirmaryReport`).

6. ⚠ Comment un terminal sait où envoyer ses événements

C'est le réglage qui casse en silence, et le menu n'est pas là où on le cherche.

Le chemin réel, sur l'appareil : Système et maintenance → Réseau → Service réseau → HTTP(S) → Écoute HTTP

Ce n'est PAS le menu « Événement ». C'est là que tout le monde commence, et on peut y passer une heure.

On y renseigne l'IP du serveur, le port `8080`, l'URL avec le jeton, et `HTTP`.

Trois formes d'authentification du webhook sont acceptées, parce que tous les appareils ne savent pas faire la même chose :

Forme	Endpoint	Pour qui
En-tête <code>X-MAGBO-WEBHOOK-TOKEN</code>	<code>POST /api/hikvision/webhook</code>	le cas normal
<code>?token=...</code> dans l'URL	idem	les MinMoe
Jeton dans le chemin	<code>POST /api/hikvision/webhook/t/{token}</code>	⚠ la DeepinView, qui jette la query string

⚠ `/api/hikvision/webhook/capture` est un outil de BANC, jamais de production. Il écrit le corps de la requête dans le log — et sur une caméra du portail, ce corps contient des **noms d'élèves** lus dans la bibliothèque faciale. Le log n'a pas la protection de la base.

Ce qui casse en silence, et comment le voir

⚠ **Les IP changent par DHCP.** Ni erreur, ni alerte : les événements cessent simplement d'arriver. C'est arrivé le 16/07 (terminal `.12` → `.10`, serveur → `.9`).

Avant toute session sur le matériel, les quatre vérifications :

1. l'IP du serveur ;
2. l'IP au dos de chaque terminal (elle s'affiche sur l'écran de l'appareil) ;
3. l'URL de l'Écoute HTTP — pointe-t-elle vers l'IP **actuelle** du serveur ?
4. la colonne `terminal_ip` de `door_mappings` — correspond-elle à l'IP **actuelle** du terminal ?

Une réservation DHCP a été demandée au service informatique pour les terminaux et la VM. [À COMPLÉTER — Sam ne se souvient plus de l'état de cette demande (répondu le 31/08), et n'a pas retrouvé le contact.] À qui demander : le secrétariat ou la direction connaissent le service informatique. ⚠ **N'attendez pas la réponse pour vous protéger** : les quatre vérifications ci-dessus donnent le risque réel en quelques minutes, sans contact.

⚠ **Fuseau d'usine.** Les terminaux sortent en **GMT+8**. Il faut corriger en GMT−3 et remettre l'heure à l'installation — sinon ils émettent des événements « d'authentification expirée ».

7. Ce que le terminal envoie, et ce que le MAGBO en fait

La liste des sous-types a été confirmée avec du matériel le 13/07/2026 (firmware V4.13.0) :

Sous-type	Ce que c'est	Ce que le MAGBO en fait
75	authentification par VISAGE , approuvée	<code>access_logs</code> , <code>auth_method=FACE</code>
1	authentification par CARTE , approuvée	<code>access_logs</code> , <code>auth_method=CARD</code>
8	authentification refusée / expirée	<code>access_attempts (DEVICE_DENIED)</code> — jamais <code>access_logs</code>
9	événement d'appareil, sans personne	ignoré, <code>200</code>
21 / 22	la porte s'ouvre / se ferme	ignoré, <code>200</code>
autres	démarrage, configuration	ignoré, <code>200</code>

⚠ **La liste blanche est rigide** : seuls **75** et **1** peuvent produire un accès. Un sous-type inconnu accompagné d'un `employeeNoString` devient une **tentative**, jamais un accès.

⚠ **Le sous-type 8 porte aussi un `employeeNoString`**. C'est le piège qui a justifié la liste blanche : sans elle, un refus du terminal serait entré dans `access_logs` comme un accès valide. Confirmé avec du matériel en plaçant la validité d'une personne dans le passé — le terminal refuse **à la voix**, et l'événement arrive quand même.

⚠ ****On ne distingue pas *quel* carte : le terminal traduit la carte en `employeeNoString` en interne et le numéro de carte n'arrive jamais dans le payload**** (ADR-002). C'est pourquoi le MAGBO ne garde aucun numéro de carte.

Les images ne sont jamais conservées. Les parts `faceImage` , `backgroundImage` et `faceLibImage` sont écartées. `modelData` (le gabarit biométrique) n'est ni conservé ni committé — dans les fixtures de test il est remplacé par `<<modelo-biometrico-removido>>`.

8. Où regarder ensuite

Question	Document
Le défaut du portail, avec ses chiffres	<code>docs/operacional/handoff.md</code> (en tête)
Ce qui a été écarté, et ce qui reste à mesurer	<code>docs/operacional/diagnostic-portaria-2026-08-27.md</code>
Tous les pièges Hikvision, dans le détail	<code>.claude/rules/hikvision.md</code>
Les bibliothèques faciales et le cycle des personnes	<code>docs/operacional/procedimento-hikcentral.md</code>
Les règles appliquées à chaque passage	chapitre 3
Ce que l'opérateur voit et doit faire	chapitre 4

Chapitre 3 — Les règles métier

Ce chapitre s'adresse à qui doit **modifier, expliquer ou défendre** une décision du MAGBO : pourquoi tel élève est apparu en rouge au portail, pourquoi telle ligne porte un drapeau, pourquoi un refus n'a fermé aucune porte. Il répond à trois questions pour chaque règle : *où vit-elle* (fichier, migration), *à quelle heure est-elle jugée*, et *que se passe-t-il quand elle dit non*.

3.0 La règle des règles : **DENY** est logique, jamais physique

Le MAGBO **n'a jamais fermé une porte**, et il n'y a aucun code pour le faire.

- Le webhook est **post-événement** : quand la requête HTTP arrive, le terminal a déjà décidé et la porte a déjà bougé. Séquence mesurée avec le matériel le 13/07/2026 : `21` (porte ouvre) → `75 / 1` (authentification) → `22` (porte ferme), le HTTP après. L'appareil **ignore** la réponse du MAGBO. → `docs/architecture/decisoes/ADR-003-webhook-pos-evento.md`
- Pour la cantine, c'est une décision explicite et **définitive** : *blocage opérationnel assisté*. Le terminal valide l'**identité**, le MAGBO valide la **règle**, l'**opérateur** applique l'exception. Pas de blocage physique via HikCentral pour le repas, **ni maintenant ni dans la feuille de route**. → `docs/architecture/decisoes/ADR-004-bloqueo-operacional-assistido.md`

Ce que **DENY** veut donc dire, très exactement, dans

`backend/src/main/java/com/magbo/access/services/AccessDecisionService.java` :

Mode	Effet réel
OBSERVATION	la passage est enregistrée dans <code>access_logs</code> et une ligne est écrite dans <code>access_attempts</code> pour l'audit
DENY	rien dans <code>access_logs</code> ; seule la ligne <code>access_attempts</code> est écrite

Dans les deux cas la personne est **physiquement passée**. **DENY** signifie « le MAGBO ne compte pas ceci comme un accès valide » — un fait comptable, pas une serrure. C'est même une propriété *souhaitée* : si la VM ou le réseau tombent, le déjeuner et le portail continuent de fonctionner (ADR-003).

La séparation des deux tables est structurelle : `access_logs` = accès effectif, `access_attempts` = tout ce qui a été tenté et refusé (`ADR-001-attempts-vs-logs.md`). L'écart entre les deux est **mesuré** par le KPI `divergenciaHoje` (`auth_result=SUCCESS` et `authorization_result=DENIED`). Pour la cantine, cet écart n'est pas un défaut à combler : c'est la charge de travail de l'opérateur.

3.1 ⚠ L'horloge : quelle heure juge quoi — le piège n°1 du projet

Deux horloges circulent dans `AccessDecisionService.process(...)` :

- `eventTime` — l'heure où la passage a eu lieu (le `dateTime` du payload, converti en `America/Sao_Paulo` par `services/EventTimeResolver.java`);
- `now` — l'heure où le MAGBO décide (`LocalDateTime.now()`, ligne ~170).

La règle générale du projet est : **le registre utilise `eventTime`, les règles utilisent `now`**, pour qu'une file hors-ligne vidée d'un coup ne change pas rétroactivement un `DENY` / `ALLOW`. Mais il y a des **exceptions assumées**, et chacune a été payée par un incident. Le 03/08/2026, 33 événements en file sont tous entrés à 14h51 et ont produit des **durées négatives**.

Règle	Horloge jugée	Où
Horodatage écrit dans <code>access_logs</code> / <code>access_attempts</code>	événement	<code>EventTimeResolver</code>
Utilisateur inactif	<code>now</code>	<code>AccessDecisionService</code>
Dédup repas (90 s)	<code>now</code>	<code>DeduplicationService</code>
Droit au repas (entitlement)	<code>now.toLocalDate()</code>	<code>MealEntitlementService</code>
Créneau de cantine (AVANT/APRES)	événement	<code>MealSlotService.resolver</code>
Durée en cantine (<code>EXCEDEU_TEMPO</code>)	événement	<code>AccessDecisionService.validateExitTime</code>
Autorisation de sortie ponctuelle	<code>now</code>	<code>ExitPermissionService.evaluate</code>
Régime de sortie	événement	<code>RegimeSortieService.avaliar</code>
Exclusion CDI	événement	<code>CdiExclusionService.avaliar</code>
Registre d'alerte CDI (<code>event_time</code>)	événement (heure du badge)	<code>CdiAlertService.registrar</code>
Même passage (30 s)	fenêtre autour de l'événement	<code>SamePassageService</code>
Dédup d'ingestion (60 s)	horloge du processus (<code>nanoTime</code>)	<code>WebhookIngestionDedupService</code>
Retrait manuel du Moniteur	horloge de l'école (<code>Clock.system(America/Sao_Paulo)</code>)	<code>CantineRemovalService</code>
Fermeture automatique	l'heure de fermeture (17:00), pas celle du job	<code>PresenceAutoCloseService</code>

Le critère qui départage : **une règle qui NE REFUSE JAMAIS est jugée à l'heure de l'événement** ; une règle qui peut refuser est jugée à l'heure de la décision. La raison est écrite en toutes lettres dans le code : jugée par `now`, une sortie de 10h traitée à 18h deviendrait « fin de journée — sortie normale », et l'alerte que la Vie Scolaire devait voir n'aurait jamais existé.

⚠ Deux tests figent ce choix parce qu'il est fragile : `RegimeGateWiringTest#regimeUsaAHoraDaPassagem` capture **l'argument** passé (et non le verdict — la première version passait au vert toute la journée et ne cassait qu'après 17h), et `MealSlotWiringTest` fait la même chose pour la fenêtre de cantine.

3.2 Droits repas — `meal_entitlements` (V002) et son historique (V003)

Fichier : `backend/src/main/java/com/magbo/access/services/MealEntitlementService.java`

Trois états dans `EntitlementStatus` : `AUTHORIZED`, `NOT_AUTHORIZED`, `PENDING`.

⚠ **Pas de ligne en base** = `PENDING`, et `PENDING` n'est **pas** un refus : c'est « personne n'a rempli cette case ». Le webhook ne crée **jamais** de ligne tout seul. `evaluate()` vérifie aussi `valid_from` / `valid_until` : un droit `AUTHORIZED` hors de sa fenêtre de dates redevient non-attribué.

Toute modification écrit une ligne dans `meal_entitlement_events` **dans la même transaction** (qui, quand, de → vers, `source` = `UI` | `BULK` | `API`). ⚠ Le CHECK sur `source` est **manuel** (V003) : il n'existe que sur la VM, pas sur le PC ni dans les tests. Une valeur nouvelle casse **uniquement en production**.

Les politiques (`config/PolicyProperties.java`)

Politique	Défaut du code	
<code>meal-not-entitled</code>	<code>DENY</code>	<code>DENY</code>
<code>meal-pending</code>	<code>OBSERVATION</code>	⚠ <code>DENY</code> (ligne 72)
<code>outside-meal-time</code>	<code>OBSERVATION</code>	<code>OBSERVATION</code>
<code>meal-slot-not-configured</code>	<code>OBSERVATION</code>	<i>absent des propriétés</i> → le défaut Java s'applique
<code>duplicate-meal</code>	<code>OBSERVATION</code>	<code>OBSERVATION</code>
<code>exit-not-authorized</code>	<code>DENY</code>	<code>DENY</code>
<code>user-inactive</code>	<code>DENY</code>	<code>DENY</code>
<code>missing-door-mapping</code>	<code>FALLBACK</code>	<code>FALLBACK</code>

⚠ `meal-pending=DENY` **en production a un prérequis opérationnel** (décision D5 de Sam, 16/07/2026, ADR-004) : la liste des élèves autorisés doit être importée **en masse avant le jour 1**. Sans cet import, tout élève est `PENDING`, donc refusé, et **aucun repas n'est enregistré**. C'est écrit dans le fichier de propriétés lui-même (lignes 60-63).

L'ordre des règles du réfectoire est **obligatoire** et la première qui dit `DENY` arrête tout : **dédup → droit (entitlement) → créneau horaire**. Seule `action=ENTRADA` sur `REFEI*` / `CANTINA*` y passe ; la `SAIDA` suit la logique de durée (§3.5).

Écrans : *Droits repas* (`js/components/MealEntitlementManagement.js`), permission `MEAL_ENTITLEMENT_WRITE`. ⚠ À l'import xlsx, les matricules Pronote ont des **zéros en tête** : traiter la colonne comme **texte**, jamais comme nombre.

3.3 Créneaux de cantine — V021, V022, V023 (ADR-005)

Fichier : `services/MealSlotService.java` · ADR : `docs/architecture/decisoes/ADR-005-creneaux-cantine.md`

Le fait qui a déclenché tout ça : le 25/08/2026, 164 entrées à la cantine, dont 63 `OUTSIDE_MEAL_TIME` **répartis sur 22 turmas**. Aucune n'était en faute : les fenêtres de `class_schedules` dataient de 2025, et l'affiche que la Vie Scolaire tient au mur avait changé. La cause n'était pas une faute de saisie — **le planning n'avait pas d'écran**, donc personne ne l'a jamais changé.

L'ordre de résolution (obligatoire)

1. **Exception de l'élève** pour ce jour → elle vaut, **et elle seule** ;
2. sinon, le ou les **créneaux de la turma** → il suffit **qu'un seul** corresponde ;
3. sinon → `NAO_CONFIGURADO` .

△ L'exception **remplace** la turma, elle ne s'y ajoute pas : l'élève de Terminale déplacé au second service **a cessé** d'appartenir au premier. `excecaoAluno()` supprime les exceptions du même jour avant d'insérer la nouvelle.

△ **Il suffit qu'un créneau corresponde** parce qu'une turma peut être dans deux créneaux le même jour — fait réel de l'affiche 2026 : le mardi, 1E2 et 1E3 sont dans les **deux** passages. Exiger que tous correspondent aurait refusé les deux moitiés du groupe à la fois.

Les quatre verdicts, et pourquoi deux d'entre eux ne sont pas la même chose

Verdict	Sens	Trace
<code>DENTRO</code>	la passage tombe dans un créneau	aucune
<code>FORA</code>	il y a un créneau, la passage est en dehors	drapeau <code>AVANT_CRENEAU</code> / <code>APRES_CRENEAU</code>
<code>NAO_CONFIGURADO</code>	personne n'a dit à quelle heure cette personne mange	<code>access_attempts</code> en <code>OBSERVATION</code> , motif <code>MEAL_SLOT_NOT_CONFIGURED</code> — aucun drapeau sur la passage
<code>NAO_APLICAVEL</code>	la règle n'est pas pour cette personne (non-élève, élève sans turma, turma dispensée)	rien du tout

△ `NAO_CONFIGURADO` **n'est jamais un refus**. La maternelle et l'élémentaire ne figurent pas sur l'affiche et ne peuvent pas être punies pour une case vide. C'est une **question adressée à l'adulte** qui tient le planning. Politique `OBSERVATION` , jamais `DENY` .

△ **La distinction avec `NAO_APLICAVEL` est ce qui empêche la trace de noyer qui la lit** : ~200 agents × 2 repas = **400 lignes/jour** disant qu'il manque de configurer une chose qui n'existe pas. C'est la leçon de l' `INCONNU` du régime (§3.9), appliquée avant de la repayer.

Le seed (V023) vient de deux sources — et c'est délibéré

L'affiche ne couvre que le collège et le lycée. Mais la maternelle et l'élémentaire **passent au réfectoire**, beaucoup : mesuré sur la base locale — CM2A 5226 passages, TPS/PS A 4923, MSB 4671, CPB 4372. Semer seulement l'affiche les aurait toutes basculées en « non configuré » du jour au lendemain.

Donc : **l'affiche fait autorité pour les turmas qu'elle nomme**, et **les autres sont reprises depuis `class_schedules`** . Tolérances semées : **15 min avant** (on arrive en rang) et **45 min après** (le service s'étale), réglables par créneau à l'écran.

△ Deux turmas de l'affiche (**5E3, 3E3**) n'ont **aucun élève** en base — vérifié, pas deviné. Elles sont semées **quand même** : la table est la transcription de l'affiche, et laisser l'écran signaler « turma sans élève » met le désaccord sous les yeux de la Vie Scolaire. Les taire aurait fait disparaître la question.

⚠ **V023 n'a pas de rollback propre** : ses lignes vivent dans les tables de la V021 et partent avec `R021`. C'est documenté dans `deploy/migrations/README.md` (§ V021/V022/V023).

⚠ La dette ouverte n°1 : deux tables, deux vérités possibles

`class_schedules` **n'est plus lu par la cantine**. Il survit pour une **autre** question, posée par `RegimeSortieService` : « à quelle heure finit la matinée de cette turma, et mange-t-elle ici aujourd'hui ? » — ce qui décide la fenêtre de **sortie** du midi.

Si la Vie Scolaire déplace une turma de 12h30 à 13h00 dans les créneaux, le régime continuera de lire l'ancienne heure de fin de matinée. **Ça n'ouvre ni ne ferme aucune porte** (le régime observe), mais ça peut afficher « fin de journée » au mauvais moment. Porté dans `docs/operacional/inventaire-configurabilite.md`.

`AccessDecisionService.validateEntryWindow` est marquée `@Deprecated` et **ne doit pas être rebranchée** : elle n'existe plus que pour que `EntryWindowRegressionTest` fige le comportement historique.

Turmas dispensées de badge

Clé de réglage `magbo.cantine.turmas-dispensees` (CSV, **vide par défaut**, donc personne n'est dispensé). Une turma dispensée ne produit **ni drapeau ni refus** de cantine — mais **la passage reste enregistrée**. ⚠ La conséquence PPMS est écrite à l'écran, à côté du réglage (`js/components/MealSlotManagement.js`).

● N'activez PAS la dispense en l'état — la raison est le PPMS

(Question posée à Sam le 31/08/2026. Réponse : **la conséquence PPMS n'avait pas été mesurée.**)

Le code permet la dispense ; la décision n'a jamais été prise, et **elle n'était pas comprise comme une décision de sécurité**. Elle en est une :

⚠ **Une classe dispensée de badge ne badge plus — donc elle n'apparaît plus dans le décompte d'évacuation du PPMS**, et l'écran PPMS **ne le signale pas**. Le jour d'une évacuation réelle, l'équipe de crise compterait des enfants en moins sans qu'aucun écran ne dise pourquoi. C'est exactement le défaut que le chapitre sur les leçons appelle « je n'ai pas vu » lu comme « il n'était pas là », avec des enfants dans un bâtiment.

Condition bloquante : ne pas activer tant que **le PPMS ne nomme pas explicitement les classes dispensées** et ne dit pas comment elles sont comptées autrement (appel papier de l'enseignant, par exemple).

Et ce n'est pas à la personne qui reprend le code de trancher — c'est une décision de **direction**, parce qu'elle arbitre entre le confort d'un service et un décompte d'évacuation. Détail et condition au § 8.2.9 du handoff.

(Clé de réglage : `magbo.cantine.turmas-dispensees`, CSV, **vide par défaut** — l'état actuel est donc le bon.)

3.4 Les quatre familles de drapeaux

Le champ est `access_logs.flag`, une **String de 32 caractères, sans CHECK en base** (vérifié sur PostgreSQL réel le 10/08/2026). Ajouter une valeur ne demande donc **pas** de migration.

△ **Le champ est UNIQUE — il n'empile pas.** L'ordre de priorité dans `registrarPassagem` est : **drapeau métier** → `POSTO_FIXO` → `JA_PRESENTE`. Le drapeau métier gagne toujours, parce qu'il nomme un problème que quelqu'un doit voir, alors que `POSTO_FIXO` dit seulement « ceci est la routine ».

Famille	Valeur	Écrit par	Horloge	Où ça s'affiche
Arrivé avant son service	<code>AVANT_CRENEAU</code>	backend, à l' <code>ENTRADA</code>	événement	Moniteur Cantine, Rapport réfectoire, Journal
Arrivé après son service	<code>APRES_CRENEAU</code>	backend, à l' <code>ENTRADA</code>	événement	idem
(historique)	<code>FORA_HORARIO</code>	lignes antérieures au 27/08/2026	—	compté avec les deux ci-dessus
Séjour trop long	<code>EXCEDEU_TEMPO</code>	backend, à la <code>SAIDA</code>	événement	Journal, Rapport ; en direct : colonne DOIT SORTIR
Passage trop court	(aucun drapeau)	calculé à l'écran sur la paire entrée/sortie	—	colonne SORTIS , marque discrète

Deux nuances qui comptent :

1. **Le passage trop court n'est pas un drapeau.** Il est calculé côté client (`js/utils/cantine.js`, fonction `faixaDe`) à partir de la **paire** entrée + sortie. △ Sans entrée appariée, `faixa` vaut `null` et la **ligne ne reçoit aucune marque** : inventer une durée depuis le début du service aurait accusé de « n'a pas mangé » ceux que le lecteur d'entrée n'a pas vus — le défaut réel de la cantine en production (95 entrées perdues en un jour).
2. **Le séjour trop long existe deux fois** : comme drapeau `EXCEDEU_TEMPO` écrit à la sortie (le fait est consommé), et comme **colonne DOIT SORTIR** calculée en direct pendant que la personne est encore dedans (l'alerte est utile *avant* la sortie).

△ **Lire la FAMILLE, jamais la valeur ancienne.** La liste canonique vit dans `backend/.../models/AccessLog.java` : `FLAGS_FORA_DO_CRENEAU = {"FORA_HORARIO", "AVANT_CRENEAU", "APRES_CRENEAU"}`, miroir dans `js/utils/cantine.js` (`FLAGS_FORA_CRENEAU`) — **les changer ensemble**. Le 27/08, un `if` testant `"FORA_HORARIO".equals(flag)` est devenu **code mort en silence** au moment où les drapeaux sont devenus directionnels : plus aucune ligne `OUTSIDE_MEAL_TIME` n'était écrite, et la politique correspondante ne faisait plus rien du tout.

Les **deux drapeaux de répétition** — `POSTO_FIXO` (la personne travaille à ce point) et `JA_PRESENTE` (elle entre alors qu'elle est déjà dedans) — vivent dans `AccessLogRepository.REPETICOES` et sont traités au chapitre des passages, pas ici : ils ne signalent pas une anomalie, ils expliquent une répétition.

3.5 Les durées : 15 et 30 minutes — réglables à l'écran depuis la V024

backend/src/main/java/com/magbo/access/config/CantineProperties.java

Réglage	Défaut	Sens
magbo.cantine.duracao-curta-minutos	15	en dessous, la personne est entrée mais n'a probablement pas mangé
magbo.cantine.duracao-maxima-minutos	30	au-dessus, le séjour est excessif → EXCEDEU_TEMPO + colonne DOIT SORTIR
magbo.cantine.decantacao-minutos	15	combien de temps une ligne reste visible dans DOIT SORTIR
magbo.cantine.sortis-visiveis-minutos	40	combien de temps un sortant reste visible dans SORTIS
magbo.cantine.lycee-inicio / lycee-fim	11:00 / 15:00	référence de l'alerte « la cantine a ouvert plus tôt »

Le 15 min **n'est ni un refus ni une accusation** : c'est un signal. Un élève qui traverse le réfectoire en six minutes est allé chercher quelqu'un, a renoncé à la file, ou le lecteur de sortie l'a attrapé en passant.

Le 30 min était **une heure** jusqu'au 24/08/2026 (constante MAX_CANTINA_TIME dans le code Java) : une heure dépasse le service entier d'une turma, donc l'alerte ne partait pour ainsi dire jamais.

⚠ **La décantation n'est pas une suppression** : passé le délai, la ligne quitte la colonne et va dans la pastille de l'en-tête, qui continue de la compter et ouvre la liste complète en un clic. Rien n'est effacé. La colonne est un **instrument d'action** : un opérateur qui voit trente anomalies n'agit sur aucune.

Comment la valeur effective est calculée — c'est le motif de toute la V024 :

```
// AccessDecisionService.validateExitTime
Duration teto = Duration.ofMinutes(settingsService.efetivoInt(
    "magbo.cantine.duracao-maxima-minutos",
    cantineProperties.getDuracaoMaximaMinutos()));
```

et l'écran reçoit **les mêmes valeurs effectives** par GET /api/access/report-config (AccessController, bloc cantine). ⚠ Ce point d'entrée existe précisément pour qu'il n'y ait **jamais** deux nombres pour le même jour sur le même écran : tant que la constante était recopiée dans le JS, changer la property côté serveur laissait l'écran dire autre chose, **sans erreur**.

3.6 Retraits manuels du Moniteur Cantine — V020

deploy/migrations/V020__cantine_removals.sql · services/CantineRemovalService.java

L'opérateur au comptoir voit dans « Dans la cantine » ou « Doit sortir » une ligne qu'il **sait** fausse : la personne est sortie, le lecteur ne l'a pas vue.

⚠ **C'est un geste d'ÉCRAN, et rien d'autre**. Le retrait ne touche pas access_logs, n'écrit **aucune** sortie synthétique, **ne ferme pas** la présence du PPMS et ne change aucun rapport de visite.

L'alternative refusée est écrite dans la migration, et il faut la connaître : réutiliser le mécanisme de `FECHAMENTO_AUTO` (une `SAIDA` synthétique avec un drapeau nouveau) coûtait **zéro migration**. Mais une sortie synthétique **ferme aussi la présence PPMS** : l'écran d'évacuation aurait affirmé qu'un enfant a quitté l'école parce que quelqu'un a nettoyé une colonne. Cet écran s'ouvre dans une cour, et il répond à une seule question.

- Clé : `(user_id, point_id, dia)`. `point_id` parce que le moniteur affiche REFEI1, REFEI2 et CANTINA1 sur le même écran ; `dia` parce que le moniteur repart à minuit.
- ⚠ `removido_em` **n'est pas de l'audit, c'est LA RÈGLE** : seules les passages **antérieures** à cet instant sont cachées. Si la personne rentre à 13h après un retrait à 12h30, l'entrée nouvelle **réapparaît** — celui qui a cliqué à 12h30 ne savait rien de 13h.
- Annuler est **soft** (`desfeito_em` / `desfeito_por`) ; un nouveau retrait **réutilise** la ligne (l'UNIQUE en garantit une seule).
- ⚠ Piège de fuseau, mesuré : le conteneur backend démarre **sans TZ**, donc en UTC. Avec `Clock.systemDefaultZone()`, un retrait fait à 14h26 était enregistré à 17h26 — **trois heures dans le futur** — et le × cessait de cacher une ligne pour **faire taire la personne pendant trois heures**. D'où `Clock.system(EventTimeResolver.ZONA_ESCOLA)`.
- Autorisation : permission `CANTINE_REMOVAL_WRITE` **plus** `@areaSecurity.can(#pointId)` — la permission est globale, le point ne l'est pas.

3.7 `system_settings` — la surcouche (V024)

`deploy/migrations/V024__system_settings.sql` · `services/SettingsService.java` · `services/SettingsCatalog.java`

Le contrat, en une phrase : « **défaut = comportement actuel** ». Une base sans **aucune** ligne dans `system_settings` se comporte **exactement** comme avant la V024. Une ligne n'existe que quand quelqu'un a **modifié** un réglage à l'écran, et elle porte **qui** et **quand**.

- Valeur en **texte**, pas de colonne typée ni d'enum (leçon V014/V017 : pas de CHECK qui diverge entre installations). Le typage vit dans `SettingsService`, à côté du défaut.
- ⚠ **Valeur illisible = défaut + WARN**, jamais d'exception : un « abc » dans une clé numérique ne peut pas faire tomber la décision d'une passage.
- Cache **15 s** + invalidation à l'écriture — parce que `efetivo*` est lu sur le chemin le plus critique du système (chaque passage).
- Valeur **vide = suppression de la ligne** : « revenir au défaut » est une action de première classe.
- ⚠ **Jamais de secret ici**. Tokens, mots de passe, PIN et clé JWT vivent dans l'environnement (`.env` de la VM, `setx` du PC).
- `SettingsCatalog.entradas()` est **la seule déclaration** de ce qui est réglable ; une clé lue par le code mais absente du catalogue serait invisible et irréparable. `SettingsCatalogGuardTest` échoue dans ce cas.

- ⚠ Le défaut affiché **n'est jamais recopié** dans le catalogue : chaque entrée va le chercher à la source réelle (le bean de propriétés, la constante du contrôleur). Un « défaut : 50 » recopié aurait menti le jour où la property change.
- Les clés **orphelines** (présentes en base, absentes du catalogue) sont quand même affichées, groupe `orphelins` — sinon aucun écran ne pourrait les effacer.

Clés déclarées aujourd'hui : cinq pour la cantine (§3.5 + `turmas-dispensees`) et cinq pour le CDI (`magbo.cdi.capacidade`, `.estado`, `.estado-inicio`, `.estado-fim`, `.estado-nota`).

3.8 CDI — capacité, exclusions (V025), registre des alertes (V026)

Capacité et état

`CdiController` : `magbo.cdi.capacidade` (défaut 50, qui vivait en dur dans `js/cdi/cdiData.js`), et `magbo.cdi.estado` ∈ `OUVERT` | `RESERVE` | `FERME` (défaut `OUVERT`), avec heure de début, de fin et une note. Écran : *Configuration*, permission `CONFIG_WRITE` ; ou l'écran CDI lui-même (`PUT /api/admin/cdi/etat`). ⚠ `SettingsCatalog.validar` refuse une capacité `< 1` : sans elle, un `PUT` sur la clé brute passait à « 0 » et le CDI se déclarait **plein pour toujours**.

Exclusions (V025)

`services/CdiExclusionService.java`

- Portée : **un élève OU une turma**, jamais les deux (`CHECK ck_cdi_exclusions_alvo`).
- ⚠ **Ça n'empêche personne d'entrer**. Le terminal ouvre de toute façon (ADR-003). Ce que ça fait : **prévenir l'adulte présent**, fort et clair, au moment du badge.
- L'exclusion **individuelle** l'emporte sur celle de la turma — pas pour le verdict (les deux disent « exclu ») mais pour le **message** : la conversation n'est pas la même.
- `ate` **nul est légitime et fréquent** (« jusqu'à nouvel ordre »). Une date de fin déjà passée est **refusée** à la création : elle n'aurait jamais prévenu personne.
- ⚠ Pas de colonne de début : `criado_em` **EST la date de début**, et `ativaEm` refuse les jours antérieurs — sans cette borne, une mesure posée aujourd'hui marquerait les passages de la semaine dernière, puisque le verdict est jugé à l'heure de l'événement.
- ⚠ `turma` **n'est pas une photographie** : la comparaison se fait avec la turma **actuelle** de l'élève. Qui entre dans la classe hérite de l'exclusion, qui en sort la perd. C'est assumé dans cette version et écrit sur l'écran de création.
- Levée = **soft** (`revogado_por` / `revogado_em`), la ligne reste. Même doctrine que `student_exit_permissions` et `student_regimes` : une mesure prise sur un enfant est une preuve.
- ⚠ Lecture **et** écriture derrière la permission `CDI_EXCLUSION_WRITE`, pas derrière l'aire `cdi` : une exclusion nomme un enfant et raconte une sanction. L'écran du CDI, lui, ne reçoit que de quoi **reconnaître** la personne au badge — sans motif ni auteur.

Registre des alertes (V026)

`services/CdiAlertService.java` · trois types, CHECK manuel : **EXCLUSION** (une personne ou une classe exclue a badgé), **CAPACITE** (le badge a franchi la capacité), **FERME** (badge pendant un état fermé/réservé).

- `event_time` = l'heure du **badge**, jamais celle du traitement.
- **△ criado_por** est estampillé par le **SERVEUR** (le principal authentifié), jamais par le corps du POST. Ajouté **avant tout déploiement** par le panel du 28/08 : après la première ligne écrite, ce serait irréparable.
- L'alerte **CAPACITE** est enregistrée **sans nom** : le premier d'un tick de polling n'est pas « qui a rempli la salle », c'est l'ordre d'un tableau.
- Écriture en `REQUIRES_NEW` + `catch` chez l'appelant : un registre qui tombe ne doit **jamais** emporter ce qui se passait autour.
- Lecture derrière `CDI_EXCLUSION_WRITE`, pas derrière l'aire.

△ LA LIMITE STRUCTURELLE DU REGISTRE — à dire à quiconque s'en sert comme preuve

Le seul écrivain est l'écran du CDI : `js/cdi/BibliotecaView.js`, fonction `registrarAlerta`, appelée dans `avisar(...)`, en **fire-and-forget** (le son et la modale partent **avant** le POST, jamais en fonction de lui — un échec réseau ne laisse qu'une ligne de console).

Conséquences, toutes vraies en même temps :

1. Si l'écran du CDI n'est pas ouvert, un élève exclu peut badger et **aucune ligne n'est écrite**. Le webhook, lui, enregistre bien la passage dans `access_logs` — mais il n'évalue **pas** l'exclusion.
2. Si le POST échoue (réseau, backend redémarré), l'alerte a été **vue et entendue** au comptoir sans laisser de trace.

△ Donc : **l'absence de ligne ne prouve pas l'absence de badge**. Le registre répond à « l'écran a-t-il signalé cette personne, quand, combien de fois » — et à rien d'autre. Pour « cette personne est-elle passée au CDI ce jour-là », la source est `access_logs`, pas `cdi_alert_events`.

3.9 Régimes de sortie — V014, V015, V017

`services/RegimeSortieService.java` · `models/RegimeVerdict.java`

Le **régime de sortie** est le droit **annuel** de sortir, déclaré par écrit par les responsables légaux (circulaire n° 96-248). Il **ne remplace pas** `student_exit_permissions` : le régime est la règle de l'année, la permission est l'exception du jour, et **l'exception l'emporte**.

Le service cite le devoir quotidien du CPE comme spécification : « contrôle du carnet, des signatures, des régimes et des emplois du temps ». **Quatre** vérifications ; le MAGBO en assume **deux** (régime, signatures — elles sont en base), et il est **explicite** sur la troisième : l'emploi du temps vit dans Pronote et **n'arrive jamais ici**.

L'ordre des règles (obligatoire)

1. Pas élève → NON_APPLICABLE ;
2. **Permission ponctuelle** valide maintenant → AUTORISE (l'exception gagne, et c'est à ça qu'elle sert) ; 2-bis. **la permission que cette sortie vient de consommer** (USED dans une fenêtre de ± 2 min) → AUTORISE . Sans ce degré, un élève sorti **avec** autorisation signée réapparaissait en **rouge** trois secondes plus tard, une fois la permission SINGLE consommée (relevé par le chef d'établissement, 14/08) ;
3. **Fin de journée** → AUTORISE . Sans ce degré, des centaines de rouges à 17h et l'alerte qui compte meurt noyée ;
4. Le **régime** ;
5. **Pas de régime** → INCONNU .

Les cinq verdicts, et pourquoi ils sont cinq

Un verdict binaire (peut / ne peut pas) obligerait le système à **mentir trois fois** : inventer « peut » pour qui dépend de l'emploi du temps, « ne peut pas » pour qui n'a simplement pas de régime saisi, et quelque chose pour le professeur qui n'est pas élève.

Verdict	Couleur	Sens	Trace dans
AUTORISE	vert	permission ponctuelle valide, ou le régime annuel autorise la sortie autonome	aucune
NON_AUTORISE	rouge	régime 1 (surveillé) sans permission ponctuelle	REGIME_NOT_ALLOWED
A_VERIFIER	ambre	régime 2 (semi-libre) : dépend d'une absence de professeur, que le MAGBO ne sait pas	REGIME_TO_VERIFY
INCONNU	ardoise	aucun régime saisi pour cet élève	⚠ aucune, volontairement
NON_APPLICABLE	sans couleur	la personne n'est pas élève	aucune

⚠ **Ce que le vert signifie exactement** : « le régime annuel de cet élève autorise la sortie autonome ».

Pas « j'ai vérifié l'emploi du temps et il est libre maintenant ». L'écran doit le dire **avec les mêmes mots** (clés regime.motivo.*).

⚠ **A_VERIFIER est le verdict le plus important de l'enum**, et ce n'est **pas une objection** : le MAGBO ne conteste pas la sortie, il ne sait pas. Il laisse une trace en OBSERVATION avec un motif propre, parce qu'un verdict que personne ne peut compter après coup ne peut pas être amélioré — et c'était la moitié des cas douteux.

⚠ **INCONNU ne laisse pas de trace, exprès** : au jour 1, **923 élèves** n'ont pas de régime, et 923 lignes/jour noieraient les deux familles qui comptent. Même discipline que PENDING pour le repas : **une donnée non remplie n'est pas une donnée négative.**

Fin de journée : l'heure de la **turma**, pas celle de l'école

`RegimeProperties` : `fim-manha=12:00`, `retomada-tarde=14:00`, `fim-dia=17:00`, `tolerancia-minutos=15`.

- Pour l'**externe**, la fin de la matinée termine la demi-journée ; pour le demi-pensionnaire et l'interne, seule la fin du jour — sauf le jour où la grille dit `'N'` (la turma ne mange pas ici : le mercredi typique, 30 turmas sur 43), où le demi-pensionnaire obtient **la même fenêtre de midi** que l'externe, **pas la journée entière**.
- ⚠ L'heure de fin de matinée est lue **par turma** dans `class_schedules` (11H00, 12H30, 13H00 selon la turma). Avec un 12:00 unique, un élève d'une turma finissant à 13h00 recevait un **vert** « **sortie normale** » **une heure trop tôt**.
- ⚠ La fenêtre de midi **se referme** à `retomada-tarde`. La première version ne demandait que « est-ce après la fin de la matinée ? » et répondait donc « fin de journée » à 14h30, à 16h et jusqu'à minuit : l'élève de régime 1 qui filait après le déjeuner recevait un vert.

Ce que le régime fait, et ne fait pas

⚠ **Il observe, point.** Il ne retourne jamais, n'empêche jamais l'enregistrement, ne refuse jamais (ADR-003). Il tourne **au portail, à la SORTIE**, pour les deux branches (terminal et caméra). L'écriture de l'observation est **isolée** (`REQUIRES_NEW` + `catch` chez l'appelant) : sans ce `catch`, un échec du registre faisait sauter la transaction de la passage et **effaçait l'**`access_log` — le défaut même que le mécanisme existe pour empêcher.

État d'activation

⚠ `magbo.regime.habilitado` = `false`, dans `application.properties` (ligne 248) **et** dans le champ Java, et `MAGBO_REGIME_HABILITADO=false` dans `deploy/.env.example`. Régime désactivé → `NON_APPLICABLE` pour tout le monde.

⚠ **Appliquer V015 (et V017) AVANT de l'activer** : sans elles, le CHECK de `access_attempts.denial_reason` ne connaît pas `REGIME_NOT_ALLOWED` et l'INSERT échoue **uniquement sur la VM**, à l'intérieur de la transaction de la passage.

⚠ `magbo.regime.desconhecido` doit **rester** `OBSERVATION` jusqu'à ce que les régimes soient chargés. Le passer à `DENY` avant transformerait l'école entière en rouge — l'erreur déjà documentée pour `meal-pending` (D5/ADR-004).

[À COMPLÉTER PAR SAM] À quelle condition précise le régime doit-il être activé en production (nombre d'élèves saisis ? date ? accord de la Vie Scolaire ?), et qui décide de passer `desconhecido` à `DENY` ?

3.10 Autorisations de sortie ponctuelles — `student_exit_permissions` (V004, V012)

`services/ExitPermissionService.java`

Quatre types (`ExitPermissionType`) : `PERMANENT` , `DATE_RANGE` , `RECURRING` (jours ISO en CSV), `SINGLE` . Chacun peut porter une **fenêtre horaire** (`start_time` / `end_time`) qui est vérifiée **avant** le type.

- Évaluée à `now` (horloge de la décision), sur `PORT*` + `SAIDA` seulement.
- `SINGLE` est **consommée** uniquement à la sortie effective (`consumeIfSingle` , après l'enregistrement) — et **après** la règle de même passage, pour qu'une lecture répétée ne consomme pas deux fois la même autorisation.
- La révocation est **soft** — jamais de `DELETE` .
- Politique `exit-not-authorized=DENY` : sans permission active, `evaluate()` rend `EXIT_NOT_AUTHORIZED` ; hors fenêtre, `OUTSIDE_EXIT_WINDOW` .
- Écran : *Sorties*, permission `EXIT_PERMISSION_WRITE` . « Autorisé par » vient du **cadastre** (`responsavel_id`), pas d'un champ libre : le champ est une preuve.

⚠ **La règle de sortie est DÉSACTIVÉE sur les caméras du portail**, et c'est délibéré. Le champ `Passagem#aplicarPermissaoDeSaida` vaut `true` pour les terminaux et `false` pour les caméras : `evaluate()` refuse **toute** personne sans permission active et ne regarde pas le type d'utilisateur, donc l'activer ferait de **chaque sortie d'agent à 17h** un `EXIT_NOT_AUTHORIZED` — des centaines par jour, sans que personne l'ait décidé.

[À COMPLÉTER PAR SAM] Faut-il brancher l'évaluation de sortie sur les caméras du portail, et faut-il d'abord restreindre la règle aux `ALUNO` (l'entité s'appelle `StudentExitPermission`) ? Le javadoc renvoie explicitement la décision à Sam.

3.11 PPMS — qui est encore à l'intérieur

`services/PpmsService.java` · `GET /api/ppms/inside` · permission `PPMS_READ`

Résumé (le détail est dans le chapitre des écrans et dans `docs/manual-utilisateur.md`) :

- Calculé **en Java**, pas par la requête `currentOccupancyByPoint` — celle-ci est PostgreSQL-only et `@Disabled` dans la suite. Le nombre qui dit s'il reste un enfant à l'intérieur **ne peut pas venir d'une requête qu'aucun test n'exécute**.
- ⚠ Exclusion **asymétrique** : on écarte l'**ENTRÉE** marquée en répétition, **jamais la SORTIE**. Exclusion symétrique = qui a un poste fixe reste « dedans » jusqu'à minuit (défaut payé le 10/08/2026).
- Zone `EM_TRANSITO` pour qui a quitté un point mais reste dans l'école : dire « CDI » enverrait l'équipe chercher dans une salle vide.
- Avertissement supplémentaire quand quelqu'un se trouve dans un point **sans fermeture automatique** (l'infirmier, dont la sortie est manuelle).
- ⚠ **Ne remplace pas l'appel**. L'écran le dit au-dessus du nombre. Cache hors-ligne en `localStorage` , avec l'heure du cliché en évidence.
- Pas d'export CSV de masse : décision de Sam — la loi demande du papier pour la cellule de crise, et un CSV avec le nom de tous les enfants vit pour toujours sur le portable de quelqu'un.

3.12 Fermetures automatiques de présence

`services/PresenceAutoCloseService.java` · `config/PresenceAutoCloseProperties.java`

Le CDI ferme à 17:00 et personne n'y dort — mais la présence dérive du **dernier événement**, donc qui n'a pas badgé en sortant reste « dedans » pour toujours.

- Une **SAIDA synthétique** est écrite, **déclarée et jamais déguisée** : `flag=FECHAMENTO_AUTO`, `created_by_user=system`.
- **⚠ Heure écrite = l'heure de FERMETURE** (17:00), pas celle du job. C'est ce qui rend le résultat déterministe même si le backend était arrêté.
- **Idempotent par deux voies** : après le premier passage le dernier événement est une **SAIDA** ; et de toute façon, rien n'est écrit s'il existe déjà un `FECHAMENTO_AUTO` pour cette personne / ce point / ce jour. La seconde voie est nécessaire : quelqu'un qui entre **après** la fermeture redeviendrait candidat.
- Job toutes les 5 minutes ; il ferme tout point dont l'heure **est déjà passée**.

Configuration en production (`application-prod.properties`, lignes 120-127) : `BIBLIO=17:00` et `REFEI1=15:00`.

✓ L'heure de 15:00 est confirmée — mais la sortie reste synthétique

(Répondu par Sam le 31/08/2026.)

Le fichier porte la mention « **⚠ CONFÉRER L'HEURE AVEC LA CANTINE** avant le jour 1 ». **La confirmation a été faite : 15:00 correspond au service réel.** Personne n'est fermé au milieu de son repas.

⚠ Ce que cela ne dit pas, et qu'il faut lire lentement : 15:00 est un **plafond juste** — à cette heure-là le service est fini, donc la fermeture ne ment pas sur la *présence*. Ce n'est pas une heure de sortie *individuelle*. Mesuré en production le 25/08 : **72 sorties écrites à 15:00 pile, dans la même minute.** Ce ne sont pas 72 personnes parties à 15:00 — ce sont 72 personnes dont la sortie n'a **jamais été lue**.

Conséquence : pour ces lignes, la durée de repas est un MAXIMUM, pas une mesure. Un déjeuner de vingt minutes peut apparaître à trois heures. Toute moyenne qui les inclut est fausse vers le haut. Elles sont reconnaissables (`flag=FECHAMENTO_AUTO`, `created_by_user=system`) et donc excluables — le handoff, § 2.4, donne la requête et le piège du `flag IS NULL OR` qu'elle contient.

La question ouverte n'est donc plus l'heure, c'est le taux : pourquoi 72 sorties ne sont-elles pas lues ? Terminal de sortie absent, mal placé, ou personne ne badge en partant. Cela s'observe à la cantine à 13h, pas dans le dépôt.

3.13 Les TROIS couches de déduplication — elles ne sont pas la même chose

C'est la confusion la plus fréquente du projet. Trois mécanismes, trois causes, trois clés. Aucun ne remplace un autre.

#	Service	Nature	Clé	Fenêtre	Ce qu'il attrape
1	WebhookIngestionDedupService	infrastructure	IP source + serialNo (caméras : pId)	magbo.ingestion-dedup.ttl-seconds = 60 s, glissante	l'appareil réenvoie le MÊME paquet parce que la destination était tombée
2	DeduplicationService	règle métier de cantine	personne + point + action	magbo.dedup.window-seconds = 90 s	quelqu'un veut un second repas
3	SamePassageService	lecture répétée	personne + point + action	magbo.same-passage-window-seconds = 30 s, fenêtre des deux côtés de l'événement	le terminal a lu la même face deux fois — événements <i>différents</i> , serialNo neuf, donc (1) les laisse passer, correctement

Notes qui sauvent du temps :

- La fenêtre de (1) est **glissante** exprès : un appareil bloqué en boucle reste supprimé tant que la boucle dure. Le 60 s est borné des deux côtés — assez long pour couvrir une boucle à ~1 req/s, assez court pour ne jamais atteindre deux passages réelles de la même personne.
- Dans (3), l'**action** fait partie de la clé : une ENTRADA suivie d'une SAIDA dans la même minute est une personne qui est entrée puis sortie, pas une lecture répétée.
- ⚠ Dans (3), **la requête en base ne suffit pas**, et la cantine l'a prouvé le 24/08/2026 : quatre appareils sur un même point (.10/.12 en entrée, .13/.14 en sortie) lisent la même personne à ~300 ms d'intervalle ; la deuxième transaction interroge **avant** que la première ait commité, les deux lisent « n'existe pas » et les deux écrivent. D'où réserver — une réservation **atomique en mémoire**, avec **exactement la même fenêtre**. La requête en base reste : elle couvre ce que la mémoire ne couvre pas (redémarrage, et la fenêtre entière après expiration).
- Une **quatrième** couche existe, de nature différente : PostoFixoService et PresencaAbertaService ne suppriment rien — ils **marquent** (POSTO_FIXO , JA_PRESENTE) des passages **physiques réelles**, séparées de plusieurs minutes. Élargir une fenêtre de dedup pour les couvrir effacerait des entrées et des sorties légitimes de tout le monde.

3.14 Les permissions

backend/src/main/java/com/magbo/access/security/Permissions.java — la liste TODAS les rassemble, et **c'est la seule** : jusqu'au 14/08/2026 elle existait en double dans SystemUserController , et un test cherchant le nom de la permission passait même avec la vérification supprimée.

Permission	Gouverne
MEAL_ENTITLEMENT_WRITE	modifier les droits repas
EXIT_PERMISSION_WRITE	créer/révoquer les autorisations de sortie

Permission	Gouverne
ATTEMPTS_READ	lire les tentatives refusées
REGIME_WRITE	saisir le régime de sortie d'un élève
PPMS_READ	lire la liste nominative de qui est dans l'école
CANTINE_REMOVAL_WRITE	retirer une ligne du Moniteur (+ can(#pointId))
MEAL_SLOT_WRITE	modifier le planning de cantine (lecture : par aire)
PARCOURS_READ	lire le parcours du jour d'une personne, tous points confondus
CONFIG_WRITE	lire et écrire les réglages system_settings
CDI_EXCLUSION_WRITE	lire et gérer les exclusions du CDI + le registre d'alertes

Trois principes lisibles dans les javadocs :

1. **Une permission, pas une aire**, quand la donnée traverse l'école (PPMS_READ , PARCOURS_READ) ou nomme un enfant sous sanction (CDI_EXCLUSION_WRITE). « Restreindre, pas fermer » — décision de Sam, 14/08/2026.
2. **Lire ≠ écrire**, sauf pour CONFIG_WRITE et CDI_EXCLUSION_WRITE , où la lecture est déjà du matériel d'administration ou une donnée sensible.
3. Sans permission granulaire, les champs sont **désactivés, pas cachés** : la lecture reste ouverte par aire.

⚠ "*" est accepté par SystemUser.hasPermission (compatibilité, et seulement comme chaîne entière) mais **n'est pas proposé à l'écran** : il accorderait aussi ce qui n'existe pas encore.

3.15 Vérifications à faire sur la VM

Rien de ce qui suit n'est vérifiable depuis le dépôt. Les états de production ci-dessous ont été **affirmés par Sam le 28/08/2026**.

[À VÉRIFIER] Les migrations **V001 → V027** sont appliquées (état au 03/09/2026). Les tables créées par les six dernières :

```
docker exec magbo-postgres psql -U magbo -d magbodb -tAc \
"SELECT tablename FROM pg_tables WHERE tablename IN
('cantine_removals', 'meal_slots', 'meal_slot_classes', 'meal_slot_students',
'system_settings', 'cdi_exclusions', 'cdi_alert_events') ORDER BY 1;"
```

→ les 7 lignes doivent apparaître.

[À VÉRIFIER] Le CHECK de denial_reason contient bien les valeurs des V009, V015 et V022 — sinon l'INSERT échoue **seulement en production**, à l'intérieur de la transaction d'une passage :

```
docker exec magbo-postgres psql -U magbo -d magbodb -tAc \
"SELECT pg_get_constraintdef(oid) FROM pg_constraint
WHERE conname='access_attempts_denial_reason_check';"
```


→ doit contenir `MEAL_SLOT_NOT_CONFIGURED`, `REGIME_NOT_ALLOWED`, `REGIME_UNKNOWN`, `REGIME_TO_VERIFY`, `UNKNOWN_FACE`, `AMBIGUOUS_NAME`.

[À VÉRIFIER] Les politiques réellement chargées au démarrage — `PolicyProperties` les journalise en une ligne au boot :

```
docker logs magbo-backend 2>&1 | grep "MAGBO policies:"
```

→ attendu en production : `meal-pending=DENY`.

[À VÉRIFIER] Le seed V023 est bien en place et le planning correspond à l'affiche au mur (voir `docs/operacional/controle-affiche-cantine.md` et son `.sql` de contrôle) :

```
docker exec magbo-postgres psql -U magbo -d magbodb -tAc \
"SELECT s.dia_semana, s.hora, count(c.turma)
FROM meal_slots s LEFT JOIN meal_slot_classes c ON c.slot_id=s.id
GROUP BY 1,2 ORDER BY 1,2;"
```

[À VÉRIFIER] Quels réglages ont été modifiés à l'écran (aucune ligne = tout est au défaut du code, ce qui est l'état de naissance) :

```
docker exec magbo-postgres psql -U magbo -d magbodb -tAc \
"SELECT chave, valor, updated_by, updated_at FROM system_settings ORDER BY chave;"
```

[À VÉRIFIER] Le régime est-il activé sur la VM ? La valeur vient de l'environnement, pas du dépôt :

```
grep MAGBO_REGIME_HABILITADO deploy/.env      # sur la VM, .env n'est pas versionné
docker exec magbo-backend env | grep MAGBO_REGIME
```

[À VÉRIFIER] Les six terminaux en service (affirmé par Sam le 28/08 : portail .166 SORTIE et .167 ENTRÉE, CDI .15 et .16, cantine .10/.12/.13/.14) correspondent bien aux `door_mappings` — ⚠ les IP bougent en DHCP et la panne est **silencieuse** :

```
docker exec magbo-postgres psql -U magbo -d magbodb -tAc \
"SELECT terminal_ip, point_id, action, label FROM door_mappings WHERE ativo ORDER BY terminal_ip;"
```

3.16 Un défaut de rangement à signaler (ne pas corriger à l'aveugle)

⚠ Le dossier `docs/architecture/decisoies/` contient **deux fichiers numérotés ADR-005** :

- `ADR-005-creneaux-cantine.md` (26/08/2026 — celui que ce chapitre cite) ;
- `ADR-005-totvs-rastreabilidade-no-dono-do-dado.md`.

Les deux sont des décisions réelles et acceptées ; c'est la **numérotation** qui est en double. Renuméroter casserait les liens existants dans le code et la documentation, donc le fait est **signalé, pas corrigé**.

[À COMPLÉTER PAR SAM] Lequel des deux garde le numéro 005, et le second devient lequel ? (Une redirection depuis l'ancien nom serait préférable à un renommage sec.)

Pour aller plus loin

Sujet	Document
Ce que chaque écran affiche, écran par écran	<code>docs/manual-utilisateur.md</code>
L'état opérationnel du jour	<code>docs/operacional/handoff.md</code>
Reconstruire / restaurer, commandes exactes	<code>docs/operacional/reconstruir-do-zero.md</code>
Ce qui est réglable et ce qui ne l'est pas	<code>docs/operacional/inventaire-configurabilite.md</code>
Contrôler le planning contre l'affiche	<code>docs/operacional/controle-affiche-cantine.md</code> (+ <code>.sql</code>)
Appliquer les migrations, dans l'ordre, avec les gardes	<code>deploy/migrations/README.md</code>
Revue des V021-V023	<code>docs/operacional/revue-migrations-v021-v023.md</code>
Les décisions et leur pourquoi	<code>docs/architecture/decisoes/ADR-001</code> à <code>ADR-005</code>
Règles de code par domaine	<code>.claude/rules/backend.md</code> , <code>database.md</code> , <code>frontend.md</code>

Chapitre 4 — Guide de l'opérateur

Ce chapitre s'adresse aux personnes qui utilisent le système tous les jours. Il répond à une seule question, mais pour chaque situation : **le système vient de me dire quelque chose — qu'est-ce que je fais ?**

Le détail écran par écran vit dans `docs/manual-utilisateur.md` (964 lignes). Ce chapitre ne le recopie pas : il donne les gestes et renvoie.

⚠ **La règle qui vaut pour tout ce chapitre : le système n'a bloqué personne.** Quand un écran affiche un refus, la personne est **déjà passée**. L'alerte s'adresse à l'adulte présent, et ce qui se passe ensuite lui appartient. Voir chapitre 1, ADR-003.

1. Par profil — ce que vous ouvrez le matin

Portaria

Vous voyez : l'écran du portail, avec les passages en direct.

Votre geste quotidien : enregistrer à la main le passage de quelqu'un que la reconnaissance n'a pas vu. Le bouton est sur l'écran du secteur ; la personne est cherchée par nom ou par matricule.

⚠ **Une saisie manuelle laisse une signature** : votre login part dans `created_by_user`. Ce n'est pas une surveillance, c'est ce qui permet de répondre, six mois plus tard, à « d'où vient cet enregistrement ». Voir le chapitre 3.

Ce que vous ne pouvez pas faire : effacer un passage. Rien ne s'efface.

Cantine

Vous voyez : le **Moniteur Cantine**, trois colonnes qui se rafraîchissent toutes les 3 secondes, et à droite le flux des tentatives refusées.

Colonne	Ce qu'elle contient
Dans la cantine	les personnes entrées, qui n'ont pas encore de sortie
Doit sortir	celles dont le séjour dépasse le plafond — la seule colonne sur laquelle on agit
Sortis	celles qui sont sorties, gardées visibles un moment

Votre geste quotidien : regarder la colonne du milieu, et appliquer les exceptions que le règlement vous laisse (ADR-004 : le MAGBO valide la règle, **vous** appliquez l'exception).

Le ☐ **d'une ligne** la retire de **votre écran**, pas de la base : le passage reste enregistré, la présence PPMS reste ouverte, les rapports le comptent toujours. Une confirmation vous le dit avant.

CDI

Vous voyez : la liste des présents, un champ de scan, et le compteur avec la capacité.

Votre geste quotidien : rien, la plupart du temps — les passages arrivent seuls par le terminal. Vous pointez à la main quand quelqu'un entre sans badger.

⚠ **À 17:00, tout le monde est fermé automatiquement.** Ce n'est pas une erreur : sans ça, l'écran ouvrirait le lendemain avec les gens de la veille. Les sorties automatiques sont marquées et reconnaissables.

Vie Scolaire

Vous voyez : l'écran d'accueil, avec **la barre de recherche au centre**.

Votre geste quotidien : taper un nom. Les suggestions arrivent à la frappe, les flèches et Entrée les parcourent, et le **parcours du jour** s'ouvre : où la personne est entrée, à quelle heure, où elle est allée ensuite.

⚠ **Les trois réponses, et ce qu'elles veulent dire :**

Réponse	Ce que ça veut dire
Dans \<zone>, depuis HH:MM	le dernier événement est une entrée
Sorti de \<zone> à HH:MM	le dernier événement est une sortie — la personne n'y est plus
Aucun passage vu aujourd'hui	le système n'a rien vu . Ce n'est pas « absent »

⚠ « **Aucun passage vu** » **ne veut pas dire « absent »**. Un enfant entré par une porte non équipée est à l'école sans une seule ligne. C'est la distinction la plus importante de tout le système : voir chapitre 8, « je n'ai pas vu ≠ il n'était pas là ».

Direction

Vous voyez : les rapports, les KPI, le PPMS.

⚠ **Le PPMS ne remplace pas l'appel.** L'écran le dit lui-même, au-dessus du nombre. Il donne un point de départ, avec l'heure du portrait en évidence, et un cache local pour le cas où le réseau tombe — c'est la première chose qui tombe dans une urgence.

2. Les alertes du CDI — les trois sons et ce qu'ils veulent dire

⚠ **Il n'y a aucun fichier audio.** Les sons sont synthétisés dans le navigateur (`js/cdi/cdiData.js`). C'est pourquoi ils fonctionnent sur un poste hors ligne.

Son	Ce qu'on entend	Quand	Ce que vous faites
Succès	une note brève, aiguë (880 Hz)	entrée normale	rien
Sortie	une note plus grave (440 Hz)	sortie normale	rien

Son	Ce qu'on entend	Quand	Ce que vous faites
Erreur	double bip grave et sec (220 Hz, carré)	carte inconnue	la personne n'est pas au fichier — la pointer à la main, ou la signaler
Capacité	deux notes descendantes (520 → 390 Hz)	le seuil de capacité vient d'être franchi	la salle est pleine. La porte s'ouvre quand même — à vous de voir
Exclusion	grave → aigu → grave , plus fort que le reste	une personne exclue vient d'entrer	voir ci-dessous

⚠ **Le son d'exclusion commence par le grave, et c'est délibéré.** La première version ouvrait sur un aigu bref — c'est-à-dire sur ce qu'est le son de succès en entier. À un comptoir, on est déjà passé au suivant avant la deuxième note. Il est aussi **plus fort** que les autres : celui qui compte ne peut pas être au même volume que la routine.

Quand l'alerte d'exclusion sonne

L'écran affiche **le nom, la classe et la photo** — de quoi reconnaître la personne.

⚠ **Il n'affiche PAS le motif, et c'est voulu.** Le motif d'une exclusion est une donnée sensible sur un mineur, lisible seulement avec la permission dédiée — et l'écran du CDI est visible depuis le comptoir, par d'autres élèves.

Ce que vous faites : vous savez que cette personne ne devrait pas être là. Ce que vous en faites relève de vous et du règlement. La modale affiche la date de fin quand il y en a une (« jusqu'au 5 »), ce qui vous donne une phrase à dire sans rien révéler de la sanction.

Échap la ferme, et le bouton prend le focus — pas besoin de la souris.

3. Les flags de la cantine — le tableau à connaître

Flag	Ce que ça veut dire	Ce que vous faites
AVANT_CRENEAU	la personne est passée avant son créneau	rien d'urgent. Si ça se répète pour toute une classe, le planning est peut-être faux — voir le chapitre 5
APRES_CRENEAU	passée après son créneau	idem
passage trop court (< 15 min)	entrée puis ressortie très vite — elle n'a probablement pas mangé	ce n'est ni un refus ni une accusation . C'est un signal : elle est venue chercher quelqu'un, a renoncé à la file, ou le lecteur de sortie l'a attrapée en passant
séjour trop long (> 30 min)	elle est là depuis longtemps	c'est la colonne « Doit sortir ». C'est là qu'on agit
FORA_HORARIO	hors de la fenêtre de la cantine	idem AVANT / APRES, forme ancienne
POSTO_FIXO	quelqu'un qui travaille à ce point y repasse	rien. C'est du bruit attendu, écarté des écrans standard

Flag	Ce que ça veut dire	Ce que vous faites
JA_PRESENTE	entrée alors que la personne est déjà dedans	rien. La visite n'est pas rouverte
FECHAMENTO_AUTO	sortie posée automatiquement à l'heure de fermeture	rien. C'est le système qui range
EXCEDEU_TEMPO	le plafond de durée a été dépassé	idem « séjour trop long »

⚠ **Les deux dernières lignes du tableau sont des marques de RÉPÉTITION** (POSTO_FIXO , JA_PRESENTE) : ces passages ont bien eu lieu, ils sont bien enregistrés, ils sortent seulement des écrans standard et des compteurs. **Rien n'est effacé** — le Journal les montre, avec une lentille pour les filtrer.

4. Le flux des tentatives refusées — chaque motif, et le geste

C'est le panneau rouge, à droite du Moniteur et sur l'écran du portail.

⚠ **Un « refus » ici n'a fermé aucune porte.** C'est une décision **logique**, écrite pour que quelqu'un puisse la lire.

Motif	Ce que ça veut dire	Ce que vous faites
MEAL_NOT_ENTITLED	pas de droit au repas enregistré	vérifier avec la Vie Scolaire. Si c'est une erreur, l'écran Droits Repas la corrige
PENDING <i>(statut, pas un refus)</i>	⚠ la case n'a jamais été remplie	ce n'est PAS un refus. C'est l'état de 923 élèves le jour 1. À traiter comme une donnée manquante
DUPLICATE_MEAL	deuxième repas dans la journée	regarder. C'est souvent une double lecture, pas une fraude
OUTSIDE_MEAL_TIME	hors de la fenêtre	voir les flags ci-dessus
MEAL_SLOT_NOT_CONFIGURED	⚠ le système ne sait pas à quelle heure cette classe mange	ce n'est pas un refus : c'est un trou dans le planning. Signaler la classe à la Vie Scolaire — elle apparaîtra dans le contrôle de l'affiche
EXIT_NOT_AUTHORIZED	pas d'autorisation de sortie	c'est le motif qui compte au portail. Vérifier auprès de la Vie Scolaire
OUTSIDE_EXIT_WINDOW	autorisation existante, mais hors de sa plage horaire	idem
REGIME_NOT_ALLOWED	le régime annuel ne permet pas cette sortie	voir chapitre 3, les cinq verdicts
REGIME_TO_VERIFY	⚠ le système ne sait pas : ça dépend d'une heure de cours qu'il n'a pas	ce n'est pas une objection. L'écran vous dit quoi vérifier
REGIME_UNKNOWN	aucun régime enregistré pour cet élève	l'état normal tant que les régimes ne sont pas chargés. Ne laisse pas de trace, exprès
USER_INACTIVE	la personne est désactivée au fichier	vérifier : départ, ou erreur de saisie ?
UNKNOWN_USER	l'identifiant n'existe pas au fichier	probablement quelqu'un qui n'a jamais été importé

Motif	Ce que ça veut dire	Ce que vous faites
UNKNOWN_FACE	⚠ le portail n'a pas reconnu ce visage	voir le chapitre 2 : c'est le défaut ouvert du portail
AMBIGUOUS_NAME	le nom correspond à plusieurs personnes	le système refuse de choisir. Il a raison
MISSING_DOOR_MAPPING	l'IP du terminal n'est pas dans la table	problème technique : une IP a changé. Voir chapitre 6
DEVICE_DENIED	c'est le terminal qui a refusé, pas le MAGBO	validité expirée sur l'appareil, ou sous-type inconnu

⚠ **Trois de ces motifs disent « je ne sais pas », pas « non »** : `PENDING`, `MEAL_SLOT_NOT_CONFIGURED` et `REGIME_TO_VERIFY`. Les traiter comme des refus fait accuser des gens à la place d'une case vide. Les écrans les peignent différemment pour cette raison.

5. Les couleurs et ce qu'elles promettent

Une convention tient dans tout le système, et elle vaut la peine d'être connue :

Couleur	Ce que ça veut dire
Rouge	quelque chose que quelqu'un doit regarder maintenant
Ambre	le système ne sait pas, ou une situation à vérifier
Ardoise / gris clair	information de configuration — le genre qu'on apprend à ignorer
Vert	normal

⚠ C'est pourquoi `REGIME_UNKNOWN` est **ambre et non gris** : le gris clair est la couleur de `MISSING_DOOR_MAPPING`, que l'opérateur a appris à lire comme « affaire de technicien ». Un verdict qui a besoin d'être vu ne peut pas porter la couleur de ce qu'on ignore.

6. Trois choses à ne jamais conclure d'un écran

1. « **Le compteur dit 0, donc il n'y a personne.** » Non : ça peut vouloir dire que le cache n'est pas encore chargé. Les écrans affichent « — » pour « je ne sais pas » et « 0 » pour zéro — la différence est délibérée.
2. « **Il n'y a pas de ligne, donc ça n'est pas arrivé.** » Le registre des alertes du CDI (chapitre 3) n'écrit que quand l'écran est ouvert. `access_logs`, lui, ne dépend d'aucun écran — c'est là qu'il faut regarder.
3. « **L'écran est vide, donc le système est en panne.** » Sur un poste, une application vide vient presque toujours du lancement par le `.exe` au lieu du `.bat`. Voir chapitre 6.

7. Où regarder ensuite

Question	Document
Chaque écran, bouton par bouton	<code>docs/manual-utilisateur.md</code>
Le guide dédié de la cantine	<code>docs/operacional/guia-operador-cantina.md</code>
D'où vient une règle, et pourquoi	chapitre 3
Un problème technique (rien n'arrive, écran vide, heures fausses)	chapitre 6, et <code>docs/operacional/handoff.md</code> §7
Administrer les droits, les imports, le planning	chapitre 5

Chapitre 5 — Administration

Ce chapitre s'adresse à la personne qui **administre** MAGBO : celle qui crée les comptes opérateurs, charge les listes (élèves, personnels, droits repas, régimes, photos), tient le planning de la cantine et règle les valeurs du système. Il répond à trois questions : *qui a le droit de faire quoi, comment entrent les données sans les abîmer, et où se règle ce qui se règle*. Il ne couvre ni l'exploitation quotidienne (voir `docs/manual-utilisateur.md`) ni le déploiement (voir `docs/operacional/reconstruir-do-zero.md` et `docs/operacional/mise-a-jour-vm.md`).

5.1 Carte des écrans d'administration

Deux portes, et elles ne mènent pas au même endroit.

Porte	Comment on y entre	Ce qu'on y trouve
Panneau Administratif	Bouton Administration (cadenas) de la barre du haut → code PIN	Compteurs du jour, journal, synchronisation Pronote, <i>Gestion des opérateurs</i> , <i>Gestion des utilisateurs</i> , et les raccourcis vers les écrans de gestion
Engrenage ⚙	Icône engrenage de la barre du haut	Imports (Excel, HikCentral, personnels, photos), enregistrement manuel, réglages du poste, et depuis le 28/08 la « Configuration du système »

Les écrans de gestion (`Droits Repas` , `Sorties` , `Régimes` , `Planning Cantine` , `Exclusions CDI`) sont déclarés `hidden: true` dans `js/data/constants.js` : ils n'apparaissent pas sur le tableau de bord public. On y entre par le Panneau Administratif (`js/App.js:435-464` , les gestionnaires `onNavigateToMeal` , `onNavigateToMealSlots` , `onNavigateToExit` , `onNavigateToRegime` , `onNavigateToCdiExclusions`) — ou, pour un opérateur non-admin, par un **raccourci** posé sur son tableau de bord dès qu'il détient la permission correspondante (`js/utils/permissions.js` , fonction `podeVerPonto` , lignes 116-148).

⚠ Le raccourci n'apparaît **jamais** pour l'ADMIN : `mostraAtalhoNoDashboard` (`js/utils/permissions.js:97-101`) renvoie `false` pour lui, parce qu'il entre par le Panneau. Deux chemins vers le même écran, c'est un défaut corrigé derrière un seul des deux.

5.2 Opérateurs et permissions

5.2.1 Les trois étages du droit

Le système empile **trois** choses distinctes, et les confondre est la source d'erreur la plus fréquente :

1. **Le rôle** — `backend/.../security/Role.java` : deux valeurs seulement, `ADMIN` (« Vie Scolaire — accès total ») et `OPERATOR`. L'ADMIN passe **toutes** les gardes.
2. **Les secteurs** (`setoresPermitidos`, CSV) — `cantine`, `infirmerie`, `cdi`, `portail`, ou `*`. Ils gouvernent la **lecture** : quels écrans de poste et quels rapports on voit. La liste des cases est dans `js/components/UserManagement.js:317-322`.
3. **Les permissions** (`permissoes`, CSV) — elles gouvernent l'**écriture**, et quelques lectures particulièrement sensibles. La liste canonique vit dans `backend/.../security/Permissions.java`, constante `TODAS` (lignes 115-125).

La garde côté serveur s'écrit toujours de la même façon : `@PreAuthorize("hasRole('ADMIN') or @areaSecurity.hasPermission('X')")` (`backend/.../security/AreaSecurity.java:41-58`).

Où l'on crée et modifie un compte : **Panneau Administratif → Gestion des opérateurs** (`js/components/AdminDashboard.js:455-490`, écran `js/components/UserManagement.js`).

CAPTURE D'ÉCRAN ATTENDUE

le formulaire d'un opérateur — le rôle, les cases de secteurs, et la grille des permissions particulières juste en dessous

5.2.2 La liste complète des permissions

La colonne « Qui devrait l'avoir » est une **proposition**, pas une configuration existante : elle n'est écrite nulle part dans le code, et la répartition réelle des comptes de production n'est pas vérifiable depuis le dépôt (voir la fin de cette section).

Permission	Ce qu'elle ouvre	Où c'est gardé	Qui devrait l'avoir (proposition)
<code>MEAL_ENTITLEMENT_WRITE</code>	Modifier un droit repas (activer, retirer, dater), l'import en masse, l'historique	<code>MealEntitlementController</code> (4 routes, l. 62-128)	L'opérateur de la cantine et la Vie Scolaire. La Direction décide, l'opérateur exécute (ADR-004)
<code>EXIT_PERMISSION_WRITE</code>	Créer et révoquer une autorisation de sortie ponctuelle	<code>ExitPermissionController</code> : 75, 91	Vie Scolaire uniquement — c'est elle qui reçoit le mot des familles
<code>ATTEMPTS_READ</code>	Consulter les tentatives refusées et leurs agrégats (<code>/api/access/attempts</code> , <code>/stats</code>)	<code>AccessAttemptController</code> : 29, 44	Vie Scolaire, direction, et l'opérateur qui doit comprendre un refus au moment où il arrive
<code>REGIME_WRITE</code>	Écrire le régime de sortie annuel d'un élève (V014)	<code>StudentRegimeController</code> : 130-170	Vie Scolaire et elle seule : c'est elle qui tient le carnet signé par la famille. Le javadoc de <code>Permissions.java:13-22</code> le dit explicitement — l'ADMIN n'est pas le bon profil

Permission	Ce qu'elle ouvre	Où c'est gardé	Qui devrait l'avoir (proposition)
PPMS_READ	Lire la liste nominative de qui est à l'intérieur (évacuation)	PpmsController:51	Vie Scolaire, direction, infirmerie. Décision de Sam du 14/08 : restreindre, pas fermer — un nom est ce qui permet de retrouver un enfant
CANTINE_REMOVAL_WRITE	Retirer une ligne du Moniteur Cantine (et la remettre)	CantineRemovalController :62, 77	L'opérateur de la cantine. ⚠ voir l'avertissement ci-dessous
MEAL_SLOT_WRITE	Modifier le planning cantine : créneaux, classes, exceptions par élève	MealSlotController (ESCRITA , l. 47)	Vie Scolaire — c'est elle qui tient l'affiche au mur
PARCOURS_READ	Le parcours du jour d'une personne, tous points confondus	ParcoursController (GATE , l. 50)	Vie Scolaire et direction. Traverse toute l'école : plus que ce qu'aucun secteur ne donne
CONFIG_WRITE	Lire et écrire les réglages du système ; capacité et état du CDI ; classes dispensées de badge	SettingsController (GATE , l. 36), CdiController:173 , MealSlotController:172	Sam, la direction, et éventuellement la Vie Scolaire. C'est la carte complète du comportement du système
CDI_EXCLUSION_WRITE	Lire et gérer les exclusions du CDI (qui, pourquoi, jusqu'à quand, qui a décidé)	CdiController (GATE_EXCLUSOES , l. 51-52)	Vie Scolaire et responsable du CDI. Une exclusion nomme un enfant et raconte une sanction

Quelques règles qui ne se devinent pas :

- **Lecture par secteur, écriture par permission.** L'opérateur de la cantine ouvre le Moniteur parce qu'il a le secteur `cantine` ; il en *retire* une ligne parce qu'il a `CANTINE_REMOVAL_WRITE` . Deux choses différentes, et c'est voulu.
- **Trois exceptions** où la *lecture* passe par une permission : `PPMS_READ` , `PARCOURS_READ` , `CDI_EXCLUSION_WRITE` , et `CONFIG_WRITE` . Chacune porte son javadoc expliquant pourquoi (`Permissions.java` , l. 25-95). La raison commune : la donnée traverse l'école entière, ou elle nomme un mineur dans une situation qu'un secteur n'a pas à connaître.
- **Sans permission, les champs sont DÉACTIVÉS, jamais cachés** (`.claude/rules/frontend.md`). Exception assumée : l'engrenage lui-même, qui est *caché* aux profils qui n'y ont rien à lire — le commentaire de `js/components/Header.js:120-152` explique pourquoi (une porte dessinée qui ne s'ouvre pas n'apprend qu'à ne plus cliquer).
- *** existe mais n'est pas offert.** `SystemUser.hasPermission` l'accepte quand c'est la chaîne entière, et `SystemUserController.validatePermissions` le laisse passer — mais aucune case ne le coche à l'écran : il accorderait aussi les permissions **qui n'existent pas encore** (`Permissions.java:111-113` , `js/components/UserManagement.js:381-386`). Un compte qui l'a reçu par API voit un bandeau d'avertissement.

⚠ `CANTINE_REMOVAL_WRITE` ne suffit pas seule, et l'oubli ne produit aucune erreur visible. Le `@PreAuthorize` du `CantineRemovalController` exige aussi `@areaSecurity.can(#pointId)`. La permission est globale, le point ne l'est pas : sans la seconde moitié, quiconque l'obtient pourrait masquer une ligne du CDI ou du portail. Une **troisième** garde vit dans le service : le point doit appartenir à la cantine. Tout est expliqué dans le javadoc du contrôleur (`backend/.../controllers/CantineRemovalController.java:14-38`).

5.2.3 Ajouter une permission : les trois miroirs

Le nom d'une permission existe à **trois** endroits qui doivent bouger ensemble :

1. `backend/.../security/Permissions.java` — la constante **et** son ajout dans `TODAS` ;
2. `js/utils/permissions.js`, objet `PERMISSIONS` — c'est lui qui dessine les cases de l'écran opérateurs ;
3. la clé `i18n` `operadores.permissao.<NOM>` dans `js/utils/i18n.js` (FR et PT).

⚠ Oublier `TODAS` est le piège historique : la permission existe alors *uniquement* comme littéral dans un `@PreAuthorize`. L'admin ne peut pas l'accorder (« Permission invalide ») et l'écran concerné devient accessible au seul ADMIN — c'est-à-dire justement au profil qui ne devrait pas l'utiliser. C'est arrivé à `REGIME_WRITE`, rattrapé le 14/08/2026 (`Permissions.java:13-22` et `:99-113`).

5.2.4 Ce que le dépôt ne peut pas dire

[A COMPLETER PAR SAM] Quels comptes opérateurs existent réellement sur la VM de production, avec quel rôle, quels secteurs et quelles permissions ? En particulier : **qui détient** `CONFIG_WRITE` aujourd'hui, et **qui détient** `REGIME_WRITE` à la Vie Scolaire ?

[A COMPLETER PAR SAM] Le mot de passe du compte applicatif `admin` et la valeur de `ADMIN_PIN` sur la VM (les défauts `admin1234` / `1234` doivent avoir été changés — voir `.claude/rules/deploy-seguranca.md`). Où sont-ils consignés ?

[A VERIFIER] La liste vivante des comptes se lit avec un jeton ADMIN : `curl -s -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/system-users` (route `GET /api/system-users`, `SystemUserController:26`).

5.3 Les imports, et leurs pièges

5.3.1 La règle qui les gouverne tous : simuler, puis confirmer

Quatre imports (HikCentral, droits repas, régimes, photos) suivent le **même** dessin, et il n'est pas décoratif :

- `plan(...)` **n'écrit rien** et rend, ligne par ligne, ce qui se passerait ;

- `apply(...)` **refait le plan** contre l'état actuel de la base — il ne fait jamais confiance au plan que l'opérateur a vu à l'écran.

La raison est écrite dans `backend/.../services/HikCentralImportService.java:109-118` et répétée dans `UserPhotoService.java:25-33` : entre la vérification et la confirmation, quelqu'un a pu modifier une fiche. Appliquer un plan périmé, c'est écrire d'après ce qui n'est plus vrai.

Un vocabulaire commun aux quatre : **CRÉER · METTRE À JOUR · IGNORER · CONFLIT · À VÉRIFIER**.

Un **conflit n'est pas une panne** : c'est une ligne que le système **refuse** d'appliquer, avec le motif écrit — et les autres lignes passent quand même.

5.3.2 ⚠ Le piège n°1 : les zéros à gauche

Les matricules Pronote ont **7 chiffres avec des zéros en tête** (`0001764`). Excel les convertit en nombre et mange le zéro — et `1764` est **un autre élève**.

Le système se protège partout où il lit un fichier : `raw: false` dans les options `xlsx` (`js/utils/hikcentralSheet.js:80-...`, `js/utils/mealSheet.js:55-60`, `js/utils/regimeSheet.js:43-51`), et le nom de fichier photo comparé **comme texte** (`UserPhotoService.java:337-343`).

Mais **rien ne peut réparer un fichier déjà abîmé**. La discipline est donc en amont :

- formater la colonne matricule en **Texte** *avant* d'enregistrer le fichier ;
- **ne jamais ouvrir dans Excel** le CSV destiné au HikCentral — l'écran le dit en toutes lettres (`js/utils/i18n.js:400-403`, affiché par `AppSettingsModal.js:1291-1293`) ;
- l'éditer en éditeur de texte si besoin.

Voir aussi `docs/manual-utilisateur.md` §A.6 « Les zéros du début, dans Excel ».

5.3.3 Import Excel — élèves et responsables

Engrenage → « **Importer Excel** ». Le format complet (colonnes, types acceptés, ordre de traitement des responsables, comportement en cas d'erreur partielle) est déjà documenté : voir

`docs/IMPORT_TEMPLATE.md`. Les points à retenir :

- l'import **crée seulement** ; une ligne dont l'ID existe déjà est rejetée, jamais écrasée ;
- les types s'écrivent **en majuscules** : `ALUNO`, `PROFESSOR`, `FUNCIONARIO`, `RESPONSAVEL` ;
- les responsables sont traités en premier, même s'ils sont en bas du fichier.

Le cadastre des **élèves** vient normalement de **Pronote** (`PronoteSyncService`, huit colonnes obligatoires — voir `js/utils/regimeSheet.js:7-13`, qui les énumère). Cet import est un **filet**, pas le chemin normal.

5.3.4 Import HikCentral — relier les visages aux personnes

Engrenage → « **HikCentral** ». C'est ce qui donne à MAGBO le lien entre le visage enregistré dans le terminal et la fiche de la personne. **Sans ce lien, le terminal reconnaît la personne et MAGBO la refuse à chaque passage.**

Le fichier attendu est l'export « **Renseignements personnels** » du HikCentral, en `.xlsx`.

⚠ **L'EN-TÊTE EST À LA LIGNE 9.** Les huit premières lignes sont des instructions du HCP lui-même. Le lecteur démarre à `range: 8` (0-indexé) — `js/Utils/hikcentralSheet.js:17-24`. Sans cela les colonnes s'appelleraient `__EMPTY_3` et le fichier entier serait lu comme du bruit (`AppSettingsModal.js:340-357`). La première ligne de données est donc la **10**, et c'est ce numéro-là que le rapport d'import affiche (`HikCentralImportService.java:143`).

Les colonnes lues — `ID`, `Prénom`, `Nom de famille`, `Service` — sont casées **par leur nom**, jamais par leur position : le HCP réordonne et ajoute des colonnes entre versions, et une position fixe casserait sans prévenir (`hikcentralSheet.js:38-53`). Les colonnes de validité et de niveau d'accès sont ignorées : ce sont des affaires du HCP (ADR-004, « le HCP est du provisionnement pur »).

Les lignes « À vérifier ». Un élève dont l'ID HikCentral n'est pas la matricule (neuf cas dans l'export réel) part en **révision humaine** : seul le nom fait le lien, et rapprocher automatiquement par le nom reviendrait à donner le visage d'un enfant à un autre (`HikCentralImportService.java:180-190`). Le pas-à-pas de l'écran, avec les deux encadrés vert/rouge à lire avant de confirmer, est dans `docs/manual-utilisateur.md` §14.2.

Le chemin inverse (F7b). Le même onglet génère le **CSV à importer dans le HCP** pour les élèves qui n'ont pas encore de visage lié — `HikCentralCsvService`, boutons en `AppSettingsModal.js:1296-1314`. Tous les champs sortent entre guillemets, précisément pour dire « ceci est du texte ».

⚠ Le **template exact du HCP** reste une inconnue. Le générateur réutilise, faute de mieux, les colonnes de l'*export* que MAGBO sait déjà lire, et l'encadré du javadoc le déclare (`HikCentralCsvService.java:23-38`). Le nom de la colonne de classe (`"Classe"`) est une **supposition assumée, isolée dans une constante** pour qu'un seul mot ait à changer (`HikCentralCsvService.java:63-82`).

Le cycle complet — MAGBO génère le CSV → l'informatique l'importe → *Apply to Device* → **0 échec** — et les six questions encore ouvertes avec le SI sont dans `docs/operacional/procedimento-hikcentral.md` (§1, §2 et la liste finale des pendências).

5.3.5 « Restaurer les paramètres par défaut » doit rester DÉCOCHÉ

 DANS LE HIKCENTRAL, POUR L'EXPORT DES PERSONNES ET POUR CELUI DES PHOTOS : LA CASE « RESTAURER LES PARAMÈTRES PAR DÉFAUT » DOIT RESTER ****DÉCOCHÉE****.

Cochée, elle remet la sélection de champs du HCP à son état d'usine. Le fichier qui en sort n'a plus les mêmes colonnes ni la même mise en page — et l'import de MAGBO, qui cherche son en-tête à la LIGNE 9 et ses colonnes PAR LEUR NOM, ne reconnaît plus rien. L'écran répond « Feuille non reconnue » et, dans le meilleur des cas, on perd une demi-heure ; dans le pire, on croit que l'export est vide.

Origine de cette consigne : Sam, le 28/08/2026. Elle n'est **pas** vérifiable dans le dépôt — aucun fichier du projet ne mentionne cette case, qui appartient à l'interface du HikCentral et non à MAGBO.

[A VERIFIER] Ouvrir le HikCentral (192.168.1.90) → module Personnes → **Exporter**, et localiser la case exacte ainsi que l'écran où elle apparaît (export des renseignements, export des photos, ou les deux). Noter la formulation exacte dans la langue de l'installation, puis la recopier ici.

[A COMPLETER PAR SAM] La consigne vaut-elle aussi pour l'**import** dans le HCP (*Apply to Device*), ou uniquement pour les deux exports ?

5.3.6 Import du personnel

Deux chemins, tous deux dans l'engrenage :

- « **Importer les personnels** » — un `.xlsx` avec `nome`, `hikvision_employee_id`, `tipo`, `departamento`, `matricula` ; la matricule laissée vide reçoit le prochain `FUNC-###` (`js/utils/i18n.js:539-541`).
- « **Enregistrement manuel** » — une personne à la fois ; l'écran affiche à l'avance la matricule qui sera émise si le champ reste vide (`AppSettingsModal.js:101-110`), parce que sans cela l'opérateur ne peut pas savoir ce que le système va graver.

L'onglet « **Personnels** » liste, cherche, édite, désactive — et porte l'outil « **C'est un élève** », qui transfère un visage vers la bonne fiche d'élève et désactive l'enregistrement `FUNC-###` créé par erreur. Les règles qu'il respecte et qu'il ne faut pas assouplir (les passages restent sur l'ancien enregistrement, confirmation explicite si l'élève a déjà un visage, jamais de rapprochement automatique par le nom) sont dans `docs/operacional/handoff.md` §2.7.

5.3.7 Droits repas

Panneau Administratif → Droits Repas (ou raccourci avec `MEAL_ENTITLEMENT_WRITE`). Édition unitaire, historique, et **import en masse** d'un `.xlsx`.

Colonnes acceptées (FR, PT et le nom du champ système, pour qu'un fichier ré-exporté depuis MAGBO revienne sans traduction) — `js/utils/mealSheet.js:47-53` : `Matrícula/Matricule/ID` · `Status/Statut/Droit` · `Valable du` · `Valable au` · `Note`.

⚠ **Un élève absent de MAGBO n'est jamais créé ici.** La fiche élève vient de Pronote ; créer produirait un « élève » sans nom, sans classe et sans responsable, pour satisfaire une ligne de tableur (`MealEntitlementImportService.java:39-43`).

⚠⚠ **PENDING n'est pas « refusé », mais en production il finit par l'être.** Un élève sans ligne de droit est `PENDING` — donnée non renseignée. Or la production met `magbo.policy.meal-pending=DENY` (décision D5, ADR-004). **Conséquence opérationnelle : la liste des autorisés doit être chargée en masse AVANT le premier service**, sinon tous les `PENDING` sont refusés le jour 1. C'est écrit dans `docs/operacional/procedimento-hikcentral.md` §4 et repris dans `docs/manual-utilisateur.md` §A.1.

5.3.8 Régimes de sortie

Panneau Administratif → Régimes (ou raccourci avec `REGIME_WRITE`). Le régime est le droit **annuel** de sortir, déclaré par écrit par les responsables légaux (circulaire n° 96-248). Il ne remplace pas les autorisations ponctuelles de l'écran *Sorties* : le régime est la règle de l'année, l'autorisation est l'exception du jour, **et l'exception l'emporte**.

⚠ **Pronote n'apporte pas le régime**, et l'import Excel des élèves non plus. Vérifié dans `PronoteSyncService#processLine` ; le constat est écrit en tête de `js/utils/regimeSheet.js:7-13` et de `RegimeImportService.java:24-29` . Le chemin bon marché a été cherché : il n'existe pas. Cette feuille **est** le chemin.

Colonnes acceptées — `js/utils/regimeSheet.js:31-41` : `Matricule` · `Régime général` · `Régime de sortie` · `Valable du` · `Valable au` · `Autorisé par` · `Document` · `Signé le` · `Note` .

⚠ **Autorisé par** (`authorized_by_family`) est **obligatoire en base** (V014). Une ligne sans auteur affirmerait qu'un enfant peut sortir seul sans que personne ne l'ait dit. Un régime n'est **jamais supprimé** : le remplacer clôt l'ancienne ligne (`encerrado_em`) et en ouvre une nouvelle — les deux restent, parce qu'un régime est une preuve (`.claude/rules/database.md` , section `student_regimes`).

L'utilité de l'import en masse est arithmétique : un élève à la fois, ce sont **923 opérations en septembre**, et le module ne sort jamais du jour 1 (`RegimeImportService.java:18-22`).

5.3.9 Photos

Voir §5.6 — ce sont des photos de mineurs et les règles sont d'une autre nature.

5.4 Le planning cantine et l'affiche imprimable

5.4.1 Pourquoi cet écran existe

Le 25/08/2026, la cantine a produit **63** `OUTSIDE_MEAL_TIME` sur **22 classes** qui mangeaient pourtant à l'heure juste : les fenêtres de `class_schedules` dataient de 2025 et l'affiche du mur avait changé. La cause n'était pas une faute de saisie — c'est que **le planning n'avait pas d'écran**, et qu'en changer demandait du SQL à la main sur la VM. Tout est exposé dans `docs/architecture/decisoes/ADR-005-creneaux-cantine.md`.

Depuis V021, trois tables — `meal_slots`, `meal_slot_classes`, `meal_slot_students` — sont la **seule source de vérité** de la fenêtre d'accès au réfectoire.

5.4.2 L'écran

Panneau Administratif → Planning Cantine (ou raccourci avec `MEAL_SLOT_WRITE`). Fichier : `js/components/MealSlotManagement.js`.

Ce qu'on y fait :

Geste	Où	Permission
Créer un créneau (jour + heure)	Bloc « nouveau créneau »	<code>MEAL_SLOT_WRITE</code>
Attacher / détacher une classe, ou tout un préfixe	Pastilles du créneau	<code>MEAL_SLOT_WRITE</code>
Régler la tolérance – avant / + après (minutes)	Sur la carte du créneau	<code>MEAL_SLOT_WRITE</code>
Éditer le rotulo (le libellé imprimé)	Champ en haut de la carte	<code>MEAL_SLOT_WRITE</code>
Exception par élève	<code>POST/DELETE /api/admin/meal-slots/{slotId}/eleve/{userId}</code>	<code>MEAL_SLOT_WRITE</code>
Classes dispensées de badge	Bloc rouge en bas	⚠ <code>CONFIG_WRITE</code> , pas <code>MEAL_SLOT_WRITE</code>

⚠ **La dispense est gardée plus fort que le reste du planning, et c'est délibéré.** Une classe dispensée disparaît du Moniteur **et cesse de compter dans le PPMS**. Déplacer une classe de créneau, c'est du planning ; ne plus la compter dans un calcul d'évacuation, non. Le javadoc de `MealSlotController:160-172` le dit, et l'avertissement est écrit **en toutes lettres à côté du réglage** — c'est le seul endroit où la personne qui décide le lira au moment de décider (`MealSlotManagement.js:216-263`).

L'écran **montre les désaccords, il ne les tranche pas** : classes sans créneau, classes du planning sans aucun élève. C'est la Vie Scolaire qui tient le mur.

5.4.3 L'affiche

Bouton **imprimante** en haut à droite → la grille d'édition est remplacée par l'affiche, puis **Imprimer** (`MealSlotManagement.js:119-133` ; composant `js/components/AfficheCantine.js`).

- L'affiche **sort de la configuration du moment** : on change les créneaux à l'écran et on réimprime. C'est tout l'intérêt — tant que le mur et la base étaient deux documents séparés, ils ont divergé.
- **En couleur, fidèle au mur** : Terminale en saumon, 1ère et 2nde en bleu, collège en blanc à liseré gris. Le code couleur porte du sens, et le changer casserait une lecture que toute l'école a apprise (`AfficheCantine.js:18-46`).
-  `print-color-adjust: exact` (avec son préfixe `-webkit-`) **n'est pas de la décoration** : sans cette ligne, les navigateurs suppriment les fonds colorés à l'impression et tout sort en gris. C'est exactement pourquoi les réimpressions étaient ternes (`AfficheCantine.js:10-16`).
- **Une page par passage, en A4 paysage** : 12H30 et 13H00 sont deux affiches distinctes au mur.
- Aucune bibliothèque PDF : c'est l'impression du navigateur.

Le rotulo est éditable depuis le 28/08/2026. Avant, la page 11h imprimait « REPRIS DE CLASS_SCHEDULES » — un nom de table interne — sur une page lue par les familles de 25 classes, et **aucun champ de l'écran ne permettait de le corriger**. Le champ est maintenant sur la carte du créneau ; l'endpoint l'acceptait déjà (`MealSlotManagement.js:289-302` , `MealSlotController:184`). Le défaut et sa correction sont racontés dans `docs/operacional/nuit-27-28-08-rapport.md` (Chantier 3).

CAPTURE D'ÉCRAN ATTENDUE

la carte d'un créneau, avec le champ rotulo, les deux tolérances et les pastilles de classes

CAPTURE D'ÉCRAN ATTENDUE

l'aperçu d'impression de l'affiche — bandeau bleu foncé, pastilles en couleur, une page par passage

5.4.4 Le contrôle à faire à chaque rentrée

Quatre requêtes SQL disent où le planning **ne correspond pas** à la réalité des élèves, sans rien corriger — parce que seule la Vie Scolaire peut trancher. Elles sont prêtes à l'emploi :

```
docker exec magbo-postgres psql -U magbo -d magbodb -f - < docs/operacional/controle-affiche-cantine.sql
```

Le document `docs/operacional/controle-affiche-cantine.md` explique chacune, avec les réponses mesurées le 26/08/2026. À relancer **avant le premier service de chaque rentrée** : c'est le moment où les codes de classe changent, et le seul où ces listes sont faciles à corriger.

 La liste B (« classes qui mangent sans figurer dans aucun créneau ») **doit rester vide**. Le jour où une vraie classe y apparaît, ses élèves mangent sans horaire connu tant que personne ne l'ajoute.

5.5 L'écran de configuration a DÉMÉNAGÉ le 28/08/2026

Il n'est plus dans le Panneau Administratif.

Chemin exact : barre du haut → **icône engrenage** ⚙ → première entrée de la colonne de gauche, « **Configuration du système** ».

Visible avec : rôle **ADMIN** ou permission **CONFIG_WRITE** .

Ce qui a changé, et pourquoi :

- L'engrenage était réservé à l'ADMIN. Il s'ouvre désormais aussi au porteur de **CONFIG_WRITE** qui n'est pas admin — et cette personne y voit **une seule** entrée, la configuration ; aucune des sept autres (imports, personnels, photos, enregistrement, réglages généraux) (`js/components/Header.js:145-170` , `js/components/AppSettingsModal.js:31-42`).
- La garde est posée sur le **contenu**, pas seulement sur les boutons de la barre latérale : un `setActiveTab` forcé depuis la console ne rend rien (`AppSettingsModal.js:1963-1974`).
- **Le card du Panneau Administratif a été SUPPRIMÉ, pas transformé en raccourci.** Il n'avait vécu qu'un jour (créé le 27/08, retiré le 27/08). Un raccourci laisserait deux chemins vers le même écran, et le prochain défaut serait corrigé derrière un seul des deux. Le point d'accès `SYSTEM_CONFIGURATION` , sa route et son raccourci ont disparu avec lui — le commentaire qui l'explique est resté à sa place dans `js/data/constants.js:35-40` , précisément pour que personne ne le recrée. Récit complet : `docs/operacional/nuit-27-28-08-rapport.md` , Chantier 1.

Ce qu'on y règle

Le catalogue est déclaré **une seule fois**, dans `backend/.../services/SettingsCatalog.java` (méthode `entradas()` , l. 72-102). Il est construit à chaque appel pour que les défauts soient lus **maintenant**, à leur source réelle, et non figés au démarrage :

Domaine	Clés
cantine	<code>magbo.cantine.duracao-curta-minutos</code> , <code>...duracao-maxima-minutos</code> , <code>...decantacao-minutos</code> , <code>...sortis-visiveis-minutos</code> , et les classes dispensées
cdi	<code>magbo.cdi.capacidade</code> , <code>magbo.cdi.estado</code> , <code>...estado-inicio</code> , <code>...estado-fim</code> , <code>...estado-nota</code>

Chaque ligne dit **trois** choses et pas une : ce qui s'applique maintenant, ce qui s'appliquerait sans intervention (le défaut), et **qui a décidé autrement** (`js/components/SystemConfiguration.js` , en-tête et l. 224-240).

- « **Rétablir le défaut** » est un **bouton**, pas une manœuvre : vider la valeur supprime la ligne en base et le code reprend la main. C'est le contrat de la V024, et il ne vaut que si l'on peut y revenir sans savoir quelle était la valeur d'origine.
- **Une clé au défaut n'a pas de ligne** dans `system_settings` . C'est pourquoi l'écran lit `/catalogue` et non `GET /` : un écran construit sur les seules lignes gravées afficherait une liste **vide** sur une base neuve, et on en conclurait qu'il n'y a rien à configurer (`SettingsController:43-52`).
- **Les orphelins** — une clé gravée qui n'est plus déclarée — apparaissent en ambre, dans un domaine à part (`SettingsCatalog.java:187-196`). Ils n'ont pas de défaut : plus rien ne les déclare.

- **⚠ Aucun secret ne passe par là.** Jetons, mots de passe et clé JWT vivent dans l'environnement (`.env` de la VM, `setx` du PC). Un écran qui les afficherait les mettrait sur une capture d'écran le jour même.

CAPTURE D'ÉCRAN ATTENDUE

l'écran Configuration du système — une ligne avec sa valeur, son défaut écrit à côté, « modifié par ... le ... », et les deux boutons Enregistrer / Rétablir

Ce qui n'est PAS dans cet écran

Les **politiques** `magbo.policy.*` se règlent par propriétés et non à l'écran. Défauts lus dans `backend/.../config/PolicyProperties.java:26-39` :

Politique	Défaut du code	Effet
<code>meal-not-entitled</code>	DENY	Refus enregistré, aucun <code>access_log</code>
<code>meal-pending</code>	OBSERVATION	⚠ la production met DENY — décision D5, ADR-004
<code>outside-meal-time</code>	OBSERVATION	Trace, sans refus
<code>meal-slot-not-configured</code>	OBSERVATION	Classe sans créneau : on laisse passer
<code>duplicate-meal</code>	OBSERVATION	
<code>exit-not-authorized</code>	DENY	
<code>user-inactive</code>	DENY	
<code>missing-door-mapping</code>	FALLBACK	

OBSERVATION enregistre **le passage et la tentative** ; DENY n'enregistre que la tentative. Rappel structurel : MAGBO **n'ouvre ni ne ferme aucune porte** (ADR-003) — un DENY est une qualification, pas un blocage physique.

[A VERIFIER] La valeur réellement appliquée en production se lit dans la ligne INFO du démarrage (`PolicyProperties.java:51`) : `docker logs magbo-backend | grep "MAGBO policies"`.

L'inventaire de ce qui reste écrit en dur, avec sa priorité et la raison de ne PAS convertir certaines valeurs, est dans `docs/operacional/inventaire-configurabilite.md`.

5.6 Les photos d'identité — ce sont des photos de MINEURS

Ces règles ne sont pas des recommandations. Elles sont écrites dans le code, testées, et rappelées dans `.claude/rules/backend.md` et `.claude/rules/deploy-seguranca.md`. Le contrat est énoncé en tête de `backend/.../services/UserPhotoService.java:35-47` et de `backend/.../controllers/UserPhotoController.java:26-44`.

Les cinq promesses

1. **Lecture authentifiée, une photo à la fois.** `GET /api/users/{userId}/photo` est `@PreAuthorize("isAuthenticated()")`, avec ETag (SHA-256) et `Cache-Control: private`, 30 minutes (`UserPhotoController.java:55-99`). `private` et jamais `public` : « public » autoriserait un proxy du réseau de l'école à stocker des portraits d'élèves et à les servir à qui les demande. ⚠ La page ne peut pas faire `` — le navigateur n'envoie pas l'en-tête `Authorization` dans une balise `<img>`. `js/Utils/photoCache.js` récupère l'image par `fetch` avec le jeton et construit un `objectURL`. **Ouvrir l'endpoint « juste pour le kiosque » publierait un catalogue de visages d'enfants sur le réseau de l'école.**
2. **Aucun export en masse.** Il n'existe pas d'endpoint qui rende l'ensemble, pas de ZIP de sortie, pas de CSV avec les images. `.claude/rules/backend.md` précise qu'un test casse si quelqu'un en crée un.
3. **Aucun octet d'image en log.** Les lignes de log portent matricule, nom de fichier et compteurs — jamais de contenu, pas même « les premiers octets pour déboguer » (`UserPhotoService.java:126-131`).
4. **Les images de la caméra restent jetées.** `faceImage`, `backgroundImage`, `faceLibImage` du webhook ne touchent jamais cette table. Seul entre ici un fichier qu'un opérateur a choisi et envoyé exprès.
5. **La suppression est définitive.** `DELETE` réel, sans effacement logique (`UserPhotoService.delete`, l. 134-141). Supprimer une fiche personnel efface sa photo dans la **même transaction** (`StaffAdminService.deleteStaff`) : un portrait orphelin attaché à un identifiant disparu, c'est une image que personne ne retrouve pour l'effacer et qui survit à tous les sauvegardes suivantes.

L'import (engrenage → « Photos »)

- **Le nom du fichier fait le lien** : `0004048.jpg` = la matricule, ou l'identifiant Hikvision. **Les zéros de gauche comptent** — la comparaison se fait comme texte (`UserPhotoService.java:331-343`).
- **Le format est décidé par les octets** (JPEG `FF D8 FF`, PNG, WebP), jamais par l'extension ni par le `Content-Type`, qui sont tous deux déclarés par l'expéditeur : un `.exe` renommé `.jpg` passerait les deux (`UserPhotoService.java:319-329` et `tipoPorConteudo`).
- **Deux fichiers pour la même personne dans un lot → CONFLIT sur les deux, aucun appliqué.** Appliquer le premier reviendrait à laisser l'ordre alphabétique d'un dossier décider quel est le visage de cette personne — et dans un portail, le mauvais visage sur la bonne fiche, c'est une sortie autorisée au nom d'un autre (`UserPhotoService.java:206-215`).
- **Simulation d'abord**, comme les autres imports ; `apply` refait le plan.
- Plafonds : **2 Mo par image** (`magbo.photos.max-bytes`), **2000 fichiers par lot** (`magbo.photos.max-arquivos-por-lote`) ; pour un ZIP, **5000 entrées** et **256 Mo décompressés**, avec le plafond par entrée vérifié **pendant** la lecture — protection contre la bombe zip (`PhotoZipReader.java:47-56, 94-141`).
- ⚠ **Le ZIP entre par corps brut** (`consumes="application/zip"`), pas en multipart : les limites `spring.servlet.multipart.*` ont été mesurées pour le webhook des **caméras**, et les desserrer pour un écran d'administration reviendrait à toucher le nombre qui protège le chemin le plus critique du système.

- Après un import en masse, appeler `MagboPhotoCache.clear()` (rechargement de l'application), sinon les personnes qui viennent de recevoir une photo gardent leurs initiales (`.claude/rules/frontend.md`).

Où elles vivent, et pourquoi

Dans **PostgreSQL**, table `user_photos` (V011), colonne `bytes` `BYTEA`. La raison est vérifiable :

`deploy/docker-compose.yml` monte **un** volume dans le conteneur du backend (`../backend/target:/app`), qui est la sortie de Maven — `mvn clean` l'efface et chaque build la réécrit.

Une photo sur disque ne survivrait pas au déploiement lui-même. En base, elles entrent gratuitement dans le `pg_dump` qui existe déjà : **la sauvegarde de la base EST la sauvegarde des photos.**

⚠ `user_photos` est la première table dont la donnée n'existe **nulle part ailleurs**. Le rollback `R011` **efface les images** ; les restaurer exige le dump précédent ou une réimportation des fichiers.

5.7 Migrations et état de la base

27 migrations, `deploy/migrations/V001` à `V027`, avec un `rollback/` pour toutes **sauf** V006, V008, V009 et V023 — soit 23 fichiers dans `rollback/` (comptés le 03/09/2026 : `ls deploy/migrations/V0*.sql | wc -l` → 27, `ls deploy/migrations/rollback/*.sql | wc -l` → 23). Pour V023 c'est normal : c'est un *seed*, et ses lignes partent avec `R021` — c'est documenté dans `deploy/migrations/README.md`. Flyway n'est **pas** adopté ; la VM applique les fichiers dans l'ordre, à la main, avant de démarrer le backend.

⚠ Le piège qui ne se voit **que sur la VM** : avec `ddl-auto=update`, Hibernate crée un CHECK à la création d'une table mais ne le **modifie jamais** ensuite. Ajouter une valeur à un enum Java (par exemple `DenialReason`) fait donc échouer l'INSERT **uniquement en production** — le PC ne bouge pas non plus, et les tests recréent H2 à zéro et restent verts. C'est pourquoi V009, V015 et V022 existent : elles élargissent le CHECK de `access_attempts.denial_reason`. Même piège pour `meal_entitlement_events.source` et `student_regime_events.source`, qui sont des CHECK **manuels** (`.claude/rules/database.md`).

[A VERIFIER — état au 03/09/2026, non vérifiable dans le dépôt] V001 à V027 seraient toutes appliquées sur la VM de production (V027 = `licence_clock`, posée au déploiement de la licence le 01/09). Le confirmer en listant les objets attendus, par exemple :

```
docker exec magbo-postgres psql -U magbo -d magbodb -tAc \
"SELECT table_name FROM information_schema.tables
WHERE table_schema='public' ORDER BY 1;"
```

(on doit y trouver `access_attempts`, `meal_entitlements`, `meal_entitlement_events`, `student_exit_permissions`, `user_photos`, `student_regimes`, `cantine_removals`, `meal_slots`, `system_settings`, `cdi_exclusions`, `cdi_alert_events`.)

[A VERIFIER – état affirmé par Sam le 28/08/2026] Six terminaux en service : portail .166 (SORTIE) et .167 (ENTRÉE), CDI .15 et .16, cantine .10, .12, .13, .14 — soit **huit** adresses pour six terminaux ; l'écart n'est pas expliqué par le dépôt. À confirmer par :

```
docker exec magbo-postgres psql -U magbo -d magbodb -tAc \
"SELECT terminal_ip, door_no, reader_no, point_id, action, label
FROM door_mappings WHERE ativo ORDER BY terminal_ip;"
```

⚠ **Les IP dansent en DHCP et cassent en silence** la « Écoute HTTP » du terminal **et** les `door_mappings` : aucune erreur, les événements cessent simplement d'arriver ou de correspondre. Avant toute séance matériel : IP du serveur, IP au écran du terminal, URL de l'Écoute HTTP, `door_mappings` (`.claude/rules/hikvision.md`).

[A COMPLETER PAR SAM] Les réservations d'IP demandées au SI par Fabiano ont-elles été exécutées ? Si oui, quelles adresses sont fixes (terminaux + VM) ? (*pendência 6 de* `docs/operacional/procedimento-hikcentral.md`)

5.8 Le calendrier de l'administrateur

Quand	Quoi	Où c'est écrit
À chaque rentrée, avant le 1er service	Relancer les 4 requêtes de contrôle de l'affiche ; corriger les créneaux ; réimprimer l'affiche	<code>docs/operacional/controle-affiche-cantine.md</code>
Avant le 1er service	Charger la liste des autorisés repas en masse (sinon tout <code>PENDING</code> est refusé)	§5.3.7 ; <code>procedimento-hikcentral.md</code> §4
Septembre	Charger les régimes de sortie en masse	§5.3.8
À chaque arrivée / départ	HikCentral : provisionner, <i>Apply to Device</i> , vérifier 0 échec ; puis import HikCentral dans MAGBO	<code>procedimento-hikcentral.md</code> §1-2
Après chaque import de photos	Recharger l'application (cache des photos)	§5.6
Avant toute séance matériel	Contrôle des IP et des <code>door_mappings</code>	<code>.claude/rules/hikvision.md</code>
Avant toute mise à jour	Sauvegarde de la base — c'est aussi la sauvegarde des photos	<code>docs/operacional/mise-a-jour-vm.md</code> §2

5.9 Défauts connus de la documentation elle-même

À signaler, pas à corriger sans décision :

- **⚠ Deux fichiers portent le numéro ADR-005** dans `docs/architecture/decisoes/` : `ADR-005-creneaux-cantine.md` (26/08/2026, le planning de cantine) et `ADR-005-totvs-rastreabilidade-no-dono-do-dado.md` (14/08/2026, la traçabilité TOTVS). Deux décisions réelles et distinctes, un seul numéro. Toute référence à « ADR-005 » est donc ambiguë : **citer le nom de fichier complet**, jamais le seul numéro.
- `docs/manual-utilisateur.md` §14 annonce « **six** onglets » dans l'engrenage. Il y en a **huit** pour un ADMIN (`config` , `import` , `hikcentral` , `staff-list` , `staff-import` , `photos` , `manual` , `general` — `AppSettingsModal.js:1896-1957`) et **un** pour un porteur de `CONFIG_WRITE` seul. Le manuel est antérieur au déménagement du 28/08.
- `docs/IMPORT_TEMPLATE.md` est en portugais alors que le reste de la documentation d'administration est en français ; son contenu reste exact.

5.10 Où aller ensuite

Sujet	Document
Chaque écran, pas à pas, pour l'opérateur	<code>docs/manual-utilisateur.md</code>
État opérationnel réel, dettes ouvertes, signatures de lecture	<code>docs/operacional/handoff.md</code>
Reconstruire de zéro / restaurer une sauvegarde	<code>docs/operacional/reconstruir-do-zero.md</code>
Mettre à jour la VM existante	<code>docs/operacional/mise-a-jour-vm.md</code>
Installer l'application sur un poste	<code>docs/operacional/guide-installation-postes.md</code>
Publier une nouvelle version portable	<code>docs/operacional/release-portable.md</code>
Cycle de vie des personnes dans le HikCentral	<code>docs/operacional/procedimento-hikcentral.md</code>
Ce qui reste écrit en dur, et pourquoi	<code>docs/operacional/inventaire-configurabilite.md</code>
Les décisions et leurs raisons	<code>docs/architecture/decisoes/</code>
Règles de code par domaine	<code>.claude/rules/*.md</code>

Chapitre 6 — Exploitation

Ce chapitre s'adresse à la personne qui ouvre le livre en urgence : le système ne répond plus, il faut déployer une version, appliquer une migration, restaurer une sauvegarde ou remettre l'application sur un poste. Il est fait pour être **exécuté**, pas lu — chaque commande a été relevée dans le dépôt, et ce qui n'a pas pu être vérifié depuis le dépôt est marqué comme tel.

Référence de cette rédaction : le **03/09/2026**, sur `main` augmentée des trois chantiers de cette date (`fix/adresse-du-login`, `feat/livre-imprimable`, `docs/verifications-finales`) ; l'état d'avant eux était `fc4359c`. Suites au vert sur cet état, mesurées à froid : backend **1055** (0 échec, exactement 2 `@Disabled`), npm **889**. ⚠ L'ancre est une **date**, pas un SHA : ces chantiers seront fusionnés par Sam et les SHA changeront.

1. Ce qui tourne sur la VM

Le déploiement canonique est `deploy/docker-compose.yml`. **Deux conteneurs, pas plus :**

Conteneur	Image	Rôle	Ports
<code>magbo-postgres</code>	<code>postgres:16-alpine</code>	la base <code>magbodb</code> , volume nommé <code>magbo_pg_data</code>	<code>127.0.0.1:5432</code> — localhost uniquement
<code>magbo-backend</code>	<code>eclipse-temurin:17-jre-alpine</code>	l'API Spring Boot, profil <code>prod</code>	<code>8080:8080</code>

Trois faits qui expliquent tout le reste du chapitre (`deploy/docker-compose.yml`, lignes 26-130) :

- L'image du backend ne contient pas l'application.** C'est un JRE nu ; le `.jar` arrive par un volume, `../backend/target:/app`, et la commande du conteneur est `java -jar access-control-*.jar`. Mettre à jour = **reconstruire le jar sur l'hôte, puis recréer le conteneur**. Il n'y a aucune image à construire ni à pousser. Conséquence directe : `mvn clean` vide le répertoire monté, et le joker `*.jar` devient ambigu s'il reste deux versions dans `target/`.
- Rien de durable ne doit être écrit sur disque par le backend** — `/app` est la sortie de Maven, effacée à chaque build. C'est la raison pour laquelle les photos d'identité vivent dans PostgreSQL (`user_photos`, V011) et non dans un dossier : elles entrent ainsi dans le `pg_dump` qui existe déjà.
- La VM ne sert que le backend.** Le tableau de bord est une application Electron installée poste par poste, mise à jour par un paquet séparé (§ 6). Un backend à jour avec des postes restés en arrière **n'est pas une panne visible** : l'écran affiche l'ancienne version sans le dire.

Ce que le compose ne fait **pas**, et qu'il faut savoir : pas de HTTPS (HTTP simple, un reverse proxy reste à poser), CORS ouvert (`*`), et le port 8080 exposé sur toutes les interfaces. La liste complète est la « Security checklist before production » de `deploy/README.md`.

⚠ `deploy/setup-server.sh` et `deploy/smoke-tests.md` décrivent un AUTRE déploiement — systemd, PostgreSQL installé par apt, un jar nommé `magbo-access-1.0.0.jar`. Ce nom n'a jamais existé : le `pom.xml` produit `access-control-1.0.0.jar` (`backend/pom.xml`, `artifactId` = `access-control`, version 1.0.0), et `DB_NAME` y vaut `magbo_db`, qui n'est aucune des deux bases réelles. Ces deux fichiers sont **historiques**. Le chemin canonique est `deploy/docker-compose.yml`. [A COMPLETER PAR SAM] Ces deux fichiers peuvent-ils être supprimés du dépôt, ou décrivent-ils encore une installation quelque part ?

2. Le `.env` — variable par variable

Le fichier vit dans `deploy/` (il est dans le `.gitignore` : `git pull` ne l'écrase pas et ne le recrée pas non plus). Modèle : `deploy/.env.example`.

Variable	Ce qu'elle fait	Ce qui casse si elle manque
<code>POSTGRES_DB</code>	nom de la base (<code>magbodb</code>)	valeur de repli <code>magbodb</code> — sans danger
<code>POSTGRES_USER</code>	rôle Postgres (<code>magbo</code>)	repli <code>magbo</code>
<code>POSTGRES_PASSWORD</code>	mot de passe du rôle	repli <code>changeme_in_prod</code> : la base de production avec un mot de passe public
<code>MAGBO_JWT_SECRET</code>	signature des jetons de session	pas de repli dans le compose : compose émet <code>warning: variable is not set</code> , une ligne qui défile, et le backend démarre avec un secret JWT vide
<code>MAGBO_WEBHOOK_TOKEN</code>	jeton attendu des terminaux Hikvision	vide = deny-by-default : le webhook rejette tout, aucun passage n'entre. Un <code>WARN SECURITY [prod]</code> le dit au démarrage (<code>config/ProdSecurityStartupCheck.java</code>)
<code>TZ</code>	fuseau des deux conteneurs	voir § 2.2 — le système horodate trois heures dans le futur
<code>ADMIN_PIN</code>	PIN du Panneau Administratif	voir § 2.1 — le compose refuse de démarrer
<code>MAGBO_ADMIN_PASSWORD</code>	mot de passe de l'administrateur initial	voir § 2.1 — le compose refuse de démarrer
<code>MAGBO_REGIME_HABILITADO</code>	interrupteur du régime de sortie (<code>false</code> par défaut)	absent = <code>false</code> : la règle ne s'évalue pas, rien ne change au portail

Les identifiants de base ne passent pas par `MAGBO_DB_*` sur la VM : le compose traduit lui-même en `SPRING_DATASOURCE_URL` / `_USERNAME` / `_PASSWORD`, qui priment sur `application-prod.properties`. Les quatre variables `MAGBO_DB_*` du CLAUDE.md ne concernent **que** le PC de développement, où le backend tourne hors conteneur.

2.1 ADMIN_PIN et MAGBO_ADMIN_PASSWORD — obligatoires, et pourquoi

Ces deux-là sont déclarées avec la syntaxe `${VAR:?message}` : `docker compose up` refuse de **démarrer** tant qu'elles n'ont pas de valeur. C'est délibéré, et c'est la même doctrine que le webhook (jeton absent = rejette, jamais « accepte n'importe qui »). La raison est écrite dans le compose (lignes 90-124) : **les deux échouent en silence quand elles sont vides**, ce qu'un `${VAR:-}` aurait laissé passer.

- `ADMIN_PIN=""` → `admin.pin` vide. Le PIN envoyé porte `@NotBlank`, donc **aucune** valeur ne correspond : le Panneau Administratif devient **inaccessible**, et l'écran dit seulement « PIN incorrect ». Ça ferme, ça n'ouvre pas — mais personne ne comprend pourquoi.
- `MAGBO_ADMIN_PASSWORD=""` → pire, et **seulement sur une base neuve** : `config/AdminBootstrap.java` lit `${magbo.admin.password:admin1234}`, et le défaut du `@Value` ne s'applique que si la propriété est **absente**, pas si elle est présente et vide. Une VM reconstruite de zéro — exactement le scénario que ce fichier protège — voit naître **un administrateur sans mot de passe**.

⚠ **MAGBO_ADMIN_PASSWORD n'a d'effet que sur une base SANS utilisateur `admin`.** `AdminBootstrap` sort à sa première ligne si le compte existe déjà (il journalise `Admin 'admin' já existe, pulando bootstrap.`). Changer le mot de passe d'un admin déjà créé se fait à l'écran Opérateurs. Modifier cette variable et recréer le conteneur **ne change rien** — et croire que ça a changé quelque chose est la pire des deux situations.

[A COMPLETER PAR SAM] Où est conservée la copie de `ADMIN_PIN` et du mot de passe `admin` de production, et qui d'autre y a accès ? Le `.env` de la VM est le seul exemplaire connu depuis le dépôt.

2.2 TZ: America/Sao_Paulo sur les deux conteneurs

C'est l'avertissement le plus long du compose, et il le mérite.

Les images `eclipse-temurin:17-jre-alpine` et `postgres:16-alpine` **démarrent en UTC**. La JVM adopte le fuseau du conteneur, donc **tout `LocalDateTime.now()` du backend rend une heure UTC** — pendant que `access_logs.timestamp` est écrit en `America/Sao_Paulo` par `EventTimeResolver`, à partir du `dateTime` que les appareils envoient. Les deux horloges cohabitent dans la **même** base, dans des colonnes `timestamp without time zone`, et **rien dans le schéma n'accuse la différence**.

Mesuré le 25/08/2026 dans le conteneur réel, par un chemin d'écriture de production (`POST /api/auth/password-reset-request`), qui écrit un `now()` brut) :

```
heure locale de l'école ..... 17:27:41
écrit dans password_reset_requests ..... 2026-08-25 20:27:42
```

⚠ **Et le dégât n'est pas un horodatage de travers dans un rapport.** Partout où une heure de `now()` est **comparée** à une heure de passage, l'écart devient une règle fautive : dans `cantine_removals.removido_em` (V020), qui décide ce qui reste caché dans le Moniteur Cantine, trois heures d'avance transformaient « retirer cette ligne » en « faire taire cette personne pendant les trois prochaines heures, y compris pour des entrées qui n'ont pas encore eu lieu ».

Pourquoi dans l'environnement et pas dans le code : le fuseau est une propriété de la **machine**, et le jar est le même partout. Corriger service par service avec une `Clock` explicite ne protège que celui qui y pense — le reste du système continue d'appeler `LocalDateTime.now()` (`createdAt`, `updatedAt`, `lastLogin`, `requestedAt` ...). Une ligne d'environnement les couvre tous.

Sur Postgres, `TZ` est là par cohérence : la base stocke de l'heure murale et ne convertit rien, mais `now()` et les logs du conteneur suivent le fuseau — et un `psql` qui affiche 20 h quand l'école vit à 17 h fait mentir toute vérification manuelle.

La vérification, à faire après chaque recreation de conteneur :

```
docker exec magbo-backend date # doit finir par -0300, jamais +0000
docker exec magbo-postgres date
```

⚠ Cette ligne était sur la VM **à la main**, et pas dans le git. Une machine reconstruite depuis le dépôt repartait en UTC et le défaut renaissait sans laisser de trace. C'est ce chemin-là qui l'a fait entrer dans le fichier.

3. Le rite de déploiement

Procédure complète et *marchée à la lettre* le 14/08/2026 contre une instance contenant les 439 993 enregistrements réels : `docs/operacional/mise-a-jour-vm.md` (`../operacional/mise-a-jour-vm.md`). Son § 11 dit où le texte n'avait **pas** suffi — huit endroits, dont quatre où « la vérification que j'avais écrite passait alors que la chose vérifiée était fausse ». Lisez-le une fois avant de déployer pour la première fois.

Ce qui suit en est la trame exécutable.

Quand. Hors horaire scolaire (mercredi après-midi, ou après 18 h). Entre l'étape 3 et l'étape 6 le backend a des fenêtres d'indisponibilité de quelques minutes ; les terminaux **mettent les passages en file et les renvoient** (comportement observé deux fois), donc rien n'est perdu — mais les écrans sont aveugles.

```
# 0. Où l'on est, et ce qui tourne vraiment
cd /opt/magbo # [A VERIFIER] chemin du dépôt sur la VM
docker ps --format '{{.Names}}\t{{.Status}}'
```

⚠ Un `docker compose ps` **vide ne veut pas dire que rien ne tourne** : il ne voit que les conteneurs créés *par compose*. Une installation démarrée à la main est invisible là alors qu'elle sert l'école. `docker ps` est la commande honnête.

```
# 0-bis. Le .env est-il vraiment lu ?
docker compose -f deploy/docker-compose.yml config | grep -E 'MAGBO_JWT_SECRET|POSTGRES_PASSWORD'
# doit afficher de VRAIES valeurs. Un .env absent ne produit qu'un warning qui défile.
```

```
# 1. SAUVEGARDE – non négociable, dix secondes
docker exec magbo-postgres pg_dump -U magbo magbodb \
| gzip > ~/magbo_avant_maj_$(date +%Y%m%d_%H%M).sql.gz

# et la seule vérification qui prouve quelque chose : comparer au vivant
zcat ~/magbo_avant_maj_*.sql.gz \
| awk '/^COPY public\.access_logs /{f=1;next} /^\\\.$/{f=0} f{n++} END{print n+0}'
docker exec magbo-postgres psql -U magbo -d magbodb -tAc "SELECT count(*) FROM access_logs;"
# → les deux nombres doivent être ÉGAUX
```

⚠ Ne jugez pas une sauvegarde à sa taille. Les 439 993 passages + les photos font **3,4 Mo compressés**. Un seuil inventé (« des dizaines de Mo ») avait failli faire conclure à un échec devant une sauvegarde parfaite, juste avant de toucher la prod.

```
# 2. Le code
git status                                # doit être propre
git rev-parse HEAD > ~/magbo_version_precedente.txt # le retour arrière en a besoin
git pull origin main
git log --oneline -3
```

```
# 3. Le jar
cd backend && mvn clean package -DskipTests && ls -l target/access-control-*.jar && cd ..
```

⚠ `mvn clean` efface le répertoire monté dans le conteneur. Pendant la construction, `/app` est vide côté conteneur ; le processus déjà lancé continue (la JVM tient le fichier ouvert), mais un redémarrage dans cette fenêtre ne trouverait aucun jar. Ne redémarrez rien avant que le `ls` réponde. Et **un seul jar dans `target/`** : la commande du conteneur est `java -jar access-control-*.jar`. Les suites (`mvn test`, `npm test`) se lancent **ailleurs**, avant de pousser : sur la VM on construit du code déjà validé.

```
# 4. Les migrations – § 4 de ce chapitre. AVANT de monter le backend.
```

```
# 5. Recréer le conteneur (pas `restart` – § 5.3)
docker compose -f deploy/docker-compose.yml up -d --force-recreate backend
docker compose -f deploy/docker-compose.yml logs backend --tail 40
curl -s http://localhost:8080/api/health      # → "database":"CONNECTED"
```

Les **deux WARN SECURITY [prod]** au démarrage sont normaux sur le PC de dev (mots de passe de dev) ; sur la VM ils signalent une vraie erreur de configuration.

```
# 6. LA vérification qui distingue l'ancien jar du nouveau
docker exec magbo-backend sh -c 'ls -l /app/access-control-*.jar'
docker inspect magbo-backend --format '{{.State.StartedAt}}'
docker exec magbo-backend date                # et le fuseau, tant qu'on y est
```

⚠ `/api/health` ne dit PAS quelle version tourne. Son champ `"version": "1.0.0"` est une chaîne écrite en dur dans `controllers/HealthController.java` (ligne 32) : identique avant et après n'importe quel déploiement. Un jar **plus récent** que `StartedAt` signifie : vous avez reconstruit et **pas** redémarré — le processus sert encore l'ancien code. C'est l'erreur de déploiement la plus probable, et c'est la seule chose qui la révèle. Le `sh -c` n'est pas décoratif : `docker exec` n'ouvre pas de shell, et sans lui le `*` arrive littéral.

```
# 7. Les permissions granulaires, AVANT le jour où elles servent
docker exec magbo-postgres psql -U magbo -d magbodb -c \
    "SELECT username, role, permissioes FROM system_users WHERE ativo = true;"
```

Les dix permissions existantes (`backend/.../security/Permissions.java`) :

`MEAL_ENTITLEMENT_WRITE`, `EXIT_PERMISSION_WRITE`, `ATTEMPTS_READ`, `REGIME_WRITE`, `PPMS_READ`,
`CANTINE_REMOVAL_WRITE`, `MEAL_SLOT_WRITE`, `PARCOURS_READ`, `CONFIG_WRITE`, `CDI_EXCLUSION_WRITE`.

Le rôle ADMIN les possède toutes. **Une permission absente ne ressemble pas à une panne : le bouton n'est simplement pas là.** Elles s'accordent à l'écran Opérateurs, dans « Permissions particulières » — un écran qui n'existe que depuis le 14/08/2026 : avant, elles n'étaient accordables que par API, ce que seul l'auteur savait faire.

CAPTURE D'ÉCRAN ATTENDUE

l'écran Opérateurs, un opérateur ouvert en modification, le bloc « Permissions particulières » et ses dix cases — grisées pour un ADMIN

7-bis. Mettre à jour les postes (§ 6) — c'est une étape, pas une note de bas de page. Les écrans du § 7 vivent dans l'application Electron, pas sur la VM.

8. Smoke avec des CLICS, pas seulement des `curl`. C'est la leçon la plus chère du projet : le 17/07, trois espèces de bug de câblage d'interface ont traversé toute la batterie de `curl` et ne sont apparues qu'en parcourant les écrans. Le parcours est dans `docs/frontend-smoke-checklist.md` (`../frontend-smoke-checklist.md`) — dont la **section 6-bis** est obligatoire, parce qu'elle couvre à la main les deux requêtes natives PostgreSQL que la suite ne peut pas exécuter (H2).

4. Les migrations, à la main

Tout est dans `deploy/migrations/README.md` (`../deploy/migrations/README.md`) — la liste ordonnée V001 → V027, et **pourquoi chacune existe**. Ce qui suit est le minimum à savoir pour ne pas se blesser.

Le principe. Le schéma naît du code Java (`ddl-auto=update`), qui **ajoute** table et colonne mais **n'altère jamais une contrainte CHECK existante** et ne retire rien. Les fichiers `V0*.sql` existent pour ce qu'Hibernate ne fera pas : les CHECK d'enum élargis, les index, les suppressions. Les oublier ne produit **aucune erreur au démarrage** — la panne s'arme et se déclenche des semaines plus tard, **dans la transaction qui enregistre un vrai passage**, et elle emporte l'`access_log` avec elle.

4.1 ON_ERROR_STOP=1 — la boucle complète

```
cd /opt/magbo # [A VERIFIER] racine du dépôt sur la VM

for f in deploy/migrations/V0*.sql; do
  printf '%-56s' "${basename "$f"}"
  docker exec -i magbo-postgres psql -U magbo -d magbodb -v ON_ERROR_STOP=1 < "$f" \
    > /tmp/mig.log 2>&1 && echo OK || { echo "ÉCHEC"; tail -3 /tmp/mig.log; break; }
done
```

⚠ Sans `-v ON_ERROR_STOP=1`, `psql` continue après l'erreur et sort avec le code 0. Mesuré et écrit dans le README des migrations : le même fichier fautif rend 0 sans l'option et 3 avec. Une boucle qui teste le code de retour affiche donc « OK » pour une migration qui n'a **rien fait**, le `&&` de la commande suivante passe, et le backend monte contre un schéma que personne n'a vérifié. C'est l'erreur qui a réellement été commise en rédigeant `mise-a-jour-vm.md` — alors que `reconstruire-do-zero.md`, écrit avant, utilisait l'option correctement.

⚠ `ON_ERROR_STOP` ne remplace pas la transaction : il arrête le *script*, il ne défait pas ce qui est passé. Les fichiers avec `BEGIN/COMMIT` (tous sauf V016, V018 et V019) se défont seuls ; sur les trois `CONCURRENTLY`, l'arrêt est tout ce qu'il y a, et le contrôle de l'index invalide reste obligatoire.

Réappliquer toutes les migrations est **sans effet** : elles sont idempotentes (`IF NOT EXISTS`, `DO $$`). Dans le doute, tout réappliquer coûte moins cher que de deviner ce qui manque.

4.2 Les pièges qui ont réellement mordu

Migration	Le piège
V009 / V015 / V022 / V026	élargissent un CHECK (<code>denial_reason</code> , <code>cdi_alert_events.tipo</code>). Hibernate crée le CHECK à la création de la table et ne le touche plus jamais : sans elles, l'INSERT échoue seulement sur la VM , et seulement le jour où la donnée apparaît.
V012	la seule obligatoire sur le PC aussi : elle supprime <code>student_exit_permissions.reason</code> , et <code>ddl-auto</code> ne supprime jamais. Sans elle, tout INSERT dans cette table échoue avec une erreur de driver.
V016 / V018 / V019	<code>CREATE INDEX CONCURRENTLY</code> , donc sans transaction . Interrompues, elles laissent un index invalide : <code>SELECT indisvalid FROM pg_index WHERE indexrelid = 'idx_access_logs_ponto_hora'::regclass;</code> doit rendre <code>t</code> . Sinon : <code>rollback R016</code> puis refaire.
V020	additive, mais à appliquer avant de monter le backend : si Hibernate crée <code>cantine_removals</code> en premier, le fichier n'a plus d'effet et les deux installations divergent — exactement ce que la V017 avait dû réparer.
V023	c'est un seed (l'affiche de la Vie Scolaire). Il n'a pas de rollback propre , et c'est assumé : ses lignes vivent dans les tables de la V021 et partent avec <code>R021</code> . Dès le premier clic dans l'écran, les lignes semées et celles éditées sont indiscernables.

Migration	Le piège
V025 / V026	leurs rollbacks effacent des données concernant des enfants (exclusions du CDI, registre des signalements). Un <code>pg_dump</code> avant, toujours.

Rollbacks présents : `R001` ... `R005` , `R007` , `R010` ... `R022` , `R024` , `R025` , `R026` . **Il n'y en a pas pour V006, V008, V009 ni V023** — vérifié dans `deploy/migrations/rollback/` .

`npm test -- tests/migrations.test.js` échoue si une migration existe et n'est pas citée dans le README, si elle n'a pas de rollback, ou si le CHECK de `denial_reason` est en retard sur l'enum Java. **Il ne prouve pas que le SQL tourne** — ça demande un Postgres, et ça reste une vérification manuelle.

4.3 Ce que les migrations ne savent PAS faire

Appliquées dans l'ordre sur une base **vide**, quatre échouent (`V005` , `V007` , `V008` , `V010` : `relation "app_users" / "system_users" does not exist`) et **six tables n'existent dans aucun fichier SQL** — elles sont nées d'Hibernate avant que ce répertoire existe. L'ordre supporté est donc : **backend d'abord, migrations ensuite** sur une base neuve ; **migrations d'abord, backend ensuite** sur une base qui tourne déjà. Détail et preuve : § 8.1 du README des migrations.

4.4 État de la production

[État au 03/09/2026, non vérifiable depuis le dépôt] **V001 à V027 sont toutes appliquées sur la VM**. La dernière est V027 (`licence_clock`), posée au déploiement de la licence le 01/09. Le rapport de la nuit du 27→28 les donnait encore « en attente » à partir de la V020 — l'écart s'explique par une application le vendredi matin. À reconfirmer en une commande :

```
docker exec magbo-postgres psql -U magbo -d magbodb -tAc "
SELECT to_regclass('cantine_removals') AS v020,
       to_regclass('meal_slots')       AS v021,
       to_regclass('system_settings')  AS v024,
       to_regclass('cdi_exclusions')    AS v025,
       to_regclass('cdi_alert_events')  AS v026;"

docker exec magbo-postgres psql -U magbo -d magbodb -tAc \
"SELECT pg_get_constraintdef(oid) FROM pg_constraint WHERE conname LIKE '%denial_reason%';" \
| tr ' ' '\n' | grep -cE 'REGIME_NOT_ALLOWED|MEAL_SLOT_NOT_CONFIGURED'
# → 2 (moins = V015 et/ou V022 manquantes : NE PAS activer ce qui en dépend)

docker exec magbo-postgres psql -U magbo -d magbodb -tAc \
"SELECT indexrelid::regclass, indisvalid FROM pg_index
WHERE indexrelid::regclass::text LIKE 'idx_access_logs%';"
# → les trois index, tous 't'
```


5. Sauvegarde, restauration, retour arrière

5.1 Sauvegarde

La procédure de référence, **exécutée de bout en bout le 12/08/2026** (base vide → application ouverte → sauvegarde → restauration → application ouverte sur la base restaurée), est

`docs/operacional/reconstruir-do-zero.md` (`../operacional/reconstruir-do-zero.md`). N'en réécrivez pas une autre. Les pièges à retenir :

- Le `pg_dump` **EST la sauvegarde des photos**. `user_photos` est la première table dont la donnée n'existe nulle part ailleurs — pas de copie sur disque, c'est le principe (§ 1, fait n° 2). Après restauration, vérifiez l'intégrité bit à bit : `SELECT count(*), count(*) FILTER (WHERE sha256 = encode(sha256(bytes), 'hex')) FROM user_photos;` → les deux nombres doivent être égaux.
- **Restaurer se fait dans une base VIERGE**, jamais par-dessus une base vivante, et **sans démarrer le backend avant** : il créerait le schéma et l'admin, et le dump — qui contient déjà tout — se heurterait à l'existant (« erreurs `already exists` en cascade »).
- **Après restauration, le mot de passe admin est celui de la sauvegarde**, pas `admin1234`. Le log dit `Admin 'admin' já existe, pulando bootstrap.`
- Un dump antérieur à la V011 **ne contient pas de photos** : tables pleines, photos absentes, et ce n'est pas une panne.

✓ La sauvegarde tourne — vérifié en production le 31/08/2026

Tous les jours à 19:00, ~109 Mo par dump, PostgreSQL 16.14, rétention 14 jours. Ce n'était pas acquis : ce chapitre posait la question, et la réponse est bonne.

⚠ Mais elle ne tourne pas là où le dépôt le laisse croire, et les deux erreurs se corrigent l'une l'autre :

Le dépôt suggère	La VM fait réellement
le script <code>deploy/backup.sh</code>	<code>/home/magbo/backup-magbo.sh</code> — non versionné
le dossier <code>/var/backups/magbo/</code>	<code>/home/magbo/backups/</code>
rétention 30 jours	14 jours

⚠ Ne « réparez » pas `deploy/backup.sh` en croyant réparer la sauvegarde, et ne remplacez pas celui de la VM par celui du dépôt : celui du dépôt appelle `pg_dump` **directement sur l'hôte** alors que PostgreSQL tourne **en conteneur**. Lancé tel quel il échoue, et l'erreur part dans un `backup.log` que personne ne lit.

● **Ce qui reste ouvert, et qui est le vrai risque : il n'y a AUCUNE COPIE HORS MACHINE.** Les sauvegardes vivent sur la VM qu'elles sauvegardent. Un disque perdu emporte la base **et** ses quatorze jours de dumps — et avec eux les photos, qui n'existent nulle part ailleurs (§ 2.11 du chapitre 7).

Pour vérifier, à tout moment :

```
ssh magbo@192.168.1.253
crontab -l | grep -i backup      # le cron est-il actif ?
ls -lht ~/backups/ | head -5    # le plus récent doit dater d'hier 19:00
```


Si le dossier est vide : **faites un dump manuel aujourd'hui**, puis corrigez le script (`DB_NAME=magbodb`) et envelopper dans `docker exec`).

[A COMPLETER PAR SAM] Le cron de sauvegarde est-il actif sur la VM, sous quel utilisateur, et existe-t-il une copie hors-machine (`MAGBO_BACKUP_REMOTE`) ? La règle 3-2-1 est écrite dans l'en-tête du script, mais la destination distante n'est nulle part dans le dépôt.

5.2 Retour arrière — quatre niveaux, du plus léger au plus lourd

1. **Comportement** : éteindre la fonctionnalité par configuration (par ex. `MAGBO_REGIME_HABILITADO=false`) et recréer le conteneur. Les tables restent inertes, rien n'est effacé. **Première option, toujours.**
2. **Code** : revenir au jar précédent — `git checkout $(cat ~/magbo_version_precedente.txt)`, `mvn clean package -DskipTests`, puis `up -d --force-recreate backend`.
3. **Schéma** (`rollback/R0*.sql`) : **rare et destructif**. Ne déroulez pas les migrations par réflexe — elles sont additives, une colonne de plus qu'un ancien jar ignore ne gêne pas. Les rollbacks servent à un objet **cassé** (index invalide de la V016), pas à un retour de version.
4. **Données** : `pg_restore` du dump. Dernier recours — il coûte tout ce qui a été enregistré depuis la sauvegarde.

5.3 `restart` ne suffit pas

△ Une variable d'environnement est fixée à la **création** du conteneur. `docker compose restart` relance le processus dans le conteneur existant, avec l'**ancienne** valeur. **Mesuré pendant le drill du 14/08** : `.env` passé à `true`, `compose restart backend`, puis `printenv` → `false`. Aucune erreur, aucun avertissement, fonctionnalité éteinte. Toujours :

```
docker compose -f deploy/docker-compose.yml up -d --force-recreate backend
docker exec magbo-backend printenv MAGBO_REGIME_HABILITADO
```

6. Le portable — construire, vérifier, distribuer

L'application des postes est un exécutable Electron. Elle **n'a rien à voir avec la VM** et se met à jour séparément.

6.1 Construire

```
npm ci
npm test           # 889 attendus (03/09/2026), 0 échec
npm run build:portable # ou build:win pour l'installateur NSIS
npm run verify:package # doit finir par RESULTADO: APROVADO
```

Le paquet est une **liste de permission** (`files` dans `package.json`) : n'entrent que `main.js`, `preload.js`, `index.html`, `package.json`, `css/`, `js/`, `libs/` et `build/icon.ico`. La v2.0.0 utilisait une liste d'exclusion et embarquait **81 fichiers internes** — `docs/` entier, `.claude/`, les scripts de banc. Historique dans `docs/operacional/release-portable.md` (`../operacional/release-portable.md`).

6.2 Vérifier — et pourquoi ça marche maintenant

`scripts/verify-package.js` répond à deux questions : *quelque chose d'interne a-t-il fuité ?* (17 règles, dont `.env`, `.sql`, `.pem` / `.key`) et *tout ce dont l'app a besoin est-il là ?*

⚠ La liste des fichiers obligatoires est DÉRIVÉE de `index.html`, plus écrite à la main. Elle était statique et connaissait 26 fichiers, nommés avant que `postoFixo.js`, `photoCache.js` et `PersonPhoto.js` existent : **un paquet sans ces trois-là était APPROUVÉ par le portail**. Le projet n'a pas de bundler — un fichier existe en runtime parce qu'il y a une balise `<script src=...>` dans `index.html`, et pour aucune autre raison. `index.html` est donc la liste de dépendances, et `scripts/indexAssets.js` la lit, pour le portail de release et pour `tests/wiring.test.js` : une lecture, une implémentation.

```
node -e "console.log(require('./scripts/indexAssets').requiredPackageFiles().length)"
# → 96 (mesuré le 03/09/2026)
```

Mesuré le 03/09/2026 : le paquet doit contenir **96/96** fichiers obligatoires. ⚠ Ce nombre n'est pas une constante à comparer d'année en année — la liste étant dérivée d' `index.html`, elle monte à chaque écran nouveau. Ce qui doit être vrai, c'est que les **deux** nombres soient égaux.

⚠ `verify:package` qui passe ne prouve pas que le paquet est À JOUR. Avant de distribuer, comparez la date de l'artefact au nombre de merges survenus depuis :

```
ls -l dist/win-unpacked/resources/app.asar
git log --oneline --merges --since="<cette date>" | wc -l # > 0 → rebuild
grep -cE 'src="https?://|cdn\.|unpkg|jsdelivr' index.html # doit rendre 0
```

Le dernier `grep` est le garde du mode kiosque : tout est vendorisé dans `libs/`, et un `<script src="https://...">` réintroduit **ne donne pas d'erreur** — l'écran ne s'affiche simplement pas sur un poste sans internet.

6.3 Distribuer

Procédure poste par poste, en français, prête à imprimer : `docs/operacional/guide-installation-postes.md` (`../operacional/guide-installation-postes.md`). Le résumé qui évite 90 % des appels :

- **Remplacer LE `.exe` seulement.** Le `.bat` d'à côté porte la configuration de ce poste ; l'écraser fait perdre son secteur.
- **Renommer l'ancien dossier, ne pas l'effacer** (`MAGBO-ancien-AAAA-MM-JJ`). C'est le chemin de retour en pleine journée. Effacer une semaine plus tard.
- **Ouvrir PAR `Abrir-MAGBO.bat`, jamais par le `.exe`.** L'exécutable ne garde **aucune** configuration : `main.js` lit les variables d'environnement et retombe sur `http://localhost:8080`. Ouvert directement, il affiche une application **vide, sans message d'erreur** — ce qui ressemble à un système cassé. Le modèle du lanceur est `deploy/portable/Abrir-MAGBO.bat` ; deux lignes à régler : `MAGBO_API_URL` (l'adresse du serveur) et `MAGBO_SECTOR` (`PORT1`, `PORT2`, `PORT3`, `BIBLIO`, `ENFERM`, `REFEI1`, `REFEI2`).
- **Refaire les raccourcis vers le `.bat`.** Un raccourci resté sur le `.exe` est la cause la plus fréquente de « l'application ne marche plus ».

- **Vérifier à l'écran, pas la fenêtre** : des chiffres au tableau de bord, des lignes avec des noms dans le Journal, le nom de l'opérateur en haut à droite.
- **Hors horaire de service** : pendant que l'app est fermée, l'opérateur ne voit plus les tentatives refusées. Le backend, lui, continue d'enregistrer — le webhook ne dépend pas de l'application. Ce qui se perd, c'est la surveillance en direct.

CAPTURE D'ÉCRAN ATTENDUE

le premier lancement sur un poste — l'avertissement bleu « Windows a protégé votre ordinateur », avec le lien « Informations complémentaires » à cliquer

[A COMPLETER PAR SAM] Quelle version tourne sur quel poste aujourd'hui, et quelle est l'adresse réelle du serveur ? Le modèle versionné porte `http://192.168.1.253:8080`, et la réservation d'IP auprès du SI n'était pas confirmée dans le dépôt. La fiche de suivi vide est à la fin du guide d'installation.

7. Ce qui se règle SANS redéployer

Trois niveaux, du moins cher au plus cher. Savoir lequel s'applique évite un déploiement inutile — ou une modification qui ne prend jamais effet.

Niveau	Où	Prise d'effet	Exemples
Écran de configuration	<code>system_settings</code> (V024), engrenage du header, permission <code>CONFIG_WRITE</code>	≤ 15 s (cache TTL)	<code>magbo.cdi.capacidade</code> , <code>magbo.cdi.estado</code> , <code>magbo.cantine.duracao-maxima-minutos</code> , <code>magbo.cantine.sortis-visiveis-minutos</code> , <code>magbo.cantine.turmas-dispensees</code>
<code>.env</code>	<code>deploy/.env</code>	recréation du conteneur (<code>up -d --force-recreate</code>)	<code>TZ</code> , <code>ADMIN_PIN</code> , <code>MAGBO_REGIME_HABILITADO</code> , <code>MAGBO_WEBHOOK_TOKEN</code>
Properties	<code>application-prod.properties</code>	rebuild du jar + redéploiement	politiques <code>magbo.policy.*</code> , fenêtres de dédup, <code>magbo.presence.auto-close.*</code>

⚠ **Contrat structurel de la V024** : une base **sans aucune ligne** dans `system_settings` se comporte **exactement** comme avant la migration. La table naît vide, il n'y a rien à semer, et une valeur illisible retombe sur le défaut avec un WARN — jamais une exception, parce qu'un « abc » dans une clé numérique ne peut pas faire tomber la décision d'un passage (`services/SettingsService.java`).

⚠ **Aucun secret ici**. Jetons, mots de passe et PIN restent dans le `.env` : une table lisible depuis un écran d'administration est exactement l'endroit où un secret ne doit pas vivre. `SettingsCatalog` ne les déclare pas, et lecture comme écriture sont derrière la même permission.

Les politiques de décision en production (`application-prod.properties` , et elles ne sont **pas** les défauts du code) :

```

magbo.policy.meal-not-entitled = DENY
magbo.policy.meal-pending      = DENY      ← décision D5 (ADR-004) ; le code par défaut dit OBSERVATION
magbo.policy.outside-meal-time = OBSERVATION
magbo.policy.duplicate-meal    = OBSERVATION
magbo.policy.exit-not-authorized = DENY
magbo.policy.user-inactive      = DENY
magbo.policy.missing-door-mapping = FALLBACK

```

⚠ **meal-pending=DENY a un prérequis opérationnel** : l'import en masse des élèves autorisés doit être fait **avant le jour 1**. Sans lui, tout élève reste **PENDING** et **aucun repas n'est enregistré**. Et rappel d'ADR-004 : **DENY** est une décision **logique**. Le MAGBO ne bloque aucune porte (ADR-003) ; le terminal ouvre, le MAGBO raconte.

8. Le tableau du matin — symptôme → cause probable → geste

Tiré de ce qui est réellement arrivé. À lire à 7 h quand rien ne marche.

Symptôme	Cause probable	Geste
Les heures sont trois heures en avance ; une ligne retirée du Moniteur Cantine reste cachée jusqu'à midi	TZ absent du conteneur : image en UTC, la JVM adopte le fuseau, <code>LocalDateTime.now()</code> rend de l'UTC (mesuré 25/08 : 17:27 → 20:27)	<code>docker exec magbo-backend date</code> → doit finir par <code>-0300</code> . Sinon : TZ dans <code>deploy/.env</code> , puis <code>up -d --force-recreate backend</code>
Aucun événement n'arrive , aucune erreur nulle part	IP changé par DHCP : l'Écoute HTTP du terminal pointe vers l'ancienne adresse du backend, ou <code>door_mappings.terminal_ip</code> ne correspond plus	Comparer les trois : l'IP affichée sur l'appareil, l'URL de l'Écoute HTTP, et <code>SELECT terminal_ip, point_id, action FROM door_mappings WHERE ativo;</code>
Le Moniteur Cantine affiche « Dentro : 0 » pendant que des élèves mangent	Corps multipart tronqué : l'appareil annonce N octets et en envoie moins ; <code>getParts()</code> jetait tout, y compris la part JSON déjà complète (95 événements perdus en un jour, 24/08)	<code>`docker logs magbo-backend --since 24h 2>&1 \</code> <code>grep -c "corpo multipart CORTADO" .</code> La lecture tolérante (<code>services/MultipartTolera nte.java</code>) rattrape ; un compte élevé désigne l'appareil
Un passage réel disparaît et le log montre une erreur d'INSERT en transaction	Un CHECK d'enum en retard : V009 / V015 / V022 / V026 non appliquée. <code>ddl-auto</code> ne modifie jamais un CHECK existant — la panne n'existe que sur la VM	<code>SELECT pg_get_constraintdef(oid) FROM pg_constraint WHERE conname LIKE '%denial_reason%';</code> puis appliquer la migration manquante (§ 4)

Symptôme	Cause probable	Geste
Déploiement « réussi », mais le comportement est l' ancien	Jar reconstruit et conteneur pas recréé — le processus tient encore l'ancien fichier	<code>docker exec magbo-backend sh -c 'ls -l /app/access-control-*.jar'</code> vs <code>docker inspect magbo-backend --format '{{.State.StartedAt}}'</code> . Jar plus récent = redémarrer
Une variable du <code>.env</code> modifiée n'a aucun effet	<code>docker compose restart</code> au lieu de <code>up -d --force-recreate</code> : l'environnement est fixé à la création du conteneur (mesuré le 14/08)	<code>docker exec magbo-backend printenv <VAR></code> , puis <code>up -d --force-recreate backend</code>
La boucle de migrations affiche OK partout et le schéma est faux	<code>-v ON_ERROR_STOP=1</code> oublié : <code>psql</code> continue après l'erreur et sort avec 0	Refaire la boucle du § 4.1 avec l'option. Ne pas monter le backend avant
<code>docker compose up</code> refuse de démarrer (« defina ADMIN_PIN... »)	C'est voulu : <code>ADMIN_PIN</code> et <code>MAGBO_ADMIN_PASSWORD</code> sont déclarées <code>\${VAR:?}</code>	Remplir les deux dans <code>deploy/.env</code> . Ne jamais leur donner un repli (§ 2.1)
Panneau Administratif inaccessible, « PIN incorrect » quel que soit le PIN	<code>admin.pin</code> vide (contournement du garde ci-dessus) : <code>@NotBlank</code> fait qu'aucune valeur ne correspond	<code>docker exec magbo-backend printenv ADMIN_PIN</code>
Le webhook rejette tout , WARN <code>SECURITY [prod]</code> au démarrage	<code>MAGBO_WEBHOOK_TOKEN</code> absent → <code>deny-by-default</code> (<code>ProdSecurityStartupCheck</code>)	Poser le jeton dans <code>.env</code> et le même sur l'appareil (header <code>X-MAGBO-WEBHOOK-TOKEN</code> ou <code>?token=</code>)
Sur un poste, l'application s'ouvre vide, sans erreur	Ouverte par le <code>.exe</code> (retombe sur <code>http://localhost:8080</code>), ou raccourci pointant sur le <code>.exe</code>	Rouvrir par <code>Abrir-MAGBO.bat</code> . Si toujours vide : ouvrir <code>http://<serveur>:8080/api/health</code> depuis ce poste
Un opérateur ne voit pas un bouton (droits repas, régimes, PPMS, configuration)	Permission granulaire absente : <code>system_users.permissoes</code> est un CSV, <code>NULL</code> = aucune	<code>SELECT username, role, permissoes FROM system_users WHERE ativo;</code> puis écran Opérateurs → « Permissions particulières »
Après 21 h, les compteurs « aujourd'hui » tombent à 0 et les rapports interrogent un jour qui n'existe pas	Le 4 ^e défaut d'horloge : <code>new Date().toISOString().slice(0,10)</code> rend demain à Rio (UTC-3). Corrigé dans <code>main</code> par <code>dayKey()</code> de <code>js/utls/helpers.js</code>	Le poste n'est pas à jour : distribuer un portable reconstruit depuis main fusionnée (§ 6)
Le backend démarre mais la base paraît vide	Sur le PC : le conteneur légal <code>magbo-db</code> a pris le port 5432, ou les 4 variables <code>MAGBO_DB_*</code> manquent dans la même session (les replis du profil <code>prod</code> visent une autre base)	<code>docker ps</code> → <code>docker stop magbo-db; docker start magbo-postgres</code> . Puis vérifier les variables

Symptôme	Cause probable	Geste	
<code>/var/backups/magbo/</code> est vide ou n'existe pas	⚠ Ce n'est pas le bon dossier — les sauvegardes sont dans <code>/home/magbo/backups/</code> (vérifié le 31/08). Cherchées au mauvais endroit, elles paraissent absentes alors qu'elles tournent	<code>ls -lht ~/backups/ \</code>	<code>head -5`</code> — le plus récent doit dater d'hier 19:00. Si là c'est vide : dump manuel aujourd'hui (§ 5.1)
<code>mvn test</code> passe avec moins de tests qu'attendu	Un test supprimé, ou le Surefire qui saute les <code>*IT.java</code> en silence (le <code>pom.xml</code> a besoin de <code><include>**/*IT.java</code>)	Le critère est 0 échec et exactement 2 @Disabled ; le total, lui, monte à chaque livraison — dernière mesure le 03/09/2026 : 1055 (npm : 889). Un total inférieur = un test supprimé. Mesurer du zéro : <code>cd backend && rm -rf target && mvn -o test</code>	

9. Ce qui reste ouvert

- `docs/operacional/handoff.md` **ne s'arrête plus au 05/08/2026**. Il a été repris le 29/08 et porte en tête « Date de coupe : 2026-09-01 », dernier merge couvert `5632d0a` (PR #87 — la licence) : vérifié le 03/09/2026 à `handoff.md:6`. Ce qui lui manque désormais, ce sont les deux PR postérieures, #89 (premier lancement) et #90 (poste administratif, ADR-007). Là où il diverge du code, **le code gagne**.
- ⚠ **Doublon réel à signaler, pas à corriger** : `docs/architecture/decisoes/` contient **deux fichiers ADR-005** — `ADR-005-creneaux-cantine.md` et `ADR-005-totvs-rastreabilidade-no-dono-do-dado.md`. Deux décisions distinctes portent le même numéro. Renommer demande de reprendre les renvois ; c'est une décision, pas une retouche.
- **Sept endpoints non gardés** (`/api/users`, `/api/access/logs/*`, `registerAccess`), dette documentée et **de taille assertée** dans `ControllerAuthorizationGuardTest.DIVIDA_CONHECIDA`. Les garder cassera des écrans : c'est un chantier avec ses propres preuves.
- **Les règles sont évaluées à l'heure de la DÉCISION**, pas à celle de l'événement (seul le timestamp écrit a changé le 03/08). Une file hors-ligne rejouée hors du créneau peut produire un `FORA_HORARIO` sur un passage parfaitement légal. Dette ouverte, décrite au § 5.1 du handoff. **Une seule exception assumée** : le régime de sortie, qui juge sur l'heure du **passage**, avec un test qui capture l'argument.
- ✓ **La cantine A une fermeture automatique, et son heure est confirmée** (`magbo.presence.auto-close.times[REFE11]=15:00` ; heure validée par Sam le 31/08 contre le service réel). ⚠ **Ce qui reste n'est pas le réglage mais le taux** : environ **72 sorties par jour ne sont jamais lues** et sont fermées par le système à 15:00. Ces lignes portent `FECHAMENTO_AUTO` — leur durée de repas est un **maximum**, pas une mesure, et il faut les exclure de toute moyenne. Détail au § 3.12 et dans le handoff, § 2.4.

[Affirmé par Sam le 28/08/2026, non vérifiable depuis le dépôt] **Six terminaux en service :** portail `.166` (SORTIE) et `.167` (ENTRÉE), CDI `.15` et `.16`, cantine `.10` / `.12` / `.13` / `.14`. [A VERIFIER] L'énumération donne huit adresses pour « six terminaux » — deux d'entre elles sont les **caméras** DeepinView du portail (`bootstrap/DoorMappingBootstrap.java` les mappe par IP seule, `.167` →ENTRADA, `.166` →SAIDA), qui ne sont pas des terminaux MinMoe. Trancher avec :

```
docker exec magbo-postgres psql -U magbo -d magbodb -tAc \
"SELECT terminal_ip, door_no, reader_no, point_id, action, label
FROM door_mappings WHERE ativo ORDER BY terminal_ip NULLS LAST, point_id;"
```

Les IP de cantine `.10` / `.12` (ENTRADA) et `.13` / `.14` (SORTIE) sont cohérentes avec le relevé de production du 24/08 cité dans `services/MultipartTolerante.java`.

[A COMPLETER PAR SAM] La caméra `.166` (SORTIE du portail) était donnée **en panne physique** au 27/08 (`docs/operacional/diagnostic-portaria-2026-08-27.md`). A-t-elle été remplacée ? Tant qu'elle est morte, toute analyse des sorties du portail est faussée, et il vaut mieux la retirer des `door_mappings` pour que son absence soit explicite plutôt qu'un trou.

Où aller ensuite

Besoin	Document
Reconstruire de zéro, restaurer une sauvegarde	<code>docs/operacional/reconstruir-do-zero.md</code>
Mettre à jour une VM qui sert l'école	<code>docs/operacional/mise-a-jour-vm.md</code>
Installer l'app sur un poste (imprimable, FR)	<code>docs/operacional/guide-installation-postes.md</code>
Publier une release du portable	<code>docs/operacional/release-portable.md</code>
Migrations : la liste et le pourquoi de chacune	<code>deploy/migrations/README.md</code>
Smoke après déploiement (avec des clics)	<code>docs/frontend-smoke-checklist.md</code>
Ce que l'opérateur voit, écran par écran	<code>docs/manual-utilisateur.md</code>
Terminaux, caméras, bibliothèques faciales	<code>docs/operacional/procedimento-hikcentral.md</code> , <code>.claude/rules/hikvision.md</code>

Chapitre 7 — Architecture technique

Ce chapitre s'adresse à la personne qui reprend **le code**. Il décrit comment le backend, le frontend, la base et les tests sont faits — et surtout **pourquoi ils sont faits comme ça**, parce que plusieurs choix ont l'air étranges tant qu'on ne connaît pas l'incident qui les a produits.

1. Le backend

Spring Boot 3.2.5 / Java 17, Maven. Deux profils : `dev` (H2) et `prod` (PostgreSQL). Paquet racine `com.magbo.access`, découpé en `controllers / services / repositories / models / dto / config / security / bootstrap`.

27 contrôleurs, 35 services, 35 modèles. Lombok partout (`@RequiredArgsConstructor` , `@Builder`), injection par constructeur.

1.1 Les services qui décident

Service	Ce qu'il décide
<code>AccessDecisionService</code>	l'orchestrateur — la seule classe qui connaît l'ordre des règles
<code>HikvisionEventClassifier</code>	sous-type → méthode et résultat. Pur, sans état
<code>EventTimeResolver</code>	quelle heure va dans la base : celle de l'événement, ou celle de la réception
<code>CameraIdentityService</code>	quelle personne est ce visage (portail)
<code>PersonNameMatcher</code>	la comparaison des noms, avec ses trois pièges
<code>MealEntitlementService</code>	droit au repas + son historique, dans la même transaction
<code>MealsSlotService</code>	à quel créneau appartient cette personne
<code>ExitPermissionService</code>	autorisation ponctuelle de sortie
<code>RegimeSortieService</code>	le droit annuel — cinq verdicts
<code>CdiExclusionService</code> / <code>CdiAlertService</code>	exclusions du CDI et leur registre
<code>SettingsService</code> / <code>SettingsCatalog</code>	la surcouche des réglages, et leur catalogue
<code>DeduplicationService</code> , <code>SamePassageService</code> , <code>WebhookIngestionDedupService</code>	trois couches distinctes (chapitre 3)
<code>PresenceAutoCloseService</code>	les fermetures de fin de journée
<code>PostoFixoService</code> , <code>PresencaAbertaService</code>	les deux marques de répétition
<code>PpmsService</code>	qui est dans l'école, par zone
<code>MultipartTolerante</code>	le parseur qui accepte ce que les appareils envoient vraiment

⚠ **`AccessDecisionService` est le seul endroit où l'ordre des règles est écrit.** Le chercher ailleurs, c'est le dupliquer — et deux ordres finissent par diverger.

1.2 ⚠ Pourquoi `/error` est ouvert dans `SecurityConfig`

C'est la ligne qui surprend en relisant `security/SecurityConfig.java`, et le commentaire du fichier l'explique :

*Sans elle, un import qui échoue renvoyait une erreur qui mentait sur sa cause : l'opérateur réessayait, ça échouait encore, sans jamais voir quelle ligne du fichier était fausse. **Une erreur qui ment sur sa cause coûte plus cher que l'erreur.***

⚠ `/error` n'ouvre rien. Il ne rend que l'erreur d'une requête **déjà arrivée** ; aucune donnée protégée ne passe par lui. Ce qu'il publie, c'est le motif de l'échec — exactement ce que l'opérateur doit lire.

Le reste de la configuration : **JWT sans session**, `permitAll` uniquement sur la connexion, la santé, les webhooks et la console H2. L'autorisation par aire passe par

```
@PreAuthorize("@areaSecurity.can('...'))", l'écriture sensible par hasRole('ADMIN') or  
@areaSecurity.hasPermission('...').
```

⚠ Les comparaisons de secret utilisent `MessageDigest.isEqual`, pas `equals` — et avec un `trim`, parce qu'un retour chariot collé au jeton a déjà fait perdre du temps.

1.3 Le webhook

Refus par défaut : jeton absent de la configuration → le webhook rejette. Un système qui accepte tout parce qu'il n'est pas configuré est pire qu'un système arrêté.

Le parsing est **tolérant** (`parsePayload`, `MultipartTolerante`) parce que les appareils n'envoient pas ce que la documentation promet. Ne pas régresser là-dessus : c'est du code qui a été écrit contre du matériel réel.

2. Le frontend

Electron + React 18 via Babel standalone. Aucun bundler.

2.1 Ce que « pas de bundler » implique

Les fichiers de `js/` sont chargés par des `<script>` dans `index.html`, et l'ordre compte. Un composant est une `function` globale ; un utilitaire est un objet posé sur `window`.

⚠ **Un module doit être chargé avant ceux qui l'utilisent au moment du rendu.** Le test `tests/wiring.test.js` vérifie que les utilitaires chargent avant les composants — il a attrapé une erreur d'ordre le 28/08.

✓ **Tout est embarqué dans `libs/`** : React, ReactDOM, Babel, Tailwind, lucide, jsPDF, xlsx, et les polices. Le kiosque rend sans réseau.

⚠ **Ne jamais réintroduire un CDN.** Un `<script src="https://...">` nouveau ne produit **aucune erreur** : l'écran ne s'affiche simplement pas sur un poste hors ligne. La vérification tient en une commande :

```
grep -cE 'src="https?://|cdn\.|unpkg|jsdelivr' index.html # doit rendre 0
```

2.2 La structure de `js/`

Dossier	Ce qu'il contient
<code>js/components/</code>	les écrans et les composants partagés
<code>js/cdi/</code>	l'écran du CDI, qui a sa propre couche de données (<code>CdiBackend</code>)
<code>js/utils/</code>	i18n, permissions, cache d'utilisateurs, cache de photos, helpers
<code>js/data/constants.js</code>	les points d'accès — miroir conscient de <code>AreaMapping</code> côté backend

⚠ `js/data/constants.js` et `AreaMapping` **changent ENSEMBLE**. Ce sont deux copies de la même vérité, assumées comme telles.

⚠ **Deux couches HTTP coexistent** : `js/api.js` (`window.api`) et `js/utils/api.js` (les normaliseurs). C'est une dette connue. **Ne pas en créer une troisième** ; consolider est un chantier à part.

2.3 L'i18n

`js/utils/i18n.js` porte deux dictionnaires, **FR d'abord, PT ensuite**.

⚠ `t()` **rend la clé elle-même quand elle manque**. Une clé absente s'affiche donc crue à l'écran (`cdi.excl.titulo`) — visible pour un humain, invisible pour une suite qui ne rend aucun composant. Trois tests couvrent ce trou : `i18n.test.js` (les deux dictionnaires ont les mêmes clés), `i18nChavesUsadas.test.js` (toute clé utilisée existe) et `i18nGuard.test.js` (pas de texte visible en dur dans les écrans migrés).

`tEnum(groupe, valeur)` est le **seul** chemin vers les libellés d'énumération — ils vivaient jusqu'au 14/08 dans des tables statiques en français fixe, même quand l'écran était en portugais.

2.4 Electron

`main.js` crée la fenêtre, `preload.js` expose la configuration (`window.magboConfig` : `apiUrl`, `sector`, `source`, `doitConfigurer`, `isProduction`, `cheminFichier`, `version` — issus de `MAGBO_API_URL` / `MAGBO_SECTOR` ou du fichier `magbo-poste.json`). `NODE_ENV=production` active le mode kiosque, à condition que le poste soit déjà réglé (`posteConfig.verrouillable`).

⚠ **Le second pont, `window.magboIpc`, est mort — et `MAGBO_KIOSK_PIN` ne sert à rien.**

`preload.js:112-123` expose `verifyKioskPin`, `exitKiosk` et `onRequestAdminPin` ; `main.js:383` enregistre `Ctrl+Shift+Alt+Q` et émet `request-admin-pin` ; `main.js:32` lit le PIN. **Aucun fichier de `js/`, d' `index.html` ni de `tests/` n'écoute** (mesuré le 03/09/2026 par recherche exhaustive). Le raccourci est donc un no-op silencieux, le PIN n'est demandé par aucun écran, et un poste verrouillé se ferme par `Ctrl+Alt+Suppr` → Gestionnaire des tâches. C'est la famille de défaut du bug `f947373` : une moitié écrite, l'autre jamais branchée, et l'échec est muet. ⚠ Ne pas confondre avec `ADMIN_PIN`, qui est le PIN du backend (`/api/admin/verify`, `js/components/AdminPinModal.js`) et qui, lui, fonctionne.

⚠ **L'exécutable ne garde aucune configuration.** Il lit des variables d'environnement et retombe sur `http://localhost:8080`. Lancé nu, il ouvre une application **vide, sans erreur** — d'où le lanceur `.bat`.

3. La base

PostgreSQL 16. `ddl-auto=update` sur tous les profils, y compris la VM.

3.1 Les tables principales

Table	Ce qu'elle porte
<code>app_users</code>	les personnes. Identifiant String (Pronote, 7 chiffres avec zéros à gauche)
<code>access_logs</code>	les accès effectifs — jamais un refus (ADR-001)
<code>access_attempts</code>	tout ce qui a été tenté et refusé — quatre axes : méthode, résultat du terminal, décision du MAGBO, motif
<code>door_mappings</code>	IP + porte + lecteur → point d'accès
<code>meal_entitlements</code> + <code>_events</code>	droit au repas et son historique
<code>meal_slots</code> + <code>_classes</code> + <code>_students</code>	le planning de la cantine (V021)
<code>student_exit_permissions</code>	autorisations ponctuelles de sortie
<code>student_regimes</code> + <code>_events</code>	le droit annuel de sortir
<code>user_photos</code>	les photos d'identité, en base
<code>system_settings</code>	la surcouches des réglages (V024)
<code>cdi_exclusions</code> , <code>cdi_alert_events</code>	exclusions du CDI et leur registre
<code>cantine_removals</code>	les retraits d'écran du Moniteur
<code>system_users</code>	les comptes opérateurs

⚠ **Les dates sont des `LocalDateTime` locaux (BRT)**, dans des colonnes `timestamp without time zone`. **Ne jamais poser `hibernate.jdbc.time_zone`**.

⚠ **`user_photos` est une table à part, jamais une colonne de `app_users`** :

`userRepository.findAll()` tourne sur un chemin chaud, et une colonne `bytea` là-dedans traînerait ~25 Mo par appel. Et c'est du **BYTEA sans `@Lob`** — avec `@Lob`, Hibernate 6 génère un `oid`, que `pg_dump` traite autrement.

⚠ **`user_photos` est la première table dont la donnée n'existe nulle part ailleurs.** Pas de copie sur disque : c'est le point. Le container du backend monte **un seul volume**, `../backend/target`, qui est la sortie de Maven — `mvn clean` l'efface et chaque build la réécrit. Une photo sur disque ne survivrait pas au déploiement.

3.2 Le tableau des migrations

#	Ce qu'elle fait	Rollback
V001	<code>access_attempts</code>	✓

#	Ce qu'elle fait	Rollback
V002	meal_entitlements	✓
V003	meal_entitlement_events (△ source est un CHECK manuel)	✓
V004	student_exit_permissions	✓
V005	system_users.permissoes (CSV, nullable)	✓
V006	index	— (un index se supprime seul)
V007	app_users.departamento	✓
V008	app_users.camera_person_id (UNIQUE)	—
V009	élargit le CHECK de denial_reason (caméra)	—
V010	app_users.posto_fixo_point_id	✓
V011	user_photos △ donnée irremplaçable	✓
V012	deux autorités sur les permissions de sortie	✓
V013	password_reset_requests	✓
V014	student_regimes + _events	✓
V015	élargit denial_reason (régime)	✓
V016	index access_logs (point, heure)	✓
V017	CHECK des énumérations de régime	✓
V018	index access_logs (heure)	✓
V019	index access_logs (user_id)	✓
V020	cantine_removals	✓
V021	meal_slots + _classes + _students	✓
V022	élargit denial_reason (MEAL_SLOT_NOT_CONFIGURED)	✓
V023	seed : l'affiche du mur + la reprise de class_schedules	— (les lignes partent avec R021)
V024	system_settings — naît vide	✓
V025	cdi_exclusions △ donnée sensible sur mineur	✓
V026	cdi_alert_events △ idem	✓
V027	licence_clock — le témoin d'horloge de la licence (ADR-006) △ à poser à la main AVANT de démarrer le backend : ddl-auto saurait créer la table, et c'est le problème — elle naîtrait sans son CHECK (id = 1) , qu'il ne corrigerait jamais	✓

△ **Chaque absence de rollback est justifiée dans** `deploy/migrations/README.md`, et `tests/migrations.test.js` échoue si une migration nouvelle arrive sans plan de retour, ou sans être nommée dans le README.

3.3 Les pièges de base à connaître avant de toucher au schéma

1. △ `ddl-auto=update` **CRÉE mais n'ALTÈRE jamais un CHECK**. Une valeur nouvelle dans un enum passe les tests (H2 recrée tout) et échoue **seulement sur la VM**. Ce piège a mordu trois fois : V009, V015, V022.
2. △ **Les CHECK sur** `access_attempts.denial_reason` **et sur** `meal_entitlement_events.source` **sont MANUELS**. Ajouter une valeur à l'enum Java **sans** élargir le CHECK produit un `INSERT` qui échoue en production.
3. △ **Les requêtes sur** `access_logs` **excluent les flags de répétition** avec `flag IS NULL OR flag <> 'POSTO_FIXO'`. Le `IS NULL` n'est **pas** un ornement : sans lui, `<>` vaut UNKNOWN pour NULL et **écarte toute la base en silence**.
4. △ **L'exclusion est ASYMÉTRIQUE** là où la requête regarde la paire entrée/sortie : on retire l'**entrée** marquée, **jamais la sortie**. Appris cher le 10/08 : avec une exclusion symétrique, une personne à poste fixe restait « dedans » jusqu'à minuit — sa sortie réelle, marquée, disparaissait avec le reste.

4. Les tests

Mesuré le **03/09/2026** (à froid : `cd backend && rm -rf target && mvn -o test`, puis `npm test`), sur `main` augmentée des trois chantiers de cette date : **backend 1055** (0 échec, exactement 2 `@Disabled`) sur **93** fichiers, **npm 889** sur **42** fichiers.

△ **Le critère n'est pas un total**. Le total monte à chaque livraison ; un nombre écrit dans un document vieillit en quelques jours. Le critère est **0 échec et exactement 2 @Disabled**. Un total inférieur à la dernière mesure veut dire qu'un test a été supprimé ; `Skipped ≠ 2` veut dire qu'une requête native a été désactivée.

△ **Mesurer depuis zéro** : `cd backend && rm -rf target && mvn -o test`. L'incrémental a déjà donné un `BUILD SUCCESS` faux.

△ Le `pom.xml` **doit contenir** `<include>**/*IT.java`. Sans lui, les tests d'intégration sont sautés **en silence**.

La philosophie — trois idées qui reviennent

a) Vérifier par mutation. Un test qui passe ne prouve rien tant qu'on n'a pas vu **échouer** la version cassée. Plusieurs gardes de ce dépôt ont été validés en remettant le défaut : `SettingsCatalogGuardTest` (en retirant une entrée du catalogue), `cdiCapaciteContract.test.js` (en remettant la double capacité), `aujourd'huiHeureLocale.test.js` (en remettant la forme UTC).

b) Vérifier les contrats par AST, pas par convention. Le frontend n'a pas de types. Quand un écran lit un champ que le DTO n'envoie jamais, JavaScript rend `undefined` sans erreur — **et toute autorisation ponctuelle s'affichait « Toujours »** au portail. `tests/exitPermissionContract.test.js` compare le DTO et l'écran par analyse syntaxique.

c) **⚠ Un garde doit DÉCLARER ses limites.** C'est l'idée la plus importante du chapitre, et elle vient d'un incident.

- `AccessLogRepositoryQueryGuardTest` tient par **chaîne de caractères** des requêtes que H2 n'exécute pas — il ne prouve pas qu'elles tournent, et il le dit.
- `ControllerAuthorizationGuardTest` porte trois listes **nommées** : `PUBLICO_POR_DESENHO`, `AUTENTICADO_POR_DECISAO`, `DIVIDA_CONHECIDA`. Cette dernière a **7 entrées et sa taille est assertée** : on ne peut pas y ajouter une dette en silence.
- `tests/aujourd'huiHeureLocale.test.js` porte une liste d'exceptions **nommées** (deux noms de fichiers téléchargés). En retirer une, c'est écrire la correction — jamais relâcher le test.

⚠ **Et le corollaire, payé le 27/08** : le garde d'autorisation lisait l'**identifiant** d'une constante au lieu de sa **valeur**, et déclarait non gardés 8 endpoints qui l'étaient. **Un garde qui accuse l'innocent apprend à être ignoré** — ce qui est pire qu'un garde absent.

Ce que les suites ne prouvent pas

⚠ **Aucune suite ne rend un composant React.** Les tests JS lisent le code source. Un écran qui ne s'affiche pas, une clé i18n crue, un débordement, un libellé tronqué : rien de tout ça n'est attrapé.

La preuve, pour une interface, c'est d'ouvrir l'écran. Le pilote E2E est dans `.claude/skills/run-magbo-app/driver.js` (Electron réel, connexion réelle, base réelle avec nettoyage). Le parcours manuel est dans `docs/frontend-smoke-checklist.md`.

Et deux requêtes natives PostgreSQL restent `@Disabled` (H2 ne les exécute pas) : leur vérification est **manuelle**, section 6-bis du même document.

5. Où regarder ensuite

Question	Document
Le schéma en détail	<code>docs/architecture/banco-de-dados.md</code>
Les endpoints	<code>docs/architecture/endpoints.md</code>
Les flux	<code>docs/architecture/fluxos.md</code>
Chaque migration, une par une	<code>deploy/migrations/README.md</code>
Les règles de code par domaine	<code>.claude/rules/backend.md</code> , <code>database.md</code> , <code>frontend.md</code> , <code>hikvision.md</code>
Pourquoi une décision est ce qu'elle est	<code>docs/architecture/decisoes/</code>
Les incidents qui ont produit ces règles	chapitre 8

Chapitre 8 — Les leçons du projet

Chaque section de ce chapitre est un défaut qui a réellement coûté quelque chose. Le format est toujours le même : **contexte** → **symptôme** → **cause** → **correction** → **règle**. La règle est ce qui reste ; le reste explique pourquoi elle mérite d'être suivie.

Chaque leçon est vérifiable : le commit, le fichier ou le test est cité.

I. ⚠ LES QUATRE DÉFAUTS D'HORLOGE

Le même piège, quatre fois, sous quatre déguisements. Il mérite d'ouvrir le chapitre parce qu'il est le seul à s'être répété — et parce que la règle qu'il donne est la plus transférable du projet.

⚠ LA RÈGLE GÉNÉRALE

Il y a toujours **DEUX heures**, et il faut savoir laquelle on écrit.

- L'heure de l'**événement** : quand la chose est arrivée dans le monde.
- L'heure du **traitement** : quand le programme l'a apprise.

Elles sont presque toujours identiques — jusqu'au jour où une file d'attente se vide, où un conteneur démarre dans un autre fuseau, ou où il est 21 h à Rio. **Ce jour-là, celui qui n'a pas choisi a choisi la mauvaise.**

Le corollaire : un défaut d'horloge ne se voit jamais pendant une démonstration. Il attend le soir, la panne, la reprise. C'est pourquoi il faut le choisir par écrit, pas le découvrir.

1. L'heure de réception au lieu de l'heure de l'événement (03/08)

Contexte. Le backend écrivait `LocalDateTime.now()` au moment où la requête arrivait.

Symptôme. Les rapports affichaient des **durées moyennes négatives**, et des élèves à des heures et des points impossibles.

Cause. Un terminal a vidé sa file hors ligne : **33 événements en deux minutes, à 14h51**, correspondant à des passages de plusieurs heures plus tôt. Les 33 sont entrés en base comme s'ils avaient eu lieu à 14h51. Mettre en file et renvoyer est le comportement **normal** des MinMoe quand la destination tombe — observé deux fois sur le banc. L'heure de réception n'a donc jamais été une approximation sûre.

Correction. `EventTimeResolver` (commit `8d78f41`) lit le `dateTime` du payload. Trois gardes seulement font retomber sur l'heure de réception : absent ou illisible, plus de 5 minutes dans le futur, plus de 30 jours dans le passé — et **chaque repli laisse une ligne INFO** avec l'IP et le motif.

Règle. *Quand un événement porte sa propre heure, c'est elle qui compte. Et tout repli doit être audible.*

⚠ **Ce qui n'a PAS changé, et c'est une dette assumée :** les **règles** (fenêtre de la cantine, dédup de repas, autorisation de sortie) restent évaluées à l'heure de la **décision**. Une file rejouée à 14h51 écrit les bonnes heures, mais a été jugée à 14h51.

2. Le régime jugé à `now` au lieu de l'heure de la passage

Contexte. Le régime de sortie décrit si un élève avait le droit de sortir.

Symptôme. Le verdict passait **vert toute la journée**. Le test ne l'a attrapé que parce que la suite a tourné à 18h45.

Cause. La règle était évaluée contre `now`. Une sortie de 10 h, traitée à 18 h, tombait dans le degré « fin de journée — sortie normale », et l'alerte que la Vie Scolaire devait voir **n'a jamais existé**.

Correction. Le régime est la seule règle jugée à l'heure de la **passage** — exception consciente, et elle a une raison : cette règle **ne refuse jamais**, elle décrit. La trave

`RegimeGateWiringTest#regimeUsaAHoraDaPassagem` capture **l'argument**, pas le verdict : un test qui échoue selon l'heure à laquelle on le lance ne prouve rien.

Règle. *Une règle qui DÉCRIT se juge à l'heure des faits. Une règle qui REFUSE se juge à l'heure de la décision, pour qu'une file ne puisse pas changer un refus rétroactivement.*

3. Le conteneur en UTC — trois heures dans le futur

Contexte. L'image `eclipse-temurin:17-jre-alpine` démarre en **UTC**.

Symptôme. Le système horodatait **trois heures dans le futur**. Mesuré en direct : 20h27 écrit pour 17h27 réelles.

Cause. La JVM adopte le fuseau du conteneur. Tout `LocalDateTime.now()` du backend rendait de l'heure UTC — pendant que `access_logs.timestamp` était écrit en `America/Sao_Paulo` par `EventTimeResolver`. **Deux moitiés du même système, deux fuseaux.**

Correction. `TZ: America/Sao_Paulo` sur **les deux** conteneurs de `deploy/docker-compose.yml`, et une `Clock` explicite dans les services concernés (`Clock.system(EventTimeResolver.ZONA_ESCOLA)`).

Règle. *Le fuseau d'un conteneur fait partie de sa configuration, pas de son environnement. Une image qui démarre en UTC le fera sur toutes les machines, y compris celle qu'on n'a pas testée.*

4. « Aujourd'hui » calculé en UTC dans le frontend (28/08)

Contexte. Une vingtaine d'endroits faisaient `new Date().toISOString().slice(0, 10)` pour obtenir la date du jour.

Symptôme. Aucun — **avant 21 h**.

Cause. `toISOString()` est en **UTC**. Rio est à UTC−3 : à partir de 21 h, l'expression rend **demain**. Le compteur « Mouvements aujourd'hui » tombait à 0, les rapports interrogeaient un jour qui n'existait pas encore, le Moniteur demandait les passages du lendemain, la sauvegarde automatique du CDI croyait

n'avoir pas encore tourné, et un régime nouveau était daté de demain.

Correction. `dayKey(date)` (`js/utils/helpers.js`), composantes locales — **la fonction existait déjà**, avec un commentaire mettant en garde contre exactement cette forme. Le garde `tests/aujourd'huiHeureLocale.test.js` parcourt `js/` et échoue sur toute ligne qui nomme un jour en UTC ; deux noms de fichiers téléchargés sont des exceptions **nommées**.

Règle. `toISOString()` ne donne jamais « aujourd'hui ». Il donne « aujourd'hui à Greenwich ».

II. LES AUTRES LEÇONS

5. `ddl-auto` crée mais n'altère jamais un CHECK

Contexte. Le schéma est géré par `ddl-auto=update`, y compris sur la VM.

Symptôme. Un `INSERT` qui échoue **seulement en production**, des semaines après la livraison. Les tests sont verts, le PC va bien.

Cause. Hibernate génère le CHECK à la création de la table. Sur une table existante, `ddl-auto=update` ajoute des colonnes mais **n'altère jamais** une contrainte. Une valeur nouvelle dans un enum Java passe donc partout — H2 recrée tout à chaque suite, le PC ne bouge pas — et échoue à la VM.

Correction. Une migration qui élargit le CHECK, appliquée **à la main, avant** de monter le backend : V009, V015, V022. Et `tests/migrations.test.js` échoue quand le CHECK de `denial_reason` oublie une valeur de l'enum.

Règle. *Ce qui n'existe que sur la machine de production doit être créé par une migration, pas espéré d'un outil de développement.*

6. `psql` sort avec le code 0 quand le SQL échoue

Contexte. Les migrations sont appliquées par une boucle shell.

Symptôme. Le script annonce le succès. Le backend démarre. Le défaut se découvre en production.

Cause. Sans `ON_ERROR_STOP=1`, `psql` continue après l'erreur et sort avec le code 0. Le `|| echo ÉCHEC` de la boucle ne se déclenche jamais.

Correction. `-v ON_ERROR_STOP=1` dans **toutes** les commandes de migration, documenté dans `deploy/migrations/README.md` et répété dans le handoff.

Règle. *Un outil qui rend 0 en cas d'échec transforme une boucle de vérification en théâtre. Vérifier le comportement de sortie AVANT de bâtir une boucle dessus.*

7. `CREATE TABLE IF NOT EXISTS` ignore la FORME de la table

Contexte. Les migrations sont idempotentes, pour pouvoir être rejouées.

Symptôme. La migration passe, la table existe — **sans ses contraintes**.

Cause. Si le backend démarre **avant** la migration, Hibernate crée la table avec les mêmes colonnes mais **sans les CHECK**. `CREATE TABLE IF NOT EXISTS` voit une table du bon nom et **ne fait rien, sans rien dire**.

Correction. Une clause de garde dans V021, V025 et V026 qui **lève une exception explicite** quand la table préexiste avec la mauvaise forme. Et, pour V026, un second bloc idempotent qui pose le CHECK s'il manque — quel que soit celui qui a créé la table.

Règle. `IF NOT EXISTS` teste un nom, pas une forme. Quand la forme compte, la vérifier explicitement.

8. Le multipart tronqué

Contexte. Les terminaux envoient du `multipart/form-data` avec une part JSON et une part image.

Symptôme. Des événements réels ignorés, avec un `200` et une ligne de log disant « écarté ».

Cause. Le format réel des appareils ne correspond pas à la documentation : en-têtes de parts incomplets, limites atypiques, et un champ `score` **emballé** (`{"value": 52}` au lieu d'un nombre) qui faisait échouer le parsing sur **tous** les événements de caméra.

Correction. `MultipartTolerante` et un `parsePayload` tolérant, écrits contre des captures réelles, avec des fixtures (`modelData` remplacé par `<<modelo-biometrico-removido>>` — un gabarit biométrique ne se committe pas).

Règle. Un parseur qui parle à du matériel se écrit contre le matériel, pas contre sa fiche technique. Et un rejet silencieux est pire qu'une erreur.

9. La course avec `@Transactional`

Contexte. Un service annoté `@Transactional` appelé depuis la même classe.

Symptôme. L'annotation ne s'applique pas — l'appel interne court-circuite le proxy Spring.

Correction. Passer par un autre bean, ou expliciter la propagation. Le cas qui compte dans ce dépôt est l'inverse et il est délibéré : voir la leçon 12.

Règle. Une annotation Spring décrit ce qui se passe quand on entre par la porte. Un appel interne n'entre pas par la porte.

10. Les hooks React — deux pièges, tous deux visibles à l'écran seulement

a) Le `return` anticipé avant les hooks.

Symptôme. L'écran entier casse avec « rendered fewer hooks than expected », mais **seulement** le jour où la condition change en cours de vie.

Cause. `if (!pode) return null;` placé **avant** un `useRef` ou un `useEffect`. React compte les hooks par leur **ordre d'appel**.

Correction. La sortie anticipée vient **après tous les hooks**.

b) Un composant défini à l'intérieur de son parent.

Symptôme. Les photos clignotaient toutes les 3 secondes dans le Moniteur Cantine : photo → initiales → photo.

Cause. Un composant défini dans le corps d'un parent reçoit un **type React neuf à chaque rendu**. React démonte tout le sous-arbre, le composant photo repart de son état initial, peint les initiales, et ne rend l'image qu'au microtask suivant. **Cinq occurrences dans le seul** `CantineMonitor.js`.

Mesuré : 30 nœuds `<img>` créés en 12 secondes → **0** après correction.

Correction. Les composants remontent au scope du module ; les dépendances descendent par props. Des clés stables (`key={ev.userId}`, jamais l'index).

Règle. *Un composant défini dans un parent est un composant remonté à chaque rendu. Le symptôme est visuel, donc invisible pour une suite qui ne rend rien.*

11. Un garde qui lit un identifiant au lieu de sa valeur

Contexte. `ControllerAuthorizationGuardTest` extrait les `@PreAuthorize` du code source pour dresser l'inventaire des endpoints gardés.

Symptôme. Le garde déclarait **non gardés 8 endpoints qui l'étaient**.

Cause. Certains endpoints portent `@PreAuthorize(ESCRITA)` — une **constante**. Le parseur lisait l'identifiant `ESCRITA` au lieu de résoudre sa valeur, et ne reconnaissait pas une garde.

Correction. `resolverConstante()` résout la constante avant de juger.

⚠ La règle, et elle vaut au-delà des tests

Un garde qui accuse l'innocent apprend à être ignoré.

Un faux positif ne coûte pas une correction inutile : il coûte la confiance dans le garde. La fois suivante, quelqu'un le contournera — et ce jour-là il aura peut-être raison.

12. L'écriture observationnelle en `REQUIRES_NEW`

Contexte. Plusieurs registres existent pour permettre de rendre compte plus tard : `access_attempts`, les fermetures automatiques, le registre des alertes du CDI (V026).

Symptôme potentiel. Un registre qui échoue fait échouer la transaction qui l'entoure — et emporte avec lui l'enregistrement d'un **passage réel**.

Correction. Ces écritures se font en `Propagation.REQUIRES_NEW`, et l'appelant **attrape**.

`CdiAlertServiceTest` vérifie l'annotation elle-même : aujourd'hui l'appelant est un endpoint dédié, mais le jour où le webhook appellera, cette annotation est ce qui empêchera un registre en panne d'emporter un `access_log`.

Règle. *Un registre de soutien ne doit jamais pouvoir faire tomber ce qu'il observe.*

13. ⚠ « Je n'ai pas vu » n'est pas « il n'était pas là »

C'est la leçon la plus importante du projet, et la seule qui soit d'abord une leçon d'écriture.

Contexte. Le système produit en permanence des états qui veulent dire « je ne sais pas ».

Symptôme. Une donnée manquante affichée comme un refus fait accuser quelqu'un à la place d'une case vide.

Les cas :

État	Ce que ça veut dire	Ce que ça ne veut PAS dire
PENDING (droit au repas)	la case n'a jamais été remplie — l'état de 923 élèves le jour 1	« pas le droit »
INCONNU (régime)	aucun régime enregistré	« pas autorisé à sortir »
REGIME_TO_VERIFY	ça dépend d'une heure de cours que le MAGBO n'a pas	une objection
MEAL_SLOT_NOT_CONFIGURED	un trou dans le planning	un refus
« Aucun passage vu aujourd'hui »	le système n'a rien vu	« absent »
Pas de ligne dans cdi_alert_events	l'écran du CDI était fermé	« il n'y a pas eu d'alerte »
Un compteur à -	le cache n'est pas chargé	« zéro »

Correction. Chacun de ces états a **sa propre couleur, son propre mot et sa propre action** à l'écran.

INCONNU est ambre et non gris, parce que le gris clair est la couleur de ce que l'opérateur a appris à ignorer. Un compteur qui ne sait pas affiche , jamais .

Règle. *Une interface doit distinguer « non », « oui » et « je ne sais pas ». Fondre le troisième dans le premier, c'est mentir sur des gens.*

14. Les dates dans Excel

Contexte. Les imports (droits repas, régimes, HikCentral) arrivent en .

Symptôme. Des matricules qui ne correspondent à personne. Des dates décalées.

Cause. ⚠ Les identifiants Pronote ont des zéros à gauche (). Excel les traite comme des nombres et **mange le zéro**. Et les dates deviennent des numéros de série.

Correction. Tout est lu **comme du TEXTE** (`raw: false`), et la comparaison retire les zéros des deux côtés. Le lecteur du HikCentral commence à la **ligne 9** (les 8 premières sont des instructions du HCP) et associe les colonnes **par nom**, pas par position — le HCP réordonne entre versions, et une position fixe casse en silence.

Règle. *Un identifiant qui commence par zéro n'est pas un nombre. Le traiter comme tel est une perte de données silencieuse.*

15. `env_file` et le compose

Contexte. La configuration de la VM vit dans `deploy/.env`.

Symptôme. Le backend démarre et se connecte à une base **vide** ou à la mauvaise, sans erreur explicite.

Cause. Les valeurs de repli du profil `prod` pointent ailleurs. Une variable absente n'est pas une erreur : c'est un repli silencieux.

Correction. Les variables qui ne peuvent pas avoir de repli utilisent la syntaxe de variable **obligatoire** (`${VAR:?message}`) : le compose refuse de démarrer et dit laquelle manque.

`ProdSecurityStartupCheck` avertit au démarrage pour le mot de passe de développement, le JWT de développement et le jeton de webhook absent.

Règle. *Une configuration manquante doit faire échouer bruyamment. Un repli silencieux vers « une autre base » est le pire des comportements.*

16. Deux verrous d'outillage, rapportés par Sam

⚠ Ces deux-là ne sont pas reproductibles depuis le dépôt. Ils sont écrits parce qu'ils feront perdre une heure à la personne suivante.

a) `build:portable` bloqué par un handle sur `app.asar`. La construction échoue en disant que le fichier est utilisé, **alors qu'aucun processus n'est visible**. Ni le gestionnaire de tâches ni la fermeture de l'application ne le libèrent. **Seul un redémarrage du poste débloque.**

b) GitHub refuse une branche rebasée quand `main` porte les commits de merge. Le merge local passe sans conflit ; la plateforme, elle, calcule des bases multiples et refuse. (*Symptôme rapporté ; la manœuvre exacte de contournement n'est pas notée.*) **[À COMPLÉTER PAR SAM] : quelle manœuvre a débloqué ce cas ?**

17. Ce que ces leçons ont en commun

En les relisant, trois familles se dégagent — et elles suffisent à prédire le prochain défaut :

1. **Le silence.** `psql` qui rend 0, `IF NOT EXISTS` qui ne fait rien, un CDN qui ne charge pas, une variable qui retombe sur un repli, un multipart écarté, un `<>` qui écarte les NULL. ⚠ **Presque tous les défauts coûteux de ce projet étaient silencieux.** Le corollaire pratique : quand vous écrivez du code qui peut échouer, demandez-vous **qui l'apprendra, et comment.**
1. **Les deux vérités.** Deux capacités sur un écran, deux couches HTTP, deux ordres de règles, deux défauts pour le même réglage, le miroir `constants.js` / `AreaMapping`. ⚠ **Une valeur qui vit à deux endroits divergera** — la seule question est quand.
1. **La confusion entre l'absence et le refus.** La leçon 13, sous toutes ses formes.

18. Où regarder ensuite

Question	Document
Le détail de chaque nuit et de chaque correction	<code>docs/operacional/nuit-26-27-08-rapport.md</code> , <code>nuit-27-28-08-rapport.md</code>
Les règles de code qui en sont sorties	<code>.claude/rules/*.md</code>
Les décisions structurelles	<code>docs/architecture/decisoes/</code>
L'état opérationnel du jour	<code>docs/operacional/handoff.md</code>
Ce qui reste ouvert	chapitre 9

Chapitre 9 — Ce qui reste

Liste datée et priorisée au **29/08/2026**. Chaque entrée dit ce que c'est, pourquoi ça compte, et ce qu'il faudrait faire.

L'ordre est justifié : d'abord ce qui empêche le système de faire son travail, puis ce qui touche des données sur des personnes, puis ce qui est de la dette technique, puis ce qui est en attente d'une décision.

PRIORITÉ 1 — le système ne fait pas son travail

1.1 Le portail ne reconnaît presque plus personne

Depuis le 25/08 : ~46 élèves distincts par jour, contre ~500 jusqu'au 24/08. Le personnel n'est pas touché.

C'est **le** problème du système aujourd'hui. Tout le reste de cette liste peut attendre ; pas celui-là.

Les chiffres complets et la note historique (la caméra `.166` en panne jusqu'au 24/08, et pourquoi les ~950 entrées d'avant n'étaient pas 950 personnes) sont **en tête de** `docs/operacional/handoff.md`. Ce qui a été écarté avec preuve, les cinq requêtes SQL à lancer sur la VM, les `grep` de logs et les actions HikCentral sont dans `docs/operacional/diagnostic-portaria-2026-08-27.md`.

Ce qu'il faut faire, dans l'ordre : lancer les cinq requêtes du diagnostic sur la VM avant de toucher à quoi que ce soit. La cause **n'est pas prouvée**.

 **La piste n° 1 s'est renforcée le 31/08** : les imports de photos des 25 et 26/08 sont passés **par HikCentral**, donc par les **bibliothèques faciales des caméras** — pas par la table `user_photos` du MAGBO, qui n'a aucun effet sur la reconnaissance. Le lien cesse d'être une coïncidence de dates : il y a un mécanisme. Les quatre vérifications côté HikCentral (la bibliothèque existe-t-elle encore et avec combien de personnes · les images ont-elles été remplacées · le `certificateNumber` a-t-il changé de format · le « Apply to Device » a-t-il été fait) sont en tête du handoff.

Effort : une matinée de mesure. La correction dépend de ce qu'elle montre.

PRIORITÉ 2 — des données sur des personnes

2.1 Les sept endpoints non gardés

Ils sont **nommés** dans `ControllerAuthorizationGuardTest`, sous la liste `DIVIDA_CONHECIDA`, dont la taille est assertée — on ne peut pas en ajouter en silence.

Endpoint	Ce qu'il expose
<code>AccessController.getLogsByPoint</code>	les passages d'un point : qui, à quelle heure
<code>AccessController.getAllRecentLogs</code>	les passages de tous les points
⚠ <code>AccessController.registerAccess</code>	une ÉCRITURE : enregistrement manuel d'un passage
<code>UserController.searchStudents</code>	recherche d'élèves par nom
<code>UserController.getUserById</code>	la fiche d'une personne
<code>UserController.searchUsers</code>	recherche de personnes, tous types
<code>UserController.listActiveUsers</code>	la liste des personnes actives — la source du cache de tous les écrans

Tous tombent dans `anyRequest().authenticated()` : il faut être connecté, mais n'importe quel compte suffit.

⚠ `registerAccess` est le plus grave : c'est une écriture. Le `created_by_user` dit **qui** a fait, mais n'empêche personne.

Ce qu'il faut faire : ⚠ les garder cassera des écrans. `listActiveUsers` alimente le cache utilisé partout ; `getLogsByPoint` alimente le SectorView et le Journal. C'est un chantier avec ses propres preuves à l'écran, pas une retouche.

Effort : une nuit de travail avec vérification écran par écran.

2.2 Le registre des alertes du CDI n'a pas de filtre par personne

`GET /api/admin/cdi/alertes` rend les 500 dernières lignes. « Combien de fois cet enfant a-t-il été signalé » se lit à l'œil.

L'index `idx_cdi_alert_events_user` a été créé (V026) **pour une requête qu'aucun code n'exécute**. C'est littéralement la question qu'une famille pose.

Effort : un paramètre `?userId=` et un champ de filtre. Quelques heures.

2.3 L'écran des exclusions n'est pas sûr à laisser ouvert

Il affiche des noms complets, des classes, **des motifs en clair** et le login de l'auteur. La seule protection est une phrase d'avertissement en tête. L'écran du CDI, lui, a un verrou `Alt+L`.

Ce qu'il faudrait : masquage des motifs derrière un clic, et retour automatique au tableau de bord après quelques minutes d'inactivité.

2.4 Une exclusion de classe suit la CLASSE, pas les élèves

Un élève muté en 6E1 en octobre déclenche l'alerte d'une mesure décidée en septembre pour d'autres ; un élève qui quitte la classe cesse d'être signalé.

C'est **assumé et écrit** dans V025, dans le modèle et sur l'écran de création. Figer la composition demanderait des lignes filles par élève — décision de modèle, pas effet de bord.

PRIORITÉ 3 — dette technique connue

3.1 Les deux onglets de l'engrenage jamais relus

Le balayage du 28/08 a relu 17 écrans. **Deux familles n'ont été relues par personne** — la limite de session a coupé les agents : *Importer Excel, HikCentral, Personnels, Importer les personnels, Photos, Enregistrement manuel, Généraux*.

Les captures existent (`%TEMP%/magbo-balayage/`), aucun relecteur n'est passé. **Ce ne sont pas des écrans propres : ce sont des écrans non examinés.**

3.2 Les quatorze réserves du rapport du 26–27/08

Elles sont listées au §5 de `docs/operacional/nuit-26-27-08-rapport.md`. Les trois qui comptent le plus :

1. `MEAL_SLOT_NOT_CONFIGURED` apparaît dans le panneau rouge « tentatives refusées », avec un **bip**. Ce n'est pas un refus, c'est « je ne sais pas ». `REGIME_TO_VERIFY` et `REGIME_UNKNOWN` ont chacun leur couleur pour cette raison exacte ; celui-ci tombe dans la couleur par défaut.
2. `PpmsView` ne mentionne pas les classes dispensées. Si la dispense est un jour activée, le décompte d'évacuation est amputé **sans le dire**.
3. Le « LRU » de `photoCache` est en réalité un **FIFO**. Sans conséquence au volume actuel ; c'est le nom qui ment, pas le code.

3.3 `duplicate-meal` en OBSERVATION gonfle le compte

La politique est en `OBSERVATION` : chaque deuxième lecture d'une même personne produit une ligne dans `access_attempts`. Ces lignes se mélangent aux vrais refus dans les compteurs.

Décision à prendre : passer en `DENY`, ou séparer les compteurs.

3.4 Le contrôle au démarrage de V022

Rien ne vérifie au démarrage que le CHECK de `denial_reason` couvre toutes les valeurs de l'enum. `tests/migrations.test.js` le vérifie **au moment des tests**, contre le fichier SQL — pas contre la base réelle.

Ce qu'il faudrait : un contrôle au démarrage, sur le modèle de `ProdSecurityStartupCheck`, qui compare l'enum au CHECK effectivement présent. C'est le troisième incident de cette famille (V009, V015, V022) — le prochain est prévisible.

3.5 La divergence `class_schedules` / `meal_slots`

Depuis la V021, la cantine lit `meal_slots` et **plus** `class_schedules` (ADR sur les créneaux). Les deux tables coexistent, et rien ne garantit qu'elles racontent la même chose.

Décision à prendre : retirer `class_schedules`, ou écrire noir sur blanc qu'elle n'est plus lue par la cantine et par qui elle l'est encore.

3.6 Deux fichiers portent le numéro ADR-005

C'est un fait, pas une hypothèse :

Fichier	Décision	Date	Ajouté au dépôt
<code>ADR-005-totvs-rastreabilidade-no-dono-do-dado.md</code>	TOTVS : la traçabilité reste chez le propriétaire de la donnée	14/08/2026	14/08 (<code>b388275</code>)
<code>ADR-005-creneaux-cantine.md</code>	Le planning de cantine devient une configuration	26/08/2026	25/08 (<code>7567a75</code>)

Proposition — je ne renumérote pas moi-même :

TOTVS garde `ADR-005` (décision du 14/08, entrée dans le dépôt le 14/08 — c'est elle qui a pris le numéro en premier). **Les créneaux deviennent `ADR-006`** (décision du 26/08).

C'est le sens de lecture d'un journal de décisions : le numéro suit l'ordre chronologique, et renumérotter la plus ancienne casserait les références déjà écrites ailleurs.

Ce qu'il faut faire : renommer le fichier des créneaux, corriger son titre et son en-tête, et vérifier les renvois — `grep -rn "ADR-005" docs/ .claude/ CLAUDE.md` avant et après.

PRIORITÉ 4 — en attente d'une décision ou d'un tiers

4.1 Les horaires réels de la maternelle et de l'élémentaire

Mesuré le 26/08 : service réel de 11h54 à 12h37. Cela **contredit** les horaires supposés.

 **Aucun horaire n'a été semé pour ces classes, et c'est délibéré.** Inventer un créneau aurait produit de fausses alertes dès le premier jour. Elles tombent donc dans `MEAL_SLOT_NOT_CONFIGURED` — le système dit « je ne sais pas », ce qui est vrai.

Il faut la décision de la Vie Scolaire, pas une mesure supplémentaire.

4.2 Six classes de collège du mercredi 13h à confirmer

1E1, 1E2, 2E1, 2E2, 3E1, 3E2. [À COMPLÉTER PAR SAM / Vie Scolaire] : ces six classes sont-elles bien au second service du mercredi ?

4.3 5E3 et 3E3 : sur l'affiche, aucun élève en base

Elles figurent au mur et sont chargées dans le planning ; **aucun élève ne leur est rattaché**. 5E3 est même absente de `class_schedules`.

Elles ne changent le verdict de personne. Elles signalent soit un code de classe qui a changé, soit une classe qui n'existe plus. Le détail est dans `docs/operacional/controle-affiche-cantine.md`, section A.

Il faut une réponse de la Vie Scolaire : ces classes existent-elles encore, et sous quel code ?

4.4 La liste DAF — ✓ la demande n'a jamais été lancée

(Répondu par Sam le 31/08/2026.) Elle n'a été formalisée auprès de personne. **Ce n'est donc pas une relance, c'est une démarche à initier.**

△ **Ce que cela laisse en place, et pourquoi l'ordre compte** : en attendant, **995 personnes** (874 élèves + 121 personnels) ont un droit au repas accordé **en bloc**, « temporairement », sans échéance ni processus de sortie. **Ne retirez pas ce droit avant d'avoir la liste** — l'inverse affamerait ceux qui y ont droit pour punir ceux qui n'y ont pas droit. Requête de contrôle au § 8.2.8 du handoff.

4.5 Le terminal .10 non enregistré au HikCentral — ✓ un ticket existe

Erreur `SYS[904]`, numéro de série en conflit. (Répondu par Sam le 31/08.) **Un ticket est ouvert chez le fournisseur** — le revendeur qui a livré les terminaux.

[À COMPLÉTER — la référence du ticket et le nom du contact n'ont pas pu être retrouvés.] Où **chercher** : la messagerie de Sam, ou le service informatique de l'établissement, qui a traité la commande. △ **Sans la référence, rouvrir une demande revient au même** : l'erreur `SYS[904]` et le numéro de série suffisent à décrire le cas.

4.6 Le terminal .14 en Wi-Fi — ✓ définitif, ce n'est pas une action ouverte

(Répondu par Sam le 31/08.) **L'emplacement ne permet pas de tirer un câble**. C'est une **contrainte permanente**, pas une tâche en attente : ne la comptez plus comme du travail à faire.

C'est donc lui qui perdra des paquets et videra une file d'un coup — exactement le scénario de la première leçon d'horloge (chapitre 8). Le système sait déjà absorber une file rejouée (`EventTimeResolver` écrit l'heure de l'événement). Ce qui reste est la dette « les règles sont jugées à l'heure de la décision ».

4.7 L'interface servie par la VM / la mise à jour automatique

La conception est faite, la décision est reportée. Aujourd'hui chaque poste porte sa propre copie du frontend, et une mise à jour se distribue à pied.

Décision à prendre : servir l'interface depuis la VM (une seule version, mise à jour instantanée, mais dépendance au réseau), ou garder le portable (autonome hors ligne, mais distribution manuelle).

△ Le portable a une propriété que la VM n'aurait pas : **il fonctionne quand le réseau tombe**. Ce n'est pas un détail dans une école.

4.8 L'e-mail à Fabiano et le PDF du guide — ✓ les deux sont partis

(Répondu par Sam le 31/08.) L'e-mail au service informatique a été envoyé et le PDF du guide d'installation a été remis. **Rien à relancer**. La source reste `docs/operacional/guide-installation-postes.md` : c'est elle qu'il faut mettre à jour puis réexporter si la procédure change.

△ **Ce qui reste ouvert, c'est le contact lui-même**. Sam ne se souvient ni du nom complet de Fabiano, ni de son e-mail, ni de l'état des réservations DHCP. **[À COMPLÉTER — Sam n'a plus l'information.] À qui demander** : le secrétariat ou la direction connaissent le service informatique. **Ce qu'il faut lui demander** : « les adresses IP des six terminaux et de la VM `192.168.1.253` sont-elles réservées en DHCP, ou peuvent-elles encore changer ? » **N'attendez pas la réponse pour vous protéger** : la méthode empirique du handoff (§ 2.1) donne le risque réel en une requête, sans contact.

5. Ce qui n'est PAS sur cette liste, et pourquoi

Trois choses reviennent régulièrement et **ne doivent pas être « corrigées »** :

1. **Le système n'ouvre pas les portes**. Ce n'est pas une lacune (ADR-003).
2. **Les règles sont évaluées à l'heure de la décision**. C'est une dette **assumée** : la changer permettrait à une file hors ligne de transformer un refus en autorisation rétroactivement.
3. **Les deux couches HTTP du frontend**. Dette connue. **Ne pas en créer une troisième** ; consolider est un chantier à part.

Chacune est gelée par un test ou par un ADR. Les « corriger » sans décision ferait échouer une suite qui protège un choix.

6. Où regarder ensuite

Question	Document
Le défaut du portail, avec ses chiffres	<code>docs/operacional/handoff.md</code> (en tête)
Ce qui a été écarté, ce qui reste à mesurer	<code>docs/operacional/diagnostic-portaria-2026-08-27.md</code>
Les 14 réserves, en détail	<code>docs/operacional/nuit-26-27-08-rapport.md</code> §5
Le balayage : corrigé vs listé	<code>docs/operacional/nuit-27-28-08-rapport.md</code> §6
Les classes de l'affiche contre la base	<code>docs/operacional/controle-affiche-cantine.md</code>
Les demandes au service informatique	<code>docs/testing/pedidos-fabiano-si.md</code>
Les questions pour Sam, toutes ensemble	<code>docs/operacional/handoff.md</code> §11

COLOPHON

Système	MAGBO Access Control, version 2.1.0
Livre arrêté le	03/09/2026
Dernier commit	<code>5a7fca2</code> — branche <code>main</code>
Contenu	9 chapitres, plus le sommaire
Établissement	Lycée Molière, Rio de Janeiro
Auteur	Sammy K. Magbo

Composé en Cambria pour le texte, Segoe UI pour les titres et Consolas pour le code — trois polices du système, aucune à installer. Le livre est un fichier HTML autonome : pas de script, pas de ressource réseau, aucune bibliothèque PDF. C'est le navigateur qui imprime.

Pour régénérer : `node scripts/build-livre.js` reconstruit le HTML depuis `docs/livre/*.md` ; `node scripts/paginer-livre.js` mesure les vrais numéros de page dans le PDF, les réinjecte dans la table des matières et produit `docs/livre/livre-complet.pdf`.

Pour l'imprimeur : A4, recto-verso, reliure côté long. La marge intérieure fait 26 mm et l'extérieure 20 mm ; **ne pas redimensionner** (« Taille réelle » / « 100 % », jamais « Ajuster à la page »), sinon les marges de reliure ne sont plus celles qui ont été calculées.